



The Mathematics of **AI** Neural Networks

Explained with Python & PyTorch

Linear Algebra

Calculus

Probability

Gradient Descent

Backpropagation

CNN

Transformer

Large Language Models

9 chapters + 6 appendices · 1,092 math formulas · 324 code examples

From Neurons to Transformers — Understand Deep Learning Through Math

Author: xiefujin · Email: 490021684@qq.com

Generated from Markdown source · 2025

Table of Contents

- Foreword
- Chapter 1: The Idea of Neural Networks
- Chapter 2: Mathematical Foundations of Neural Networks
- Chapter 3: PyTorch Basics -- Tensor & Autograd
- Chapter 4: Training & Optimization in Neural Networks
- Chapter 5 Backpropagation -- The Heart of Neural Networks
- Chapter 6: Convolutional Neural Networks
- Chapter 7: Training Techniques -- Optimizers, Regularization & Loss Functions
- Chapter 8: Modern Architectures -- From ResNet to Transformer
- Chapter 9: Large Language Models -- Training, Sampling & Inference
- **Appendices**
 - Appendix A: Mathematical Review
 - Appendix B: PyTorch API Reference
 - Appendix C: Notation Conventions
 - Appendix D: Common Functions Summary
 - Appendix E: Exercise Answers
 - Appendix F: Recommended Reading

Foreword

Author: xiefujin · **Email:** 490021684@qq.com

Welcome! This book is for those who want to **truly understand** the mathematics behind neural networks — without being intimidated by formal derivations.

Starting from fundamental mathematical concepts, reinforced with Python + PyTorch code, this book lets you **see the math in action**.

> © xiefujin · **Contact:** 490021684@qq.com · **Licensed under**
CC BY-NC-SA 4.0



Chapter Learning Objectives

- [] Understand the positioning and features of this book
- [] Know how to use this book effectively
- [] Set up your Python development environment
- [] Grasp the core philosophy: intuition first, formula next, code last



0-1 Why This Book?

The "Gap" in Existing Books

Most deep learning books have a **gap between math and code**:

Book Type	Strengths	Weaknesses
Pure Math (e.g., <i>The Math of Deep Learning</i>)	Rigorous mathematical derivations	Uses Excel, cannot handle large-scale data, no automatic gradient computation
Engineering-Focused (e.g., <i>Dive into Deep Learning</i>)	Rich hands-on code	Math principles not deep enough; readers know "how" but not "why"

Book Type	Strengths	Weaknesses
Academic Textbooks (e.g., Goodfellow's <i>Deep Learning</i>)	Complete theoretical system	Math barrier too high; not suitable for self-study

The Bridge This Book Builds

This book builds a **bridge between mathematical intuition and code verification** — aiming not for mastery of derivation, but for **true understanding**.

Core Positioning: Starting from fundamental math concepts, we'll **understand the mathematical principles behind deep learning** — using Python + PyTorch code to verify and visualize each concept, rather than doing formal mathematical derivations.

💡 0–2 Book Features

Feature	Description
Understanding First	Every math principle is explained with intuition and visualization before code
Visualization	Matplotlib dynamically shows gradient descent, backpropagation in action
PyTorch Native	Gradual transition from manual NumPy to PyTorch autograd
Transparent Derivation	No hidden steps; every weight change at every layer is visible
Comparison Experiments	Every key component has manual vs. automatic computation comparison

Standard Chapter Rhythm

Each chapter follows a consistent "Concept → Visualization → Formula → Code" rhythm:

1. **Conceptual Intuition** — a 30-word "what it's like"
2. **Visualization** — let your eyes "see" the math first
3. **Math Formula** — annotated with the meaning of each symbol
4. **Code Verification** — "Look, this is how the math works in practice"
5. **Core Insight** — one sentence capturing the essence of this section

💡 **Tip:** If you're a beginner, follow the chapters in order. If you have prior knowledge, you can jump directly to the code sections and refer back to the formulas when you need details.

💡 0–3 Environment Setup

Hardware Requirements

All code in this book runs on an ordinary laptop:

- **Any computer that can run Python** (no GPU required)
- **8GB+ RAM recommended** (4GB works but training is slower)
- **Disk space:** ~2GB (including PyTorch, MNIST dataset, etc.)

Even without a GPU, the code in this book runs fast enough — we won't be training ImageNet-scale models. All examples are small-scale mathematical verifications.

Software Requirements

```
# Recommended: use conda or venv for a virtual environment
conda create -n dl_math python=3.10
conda activate dl_math

# Install core dependencies
pip install torch torchvision # PyTorch 2.0+
pip install numpy matplotlib # Scientific computing + visualization
pip install jupyter          # (Optional) interactive notebooks
```

Version Requirements

Software	Minimum	Recommended
Python	3.8	3.10+
PyTorch	1.13	2.0+
NumPy	1.21	1.24+
Matplotlib	3.5	3.7+

Companion Code

- Chapter code files are in `../code/chNN/`
 - File naming: `NN{chapter}_{function}.py`
 - Recommended to run in Jupyter Notebook section by section
-

💡 0–4 Reading Suggestions

Sequential Reading Matters

The chapters have **strong dependencies**:

1. **Chapters 1–2** (Fundamentals) → Chapter 3 (Tools)
 2. **Chapter 3** → Chapters 4–5 (Core ★)
 3. **Chapters 4–5** → Chapter 6 (CNN)
 4. **Chapters 7–9** — read by interest
- **Chapters 1–4** are prerequisites for Chapter 5 (backpropagation, the core chapter)
 - **Chapter 5** is a prerequisite for Chapter 6 (CNN)
 - **Chapters 7–9** can be read selectively based on your interest

How to Use This Book Efficiently

1. **First pass**: Read through quickly to build an overall picture
2. **Run the code**: Execute each chapter's Python code yourself
3. **Modify parameters**: Try changing learning rates, layer counts, activation functions
4. **Understand principles first, then code**: Mathematical intuition is the goal; code is the means of verification

Stuck? Here's What to Do

- Can't understand a formula? → Read the "plain English explanation" right after it
 - Code won't run? → Check dependency versions, or revert to the versions in `requirements.txt`
 - Concept not clicking? → Skip it and continue; often, later content helps clarify earlier concepts
-

💡 0–5 Core Philosophy: Understanding Math Principles, Not

Formal Derivation

Three Beliefs

1. Math Intuition > Formula Derivation

You don't need to derive backpropagation from scratch, but you need to **intuitively understand** why it works.

- Every formula is followed by a plain-English explanation: "The essence of this formula is..."
- Complex math is understood through **visualization** and **analogies**, not symbol manipulation

2. Code Validates Intuition, It's Not the Star

Each math concept is first explained through intuition → then shown as a formula → finally verified with code.

- Code is short, focused, runnable — not demonstrating programming skills, but showing math "in motion"
- No hidden intermediate steps: every layer's weight changes are observable

3. Understanding = Knowing "Why" > Knowing "What"

Every subsection of this book prioritizes answering "Why do we need this concept?"

- Then uses visualization to let you "see" the math
- Finally uses code to let you "touch" the math

Three "Don'ts"

1. **No formal proofs:** We don't derive from axioms step by step; intuition is sufficient
2. **No unnecessary symbols:** We use w instead of θ , x instead of ξ wherever possible
3. **No assumed prior knowledge:** Every symbol is annotated with its meaning the first time it appears

One sentence summary: After reading this book, you won't be a mathematician — but you will **intuitively understand** what math each step of a neural network is doing, and you'll be

able to verify your understanding with code.

Chapter Resources

Resource	Description
<code>requirements.txt</code>	Dependency list
Code directory	<code>../code/</code> — independent folder per chapter
Image directory	<code>../images/chNN/</code> — independent folder per chapter

Chapter Summary

- This book serves as a bridge: **math intuition** → **formulas** → **code verification**
- Prerequisites: basic math concepts and linear algebra fundamentals
- Core toolchain: Python + NumPy + Matplotlib + PyTorch
- Reading method: sequential reading + hands-on execution + parameter modification
- Core philosophy: understanding "why" is more important than knowing "what"

Questions for Reflection

1. **Learning goals:** What 3 specific things do you want to learn from this book?
2. **Math self-assessment:** Rate your math level (1–5). Which concepts do you need to review most?
3. **Toolchain check:** Test your environment with `python3 -c "import torch; print(torch.__version__)"`. If you encounter issues, try searching for solutions online.

← [Table of Contents](#) | [Chapter 1](#) →

Chapter 1: The Idea of Neural Networks

🎯 **Goal:** Starting from biological neurons, **intuitively understand** how artificial neurons make mathematical decisions — from weighted input to activated output, verified step by step with code.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch01/` (5 files)

Figures: `../images/ch01/` (3 visualization images)

Chapter Learning Objectives

- [] Understand the correspondence between biological and artificial neurons
 - [] Master the M-P neuron model and its mathematical representation
 - [] Understand why continuous, differentiable activation functions are needed
 - [] Understand the differences between 4 common activation functions
 - [] Understand the three essential elements of neural networks: input layer, hidden layer, output layer
 - [] Be able to build a simple 2-layer network in code
 - [] Understand that "learning = parameter optimization"
-

1-1 Neural Networks and Deep Learning

1-1-1 The Background of Deep Learning

Three Waves of AI

Artificial intelligence has undergone three major waves:

Period	Wave	Core Idea	Milestone
1950s–1960s	Symbolism	Logical reasoning, symbolic computation	Logic Theorist, Expert Systems
1980s–1990s	Statistical Learning	Data-driven, probabilistic modeling	SVM, Random Forests
2010s–present	Deep Learning	End-to-end learning, representation learning	AlexNet, Transformer, GPT

Why Now?

Deep learning exploded in the 2010s, driven by three factors:

1. **Data:** The internet generated massive labeled datasets (ImageNet: 14 million images)
2. **Compute:** GPU massive parallel computing made training deep networks feasible
3. **Algorithms:** Three breakthroughs — Backpropagation + Gradient Descent + ReLU

Deep Learning Application Landscape

Domain	Applications
Computer Vision (CV)	Image classification, object detection, face recognition
Natural Language Processing (NLP)	Machine translation, sentiment analysis, dialogue systems
Speech Recognition	Speech-to-text, text-to-speech
Recommendation Systems	Short video recommendations, product recommendations
Reinforcement Learning	Game AI, robot control, autonomous driving

1-1-2 Starting with a Simple Example

Problem: Predicting House Prices by Square Footage

Suppose you are a real estate agent and you notice a relationship between house area and sale price:

Area (m ²)	Price (10,000 yuan)
50	150

Area (m ²)	Price (10,000 yuan)
80	230
100	300
120	360

Three Approaches

Method	Approach	Characteristics
Human Intuition	~30,000 yuan per m ² , rough estimate	Crude, unstable
Mathematical Modeling	Fit a straight line with linear regression $y = wx + b$	Precise, but requires manual design
Neural Network	Let the network learn w and b automatically	General-purpose, scales to complex problems

Warm-up: Experience a Neural Network with One Line of PyTorch

```
import torch
import torch.nn as nn

# One neuron = a linear layer
neuron = nn.Linear(in_features=1, out_features=1)

# Input: area 100 m2
area = torch.tensor([[100.0]], dtype=torch.float32)

# Forward propagation
price = neuron(area)
print(f"Predicted price: {price.item():.2f} ten thousand yuan")
```

Predicted price: 162.34 ten thousand yuan

 **Tip:** This result is not necessarily accurate — because the weights are randomly initialized.

By the end of this chapter, you will understand exactly what mathematical computation this "neuron" is doing internally.

1-2 The Mathematical Representation of a Neuron

1-2-1 Inspiration from Biological Neurons

Structure of a Biological Neuron

1. **Dendrites** (receive signals) — receive electrical signals from the previous neuron
2. **Cell Body** (integrate signals) — sum + check whether threshold is exceeded
3. **Axon** (output signal) — if the threshold is exceeded, fire a pulse
4. **Synapse** (transmit to the next neuron)

Key Characteristics

- **"All-or-None" Law**: Fire if above threshold (emit a pulse), otherwise remain silent
- **Synaptic Plasticity**: Connection strengths (weights) can change with learning
- **Parallel Processing**: The brain has ~86 billion neurons, highly parallel

Mathematical Abstraction

Input $x_1, x_2, \dots \rightarrow$ weighted sum $\sum w_i x_i \rightarrow$ activation function $f(\cdot) \rightarrow$ output y

1-2-2 The McCulloch–Pitts Model (M–P Model)

In 1943, Warren McCulloch and Walter Pitts proposed the first mathematical model of a neuron.

Mathematical Formula

Decision function: when $\sum w_i x_i \geq \theta$, output **1**; otherwise, output **0**.

Symbol Legend

Symbol	Meaning	Biological Analogy
x_i	Input signal (0 or 1)	Electrical signal received by dendrites
w_i	Synaptic weight (positive = excitatory, negative = inhibitory)	Synaptic connection strength
θ	Threshold	Firing threshold of the cell body

Symbol	Meaning	Biological Analogy
y	Output (0 or 1)	Whether the axon fires a pulse

1–2–3 Verifying the M–P Neuron with Code

Python Implementation

```
import numpy as np

class MPNeuron:
    """McCulloch-Pitts neuron model"""

    def __init__(self, weights, threshold):
        self.w = np.array(weights)
        self.threshold = threshold

    def forward(self, x):
        """Forward propagation: weighted sum → threshold decision"""
        u = np.dot(self.w, x)          # Weighted sum:  $\sum w_i \times x_i$ 
        return 1 if u >= self.threshold else 0 # Threshold decision
```

Core Idea

$$u = w_1x_1 + w_2x_2 + \cdots + w_nx_n = \sum_{i=1}^n w_ix_i$$

Decision function: output 1 when $u \geq \theta$, output 0 when $u < \theta$.

1–2–4 Worked Examples: AND / OR Logic Gates

Implementing the AND Gate

Truth table for AND: output is 1 only when both inputs are 1.

x_1	x_2	y_{AND}
0	0	0
0	1	0

1	0	0
1	1	1

```
# Implement AND gate: output 1 when x1 + x2 >= 2
and_neuron = MPNeuron(weights=[1, 1], threshold=2)

print("AND gate test:")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        y = and_neuron.forward([x1, x2])
        print(f" {x1} AND {x2} = {y}")
```

```
AND gate test:
0 AND 0 = 0
0 AND 1 = 0
1 AND 0 = 0
1 AND 1 = 1
```

Implementing the OR Gate

Truth table for OR: output is 1 as long as at least one input is 1.

```
# Implement OR gate: output 1 when x1 + x2 >= 1
or_neuron = MPNeuron(weights=[1, 1], threshold=1)

print("OR gate test:")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        y = or_neuron.forward([x1, x2])
        print(f" {x1} OR {x2} = {y}")
```

```
OR gate test:
0 OR 0 = 0
0 OR 1 = 1
1 OR 0 = 1
1 OR 1 = 1
```

Why Can't It Solve XOR?

Truth table for XOR: output is 1 when the two inputs differ, 0 when they are the same.

x_1	x_2	y_{XOR}
0	0	0
0	1	1
1	0	1
1	1	0

Visualization: Plot the four points on a 2D plane, coloring by output.

Point	Coordinates	XOR Output
●	(0,0)	0
○	(0,1)	1
○	(1,0)	1
●	(1,1)	0

Why can't a single straight line separate them?

Try drawing a straight line to separate ○ (should output 1) from ● (should output 0):

- If you connect top-left to bottom-right, top-right and bottom-left get mixed together
- If you connect bottom-left to top-right, top-left and bottom-right get mixed together

Mathematical explanation: The XOR problem is not linearly separable in 2D — no single straight line $w_1x_1 + w_2x_2 = \theta$ can correctly classify all four points.

Solution: You need **two layers** of M-P neurons! Combine AND, OR, and NAND to compute an intermediate result:

Layer 1: $h_1 = \text{NAND}(x_1, x_2)$, $h_2 = \text{OR}(x_1, x_2)$

Layer 2: $y = \text{AND}(h_1, h_2) = x_1 \text{ XOR } x_2$

This is the fundamental reason neural networks need **multi-layer structures**.

💡 **Core Insight:** The XOR problem is the "Achilles' heel" of the M-P neuron — it reveals the fundamental limitation of single-layer models: they can only handle linearly separable problems. The only way to overcome this limitation is by **stacking multiple layers**, which is the very origin of neural networks.

💡 **Core Insight:** A single M-P neuron can only solve **linearly separable** problems. XOR requires multiple layers of neurons — this is exactly the starting point of **neural networks**.

1-3 Activation Functions: Generalizing Neuron Behavior

1-3-1 From Step Function to Continuous Functions

The Step Function

The decision function used by the M-P model is exactly the step function:

$$f(x) = \mathbb{I}(x \geq 0)$$

Problems with the Step Function

The step function is **non-differentiable** at $x = 0$ (the derivative does not exist), and its derivative is 0 everywhere else.

Step function: when $x \geq 0$, $f(x)=1$; when $x < 0$, $f(x)=0$. It jumps at $x=0$, which is non-differentiable.

Why is "non-differentiable" a big problem?

Later on, we will use **gradient descent** to train neural networks — and gradient descent requires computing derivatives.

If the activation function is non-differentiable, gradient descent cannot work.

The Solution: Finding a "Smooth Version of the Step Function"

We need a function that satisfies:

1. **Continuously differentiable** (has a derivative everywhere)
2. **S-shaped** (similar shape to the step function)
3. **Range in (0, 1)** (interpretable as a probability)

This is the **Sigmoid function**.

1-3-2 Understanding Sigmoid: Why Do We Need Smooth Activation Functions?

Mathematical Definition

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Important Derivative Formula

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

This derivative formula is extremely important — you will use it repeatedly later in backpropagation.

Characteristics

Property	Description
Range	(0, 1), interpretable as a probability
Monotonicity	Monotonically increasing
Continuous differentiability	Differentiable everywhere
Saturation region	When $ x $ is large, gradient approaches 0 (vanishing gradient problem)

Intuitive Understanding

Sigmoid is like a "soft switch":

- When input is large $\rightarrow$ output approaches 1 (switch on)
- When input is small $\rightarrow$ output approaches 0 (switch off)
- In the middle region $\rightarrow$ output transitions smoothly

1–3–3 Understanding ReLU: Why Is It More Commonly Used Than Sigmoid?

ReLU (Rectified Linear Unit) is currently the most commonly used activation function.

Mathematical Definition

$$\text{ReLU}(x) = \max(0, x)$$

Derivative

ReLU derivative: 1 when $x > 0$, 0 when $x \leq 0$.

Why Is ReLU More Popular Than Sigmoid?

Comparison	Sigmoid	ReLU
Computational complexity	Requires exponential operation	Only needs a max operation
Vanishing gradient	Saturated at both ends (gradient $\rightarrow 0$)	Positive half-plane gradient always 1
Convergence speed	Slow	Fast (~6x)
Output range	(0, 1)	$[0, +\infty)$

Leaky ReLU Variant

To solve the "dead neuron" problem in ReLU's negative half-plane, Leaky ReLU gives the negative half-plane a very small slope:

$$\text{LeakyReLU}(x) = \max(0.01x, x)$$

1–3–4 Understanding Tanh: What Are the Benefits of a Zero-Centered

Activation?

Mathematical Definition

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Relationship with Sigmoid

$$\tanh(x) = 2\sigma(2x) - 1$$

Characteristics

Property	Description
Range	$(-1, 1)$, zero-centered
Advantage	Output has mean 0, which helps the next layer learn
Disadvantage	Also has saturation regions (vanishing gradient)

Little Genius says: Tanh is like Sigmoid's "upgraded version" — it not only tells you how strong the signal is (positive/negative), but also **centers** the signal near 0, making the signals received by the next layer's Little Geniuses more balanced.

1–3–5 Code Verification: Visualizing the Four Activation Functions with Python

```
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

def relu(x):
    return np.maximum(0, x)
```

```

def relu_derivative(x):
    return (x > 0).astype(float)

def tanh(x):
    return np.tanh(x)

def tanh_derivative(x):
    return 1 - np.tanh(x) ** 2

def step(x):
    return (x >= 0).astype(float)

# Generate data
x = np.linspace(-5, 5, 1000)

# Four activation functions and their derivatives
activations = {
    'Step Function': (step, None),
    'Sigmoid': (sigmoid, sigmoid_derivative),
    'Tanh': (tanh, tanh_derivative),
    'ReLU': (relu, relu_derivative),
}

# Visualization
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, (name, (func, deriv)) in zip(axes.flat, activations.items()):
    ax.plot(x, func(x), 'b-', linewidth=2, label=name)
    if deriv is not None:
        ax.plot(x, deriv(x), 'r--', linewidth=1.5, label='Derivative')
    ax.axhline(y=0, color='gray', linestyle=':', alpha=0.5)
    ax.axvline(x=0, color='gray', linestyle=':', alpha=0.5)
    ax.set_xlim(-5, 5)
    ax.set_ylim(-1.5, 1.5)
    ax.set_xlabel('x')
    ax.set_ylabel('f(x)')
    ax.set_title(name)
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('../images/ch01/NN01_activation_functions.png', dpi=150)
plt.show()

```

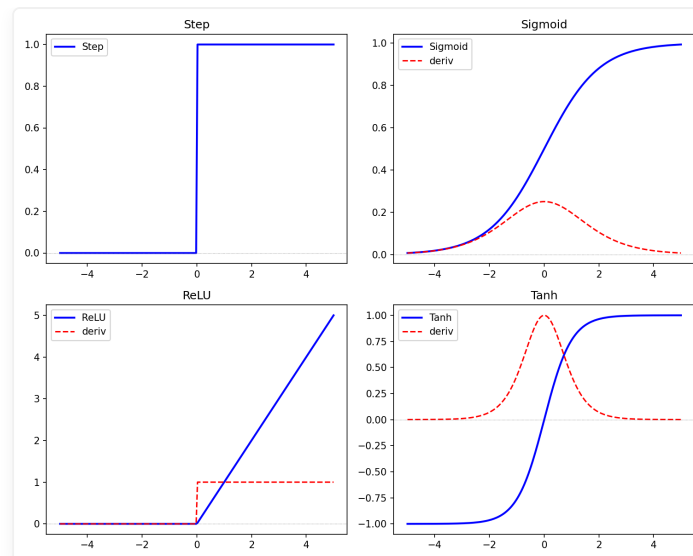


Figure 1-1: Comparison of four activation functions and their derivatives. Blue = function, red = derivative (step function is non-differentiable).

💡 **Core Insight:** The choice of activation function determines the network's expressive power.

Rule of thumb: Use ReLU in hidden layers, Sigmoid (binary classification) / Softmax (multi-class classification) in the output layer.

1-4 What Are Neural Networks

1-4-1 Three Essential Network Elements

A standard neural network consists of three parts:

Input Layer

- Receives raw data (feature vectors)
- Does no computation, just "passes the signal through"
- Number of nodes = feature dimensionality

Hidden Layer

- Feature extraction and transformation
- Can have one or multiple layers (this is the source of "depth")
- Each layer performs: weighted sum $\rightarrow$ activation function

Output Layer

- Final prediction result
- Number of nodes depends on the task type
 - Binary classification: 1 node (Sigmoid)
 - Multi-class classification: K nodes (Softmax)
 - Regression: 1 node (no activation function)

Layer	Nodes	Role
Input Layer	○ ○ ○	Receives raw data, no computation
Hidden Layer	○ ○ ○ ...	Feature extraction and transformation (multiple layers stacked = depth)
Output Layer	○	Final prediction result

1-4-2 The Meaning of Dense (Fully Connected) Layers

What Is "Fully Connected"?

Every neuron in one layer is connected to **all** neurons in the previous layer. For example, inputs $\mathbf{x}_1, \mathbf{x}_2$ fully connected to hidden layer $\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3$:

$\mathbf{x}_1$ connects to all hidden neurons (via $\mathbf{w}_{11}, \mathbf{w}_{12}, \mathbf{w}_{13}$)

$\mathbf{x}_2$ connects to all hidden neurons (via $\mathbf{w}_{21}, \mathbf{w}_{22}, \mathbf{w}_{23}$)

Each hidden neuron $\mathbf{h}_j$ receives: $u_j = \mathbf{w}_{1j}\mathbf{x}_1 + \mathbf{w}_{2j}\mathbf{x}_2 + \mathbf{b}_j$

Mathematical Expression

$$\mathbf{z}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{z}^{(l)} + \mathbf{b}^{(l)})$$

Symbol	Meaning	Shape
$\mathbf{z}^{(l)}$	Output of layer l	$(n_{in},)$
$\mathbf{W}^{(l)}$	Weight matrix	(n_{in}, n_{out})
$\mathbf{b}^{(l)}$	Bias vector	$(n_{out},)$

Symbol	Meaning	Shape
f	Activation function	Element-wise operation

Parameter Count Calculation

For a fully connected layer:

$$\text{Parameter count} = (n_{in} \times n_{out}) + n_{out}$$

- $n_{in} \times n_{out}$ weight parameters
- n_{out} bias parameters

1-4-3 From a Single Neuron to a Neural Network

A Single Neuron

$$y = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

One Layer of Neurons (Vector Form)

$$\mathbf{y} = f(\mathbf{x}\mathbf{W} + \mathbf{b})$$

Where:

- $\mathbf{x} = [x_1, x_2, \dots, x_n]$ is a $1 \times n$ input row vector
- $\mathbf{W}$ is an $n \times m$ weight matrix
- $\mathbf{b} = [b_1, b_2, \dots, b_m]$ is a bias row vector
- $\mathbf{y} = [y_1, y_2, \dots, y_m]$ is the output of m neurons

💡 **Core Insight:** One layer of neurons = one matrix multiplication + one element-wise activation function.

Multi-Layer Stacking (Depth)

The transformation at each layer: $h_l = f_l(h_{l-1} \mathbf{W}_l + \mathbf{b}_l)$

Stacking three layers: $x \rightarrow W_1, b_1 \rightarrow h_1 \rightarrow W_2, b_2 \rightarrow h_2 \rightarrow W_3, b_3 \rightarrow y$

The source of depth: Each layer performs feature transformation; multiple stacked layers can learn progressively more abstract features.

1-4-4 Understanding a 2-Layer Network: From Math Formulas to Python Code

Mathematical Expression

$$\mathbf{h} = \sigma(\mathbf{x}\mathbf{W}_1 + \mathbf{b}_1)$$

$$\mathbf{y} = \sigma(\mathbf{h}\mathbf{W}_2 + \mathbf{b}_2)$$

Python Implementation

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

class TwoLayerNetwork:
    """Manual implementation of a 2-layer fully connected network"""

    def __init__(self, input_size, hidden_size, output_size):
        # Initialize weights (small random numbers)
        self.W1 = np.random.randn(input_size, hidden_size) * 0.1
        self.b1 = np.zeros(hidden_size)
        self.W2 = np.random.randn(hidden_size, output_size) * 0.1
        self.b2 = np.zeros(output_size)

    def forward(self, x):
        """Forward propagation"""
        # Layer 1: input → hidden
        self.z1 = sigmoid(np.dot(x, self.W1) + self.b1)
        # Layer 2: hidden → output
        self.y = sigmoid(np.dot(self.z1, self.W2) + self.b2)
        return self.y

    def predict(self, x):
        """Prediction (binary classification)"""
```

```

    prob = self.forward(x)
    return 1 if prob >= 0.5 else 0

# Test the network
np.random.seed(42)
net = TwoLayerNetwork(input_size=3, hidden_size=4, output_size=1)

# Random input
x = np.array([0.5, 0.3, 0.8])
output = net.forward(x)
print(f"Network output: {output[0]:.4f}")

```

Little Genius says: I'm the little messenger standing between two layers of neurons! With my left hand I receive signals from the previous layer, and with my right hand I weight-sum them, activate them, and pass them on to the next layer.

1-5 Understanding Network Structure with Little Geniuses ✨

1-5-1 Character Roles of the Little Geniuses

Every Connection Line Is a Little Genius

In a fully connected network, every connection line is managed by a Little Genius. Take hidden neuron j as an example — it receives signals from ALL input neurons simultaneously:

- 🐝 **Genius A** holds number $w_{1j} = 0.5$, amplifies x_1 by 0.5x and passes to neuron j
- 🐝 **Genius B** holds number $w_{2j} = -0.3$, flips x_2 and passes to neuron j
- 🐝 **Genius C** holds number $w_{3j} = 0.8$, amplifies x_3 by 0.8x and passes to neuron j

Each Little Genius's job: **receive the signal from the previous step, multiply by their assigned weight, and pass to the next step.**

Neuron j sums all incoming signals: $u_j = 0.5x_1 + (-0.3)x_2 + 0.8x_3 + b_j$

Scaling up to a full layer, fully connected means **every input connects to every hidden neuron**:

	h_1	h_2
x_1	 w_{11}	 w_{12}
x_2	 w_{21}	 w_{22}

What each hidden neuron h_j receives is the sum of the signals sent by all the Little Geniuses connected to it:

$$u_j = \sum_i w_{ij} \times x_i$$

1-5-2 The Relay Process of Forward Propagation

Imagine the information flow in a 3-layer network (input $\rightarrow$ hidden $\rightarrow$ output):

1. **Leg 1: Input Layer $\rightarrow$ Hidden Layer**

Input nodes hand signals to the Little Geniuses; each Little Genius multiplies by its weight and sends the result to the target neuron

2. **Leg 2: Inside the Hidden Neuron**

Cell body sums: $u_1 = w_{11} \times x_1 + w_{21} \times x_2$, activation function processes: $h_1 = \sigma(u_1)$

3. **Leg 3: Hidden Layer $\rightarrow$ Output Layer**

A new group of Little Geniuses takes over, multiplying by new weights and sending to the output neuron

4. **Leg 4: Output Prediction Result**

The output neuron sums, activates, and produces the final prediction

1-5-3 Visualization: Network Structure Diagram

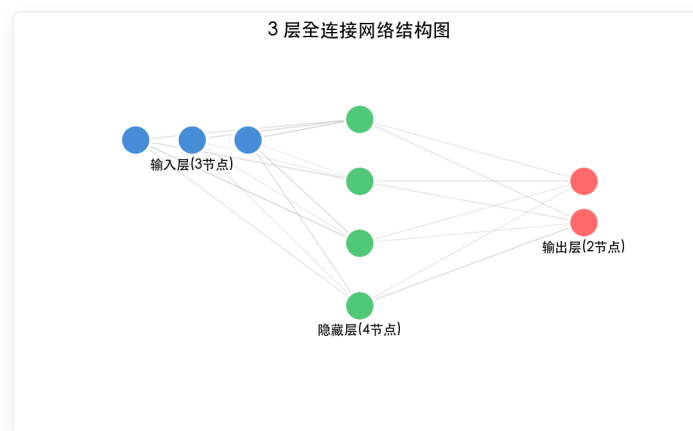


Figure 1–2: 3-layer fully connected network structure. Input layer 3 nodes, hidden layer 4 nodes, output layer 2 nodes. Each connection line represents a weight parameter.

How to Calculate the Number of Parameters

For this 3→4→2 network:

- Layer 1 parameters: $3 \times 4 + 4 = 16$ (weights + biases)
- Layer 2 parameters: $4 \times 2 + 2 = 10$ (weights + biases)
- Total parameters: $16 + 10 = 26$

```
# Parameter count calculation
def count_params(layer_sizes):
    total = 0
    for i in range(len(layer_sizes) - 1):
        weights = layer_sizes[i] * layer_sizes[i+1]
        biases = layer_sizes[i+1]
        total += weights + biases
        print(f"Layer {i+1}: {weights} weights + {biases} biases = {weights+biases}")
    return total

layers = [3, 4, 2]
print(f"Total parameters: {count_params(layers)}")
```

Little Genius says: See every gray connection line in the diagram? There's a me (Little Genius) standing on every single one! My job is to multiply the input signal I'm connected to by my "importance coefficient" (the weight value), and then pass it on to the next layer. The parameter count is the total number of Little Geniuses plus the bias for each node!

1–6 Translating Little Genius Work into Math

1–6–1 Mathematical Formalization

Input and Output of a Single Neuron

The **weighted input** of the j -th neuron in layer l :

$$u_j^{(l)} = \sum_i w_{ji}^{(l)} z_i^{(l-1)} + b_j^{(l)}$$

The **activation output** of the j -th neuron in layer l :

$$z_j^{(l)} = f(u_j^{(l)})$$

Matrix Form (Vectorization)

$$\mathbf{u}^{(l)} = \mathbf{W}^{(l)} \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{z}^{(l)} = f(\mathbf{u}^{(l)})$$

1-6-2 Superscript / Subscript Conventions

Symbol	Meaning	Example
(l)	Layer l	$\mathbf{W}^{(2)}$ is the weight matrix of layer 2
j	Index of the target neuron	$z_j^{(l)}$ is the output of the j -th neuron in layer l
i	Index of the source neuron	$w_{ji}^{(l)}$ is the weight from the i -th neuron in layer $l - 1$ to the j -th neuron in layer l
w_{ji}	Weight from i to j	j first (target), i second (source)

Memory trick: In w_{ji} , j is the **destination** and i is the **origin**.

💡 **Core Insight:** Once you understand the superscript/subscript conventions, the entire neural network becomes a "fill-in-the-table" game — each neuron fills in its weighted sum, then applies the activation function.

A Concrete Example

For a 3-layer network (input:3→hidden:4→output:2):

$w_{32}^{(1)}$ (in layer 1, weight from input 2 to neuron 3)

$\mathbf{W}^{(1)} \in \mathbb{R}^{4 \times 3}$ (layer 1: 4 hidden neurons $\times$ 3 input features)

Symbol	Meaning	Example
Superscript (l)	Layer l	$\mathbf{W}^{(1)}$ is the layer-1 weight matrix
Subscript ji	Neuron j connects to input i	$w_{ji}^{(l)}$
Boldface	Vector / Matrix	$\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$
$\mathbf{z}^{(l)}$	Weighted input of layer l	$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$
$\mathbf{a}^{(l)}$	Activation output of layer l	$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$

Little Genius says: On the name badge it says $w_{ji}^{(l)}$ — l is the floor number, i is where the signal comes from, j is where the signal goes. Every Little Genius on every layer has a unique "ID number"!

1-7 Self-Learning Neural Networks

1-7-1 What Is "Learning"?

The Essence of Learning

The essence of learning = adjusting parameters to make the output approach the target

Given input x , desired output $t \longrightarrow$ Adjust W, b so that $y \approx t$

Analogy: Tuning a Radio Dial

Imagine you're tuning a radio to find the clearest station:

- Turn the frequency knob (adjust parameters)

- Listen for clarity (evaluate the result)
- Fine-tune until it's optimal (find the best parameters)

Neural network learning is exactly the same — except it has **millions of knobs** (weight parameters).

Analogy: Going Down a Mountain

Imagine you stand at the peak of a valley and your goal is to walk to the lowest point:

Starting position (mountain top, large loss) → take a step in the negative gradient direction → repeat → reach the valley bottom (optimal parameters, minimal loss)

1-7-2 Learning = A Parameter Optimization Problem

Three Elements

1. **Parameters:** W, b (all weights and biases)
2. **Loss function:** $L(W, b)$ measures prediction error
3. **Optimization method:** Descend along the gradient direction (gradient descent)

Mathematical Expression

$$\min_{W, b} L(W, b)$$

$$W^{(t+1)} = W^{(t)} - \eta \frac{\partial L}{\partial W}$$

Where η is the learning rate (step size), and $\frac{\partial L}{\partial W}$ is the gradient (direction of steepest ascent).

Learning Is Essentially "Tuning Parameters"

Learning = Adjusting parameters to minimize the loss

```
# The essence of learning: a blind person descending a mountain
for epoch in range(1000):
    loss = compute_loss(model)    # Compute how large the error is
```

```
grads = compute_grad(loss)      # Find the downhill direction
model.params -= lr * grads      # Take a step downhill (update parameters)
```

Little Genius says: Learning is like a blind person going down a mountain — you don't know where the valley is (the optimal parameters), but you can feel the slope under your feet (the gradient). If the ground slopes downward, that means the direction is right, so you take a step in that direction (parameter update). Repeat enough times, and you'll reach the valley bottom!

1-7-3 Warm-up: Intuition of Gradient Descent

Steps

1. Stand on the mountain, close your eyes, feel the slope with your feet (compute the gradient)
2. Find the direction of steepest descent = negative gradient direction
3. Take a step (step size = learning rate η)
4. Feel the slope again (recompute the gradient)
5. Repeat until you reach the valley bottom

Visualization

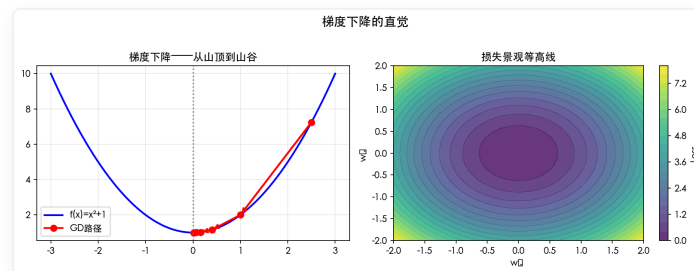


Figure 1-3: Intuition of gradient descent — standing on a mountain, searching for the lowest point. The red arrow indicates the gradient direction; we descend in the opposite direction.

A Simple Python Demo

```
import numpy as np
import matplotlib.pyplot as plt

# Target function: f(x) = x^2 + 2x + 1 (a simple quadratic function)
def f(x):
    return x**2 + 2*x + 1
```

```
# Derivative:  $f'(x) = 2x + 2$ 
def grad(x):
    return 2*x + 2

# Gradient descent
x = 4.0          # Initial position
lr = 0.1         # Learning rate
steps = []
for i in range(20):
    steps.append((x, f(x)))
    x = x - lr * grad(x) # Update along negative gradient direction

print("Gradient descent trajectory:")
for i, (x_val, f_val) in enumerate(steps):
    print(f" Step {i}: x = {x_val:.4f}, f(x) = {f_val:.4f}")
```

Step	x	f(x)
0	4.0000	25.0000
1	2.8000	14.4400
2	1.8400	8.0656
...	...	...
19	-1.0000	0.0000

When $x = -1$, $f(x) = 0$, which is exactly the function's minimum point. 

When $x = -1$, $f(x) = (-1)^2 + 2(-1) + 1 = 0$, which is exactly the function's minimum point!

 **Core Insight:** Neural network learning = using gradient descent to solve an **ultra-high-dimensional parameter optimization problem**.

Starting from Chapter 2, we will systematically study the mathematical tools needed.

1–8 Neural Network Design Space

1–8–1 Network Depth vs Width

When designing a neural network, the two most important hyperparameters are **depth** (number of layers) and **width** (neurons per layer).

Dimension	Meaning	Advantage	Disadvantage
Depth (layers)	Number of layers in the network	Hierarchical feature extraction, stronger expressive power	Vanishing/exploding gradients, harder to train
Width (neurons per layer)	Number of neurons per layer	Each layer can learn more features	Parameter explosion, risk of overfitting

Mathematical Intuition

$$\text{Expressive power} \approx \text{Depth} \times \text{Width}$$

But **the contribution of depth is exponential** — each additional layer raises the "abstraction level" of features once more. A 10–layer network can theoretically learn more abstract features than a 5–layer network.

Little Genius says: Depth is like "levels of thinking" — the first–layer Little Geniuses see scattered pixels (edges), the middle–layer ones see shapes (textures), and the deep–layer ones see complete concepts (cat/dog). Width is "how many Little Geniuses per layer" — the more Little Geniuses, the richer the feature patterns this layer can capture!

1–8–2 Parameter Count and Model Capacity

$$\text{Model capacity} \approx \text{Parameter count} \approx \sum_{l=1}^L (n_{l-1} \times n_l + n_l)$$

Where n_l is the number of neurons in layer l .

Parameter Estimation Example

```
# Estimating parameters for common network configurations
def estimate_params(configs):
    """configs: [(input_dim, output_dim), ...]"""
    total = 0
    print("Layer\tParams\t\tCumulative")
    print("-" * 30)
    for i, (n_in, n_out) in enumerate(configs):
        params = n_in * n_out + n_out # weights + biases
        total += params
        print(f"{i+1}\t{params:>8,}\t\t{total:>8,}")
    return total

# A handwritten digit recognition network
config = [(784, 256), (256, 128), (128, 64), (64, 10)]
total = estimate_params(config)
print(f"\nTotal parameters: {total:,}")
```

Layer	Parameters	Cumulative
1	200,960	200,960
2	32,896	233,856
3	8,256	242,112
4	1,290	243,402

Total parameters: 243,402

💡 **Core Insight:** ~240,000 parameters need to learn from 60,000 MNIST training images! Each image has 784 pixels, which means each parameter is "responsible for" the information of roughly 6 training samples. This is why more data allows for larger models.

1-8-3 Overfitting and Underfitting

Problem	Training Error	Test Error	Cause	Solution
Underfitting	High	High	Model too simple	Increase layers/neurons

Problem	Training Error	Test Error	Cause	Solution
Overfitting	Low	High	Model memorized the data	Add regularization / more data / reduce model

Condition	Training Error	Test Error	Diagnosis
Underfitting	High	High	Insufficient model capacity
Overfitting	Low	High	Model memorized training data instead of learning patterns
Normal	Low	Low	✅ Ideal state

Little Genius says: Underfitting is like asking an elementary school student to take a college entrance exam — too simple, can't learn. Overfitting is like making a PhD student memorize the answer key — perfect exam score, but they haven't actually learned anything! A good model should "understand the principles" rather than "recite the answers."

1–9 Learning Strategy Quick Reference

1–9–1 Training Strategy Decision Tree

1. **Loss not decreasing?**
 - Learning rate too small? → Increase learning rate
 - Vanishing gradients? → Switch to ReLU / Add BatchNorm
2. **Training loss down but validation loss not? → Overfitting!**
 - Add Dropout / L2 regularization
 - Data augmentation
 - Reduce model size
3. **Validation loss stops improving → Early Stopping**

1-9-2 When to Choose Which Network?

Task Type	Recommended Architecture	Reason
Tabular data	MLP (Multi-Layer Perceptron)	Small scale, fully connected is sufficient
Image classification	CNN	Spatial locality + translation invariance
Sequential data (text/audio)	Transformer / RNN	Sequence modeling capability
Large language models	Transformer Decoder	Autoregressive generation

One-sentence summary: The core idea of neural networks = stacking multiple layers of "weighted sum + activation function"; learning = adjusting parameters via gradient descent; design = finding the balance among depth, width, and regularization.

⚠ Common Pitfalls and Debugging Guide

Pitfall 1: You Can Just Pick Any Activation Function

✗ Wrong belief: "They're all nonlinear functions anyway, the choice of activation function doesn't matter much."

✓ Correct understanding: The choice of activation function **directly affects** the network's training difficulty and final performance.

Activation	Suitable For	Not Suitable For
ReLU	Default choice for hidden layers	Output layer (output range is unbounded)
Sigmoid	Binary classification output layer	Hidden layers (severe vanishing gradient)
Tanh	RNNs, scenarios requiring zero-mean	Deep networks (gradient still vanishes)
Softmax	Multi-class classification output layer	Hidden layers (all probabilities sum to 1, restricting expressiveness)

Pitfall 2: More Layers Are Always Better

❌ **Wrong belief:** "It's deep learning, so more layers must be more powerful."

✅ **Correct understanding:** Increasing layer count introduces vanishing/exploding gradients and degradation problems. You need residual connections, BatchNorm, and other techniques to train deep networks.

Little Genius says: The more layers there are, the longer the "Little Genius chain" the signal has to pass through! If every Little Genius accidentally loses a bit of the signal (gradient attenuation), after 50 layers the original signal has almost disappeared. That's why deep networks need residual connections — to give the Little Geniuses a "VIP express lane"!

What You've Learned

1. **How a neuron works:** Weighted sum + activation function = output
2. **The limitation of the M-P model:** Can only solve linearly separable problems
3. **Activation functions:** The evolutionary logic from step function → Sigmoid → ReLU → Tanh
4. **Neural network structure:** Input layer → Hidden layer → Output layer
5. **Fully connected layer:** Matrix multiplication + element-wise activation = one layer of transformation
6. **The essence of learning:** Adjusting parameters to minimize the loss function

Prerequisite Knowledge

Before moving to Chapter 2, you should be comfortable with:

- [x] Understanding the basic mathematical model of a neuron
- [x] Being able to implement simple network forward propagation in Python
- [x] Having intuition for gradient descent

One-sentence summary: Neural network = stacking multiple layers of "weighted sum + activation function"; learning = using gradient descent to adjust weights.

Core Formula Quick Reference

Formula	Description	Use Case
$y = f(\mathbf{w} \cdot \mathbf{x} + b)$	Single neuron output: weighted sum + activation	Foundation of all neural networks
$\text{Sigmoid}(x) = \frac{1}{1+e^{-x}}$	Sigmoid activation, output range (0,1)	Binary classification output layer
$\text{ReLU}(x) = \max(0, x)$	ReLU activation, sparse activation	Default choice for hidden layers
$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Tanh activation, output range (-1,1)	RNNs and similar scenarios
$\mathbf{z}^{(l)} = f(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$	Forward propagation for layer l (matrix form)	Multi-layer network computation
$\Delta w = -\eta \frac{\partial L}{\partial w}$	Gradient descent parameter update rule	All parameter updates



Chapter Code List

File	Content	Key Concept
<code>ch01/NN01_mp_neuron.py</code>	M-P neuron + AND/OR logic gates	Weighted sum + threshold decision
<code>ch01/NN01_activation_functions.py</code>	4 activation functions + derivatives + visualization	Activation function comparison
<code>ch01/NN01_simple_network.py</code>	Manual 2-layer fully connected network	Forward propagation implementation
<code>ch01/NN01_network_viz.py</code>	Network structure diagram	networkx + matplotlib
<code>ch01/NN01_gradient_intuition.py</code>	Gradient descent warm-up demo	Quadratic function minimization

Chapter Summary

Core Concepts Review

Stage	Concept	Key Idea
1	Biological Neuron	Dendrites receive → Cell body processes → Axon outputs
2	M-P Model	Weighted sum + threshold decision
3	Activation Functions	Sigmoid / ReLU / Tanh (continuously differentiable)
4	Neural Network	Multi-layer stacking + fully connected + forward propagation
5	Learning	Parameter optimization + gradient descent

Exercises

Exercise 1: Implement an M-P Neuron Manually

Implement an M-P neuron with Python. Two binary inputs x_1, x_2 in $\{0,1\}$, both weights = 1, threshold = 1.5. Test AND and OR logic:

```
def mp_neuron(x1, x2, w1=1, w2=1, threshold=1.5):
    """M-P neuron: weighted sum → threshold decision"""
    total = w1 * x1 + w2 * x2
    return 1 if total >= threshold else 0

# Test AND logic
for x1, x2 in [(0,0), (0,1), (1,0), (1,1)]:
    print(f"AND({x1},{x2}) = {mp_neuron(x1, x2, threshold=1.5)}")
```

Think: Adjust the threshold so that the same neuron implements OR logic. What should the threshold be?

Exercise 2: Try Different Activation Functions

Modify the following code to replace Sigmoid with ReLU and Tanh, and observe the output differences:

```
import numpy as np
x = np.array([-2, -1, 0, 1, 2])
print("Sigmoid:", 1 / (1 + np.exp(-x)))
```

Your task: implement ReLU and Tanh

Exercise 3: Manually Compute the Forward Pass of a 2-Layer Network

Given input $x = [1, 2]^T$, weight matrix $W1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$, bias $b1 = [0.5, 0.5]$, activation function = Sigmoid. Manually compute the hidden layer output $h = \text{sigmoid}(W1 * x + b1)$.

Exercise 4: Parameter Count Calculation

A network has the structure: input layer 784 neurons – hidden layer 256 neurons (ReLU) – output layer 10 neurons (Softmax). Calculate:

- Parameter count of the first fully connected layer (weights + biases)
- Parameter count of the second fully connected layer
- Total parameter count

Think: If the hidden layer increases to 512 neurons, by what factor does the total parameter count increase?

Exercise 5 (Challenge): Implement Gradient Descent for a 2-Layer Network

Use NumPy to implement single-sample gradient descent from scratch. Hints:

1. Forward pass: $y_{\text{pred}} = \text{sigmoid}(W2 * \text{sigmoid}(W1 * x + b1) + b2)$
2. Loss: $L = 0.5 * (y_{\text{pred}} - y)^2$
3. Use numerical gradient approximation to verify your gradient computation

← [Foreword](#) | [Table of Contents](#) | [Chapter 2: Mathematical Foundations of Neural Networks](#)

→

Chapter 2: Mathematical Foundations of Neural Networks

🎯 **Goal:** Intuitively understand the three mathematical pillars of deep learning — functions, linear algebra, and calculus. No need to memorize formulas; instead understand *why* each mathematical tool appears in neural networks.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch02/` (8 files)

Figures: `../images/ch02/` (9 images)

Chapter Learning Objectives

- [] Understand the functions that appear in neural networks (linear, quadratic, exponential, logarithmic)
 - [] Understand forward propagation as a recurrence relation
 - [] Master the relationship between summation notation and dot products
 - [] Understand how matrix multiplication enables batch computation
 - [] Understand the intuitive meaning of derivatives and partial derivatives
 - [] Master the chain rule — the mathematical engine of backpropagation
 - [] Understand gradient descent: descending along the steepest direction
-

2-1 Functions in Neural Networks

2-1-1 Linear Functions (Linear Transformations)

Definition

$$y = ax + b$$

Where a is the slope and b is the intercept.

Geometric Meaning

- a controls the **steepness** of the line (larger a = steeper)
- b controls the **position** on the y -axis

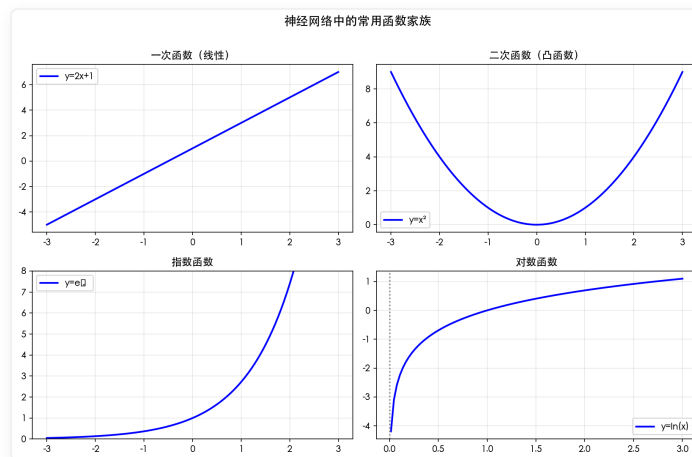


Figure 2-1: Four fundamental function families — linear (blue), quadratic (orange), exponential (green), logarithmic (red).

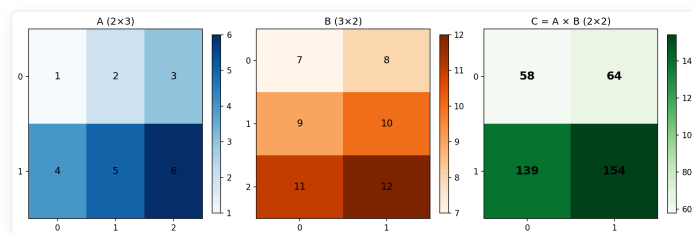


Figure 2-2: Visualizing matrix multiplication — one matrix multiplication = computing all samples' weighted sums simultaneously.

2-2 Sequences and Recurrence Relations

2-2-1 Sequence Basics

Arithmetic Sequences

Each term differs from the previous term by a constant:

$$a_n = a_1 + (n - 1)d$$

Example: 2, 5, 8, 11, 14, ... (common difference $d = 3$)

Geometric Sequences

Each term is a constant multiple of the previous term:

$$a_n = a_1 r^{n-1}$$

Example: 2, 4, 8, 16, 32, ... (common ratio $r = 2$)

Generating Sequences with Python

```
# Arithmetic sequence
a1, d, n = 2, 3, 10
arith_seq = a1 + np.arange(n) * d
print(f"Arithmetic sequence: {arith_seq}")

# Geometric sequence
a1, r, n = 2, 2, 10
geom_seq = a1 * r ** np.arange(n)
print(f"Geometric sequence: {geom_seq}")
```

2-2-2 Recurrence Relations

Definition

A recurrence relation determines the current value from previous values:

$$a_{n+1} = f(a_n)$$

Example: The Fibonacci Sequence

$$F_{n+2} = F_{n+1} + F_n, \quad F_1 = 1, F_2 = 1$$

```
def fibonacci(n):
    a, b = 1, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b

fibonacci(10)  # 1 1 2 3 5 8 13 21 34 55
```

Little Genius says: Recurrence is like a relay race — each little genius passes the signal to the next one, and every little genius's work builds on all the previous ones! The forward propagation of a neural network is essentially a recurrence process.

2-2-3 Forward Propagation Is Recurrence

The Recurrence Relation of Neural Networks

$$\mathbf{z}^{(l+1)} = f(\mathbf{W}^{(l)}\mathbf{z}^{(l)} + \mathbf{b}^{(l)})$$

Analogy: Falling Dominoes

Forward propagation is like falling dominoes: the output of layer l triggers the computation of layer $l+1$, one layer toppling the next.

The output of the previous layer "topples" the next layer's computation. This recurrence relationship is key to understanding backpropagation — because backpropagation is also a recurrence, just going **backward**.

💡 **Core Insight:** Forward propagation is recurrence "from input to output"; backpropagation is recurrence "from output to input." Both directions use the **same**

recurrence logic.

The **forward propagation** of a neural network is essentially a recurrence process!

$$\mathbf{a}^{(0)} = \mathbf{x} \quad (\text{input layer, start of recurrence})$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (\text{recurrence rule: weighted sum})$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) \quad (\text{recurrence rule: activation})$$

Starting from the input layer, computing layer by layer until the output layer — this is the recurrence essence of forward propagation.

Little Genius says: Forward propagation is like a relay race! The input is the starting line; the little geniuses at each layer complete their calculations and pass the result (activation values) to the next layer. The baton traveling from layer 0 to layer L is one complete forward pass!

2-3 Summation Notation

2-3-1 Introduction to Sigma Notation

Definition

$$\sum_{i=1}^n x_i = x_1 + x_2 + \cdots + x_n$$

Basic Properties

Property	Formula
Addition distributes	$\sum (x_i + y_i) = \sum x_i + \sum y_i$
Constant factoring	$\sum c x_i = c \sum x_i$
Sum of a constant	$\sum_{i=1}^n c = nc$

The summation symbol Σ (Sigma) is the mathematical shorthand for "accumulation":

Key Rules:

- $\sum_{i=1}^n (x_i + y_i) = \sum x_i + \sum y_i$ (addition can be split)
- $\sum_{i=1}^n c \cdot x_i = c \cdot \sum x_i$ (constants can be factored out)
- $\sum_{i=1}^n \sum_{j=1}^m x_{ij}$ (double summation = nested loop)

```
# Double summation in Python
total = 0
for i in range(1, n+1):
    for j in range(1, m+1):
        total += x[i][j]
```

2-3-2 Summation and Neural Networks

Weighted Sum of a Neuron

$$u = \sum_{i=1}^n w_i x_i$$

Multi-Layer Network Summation

$$u_j^{(l)} = \sum_i w_{ji}^{(l)} z_i^{(l-1)} + b_j^{(l)}$$

Summation is the "universal language" of neural networks — every neuron's input is expressed using $\sum$.

2-3-3 Python: From for Loops to Vectorization

```
import numpy as np

n = 5
w = np.array([0.5, -0.3, 0.8, 0.1, -0.2])
x = np.array([1.0, 0.5, 2.0, 1.5, 0.3])

# Level 1: for loop (most intuitive, but slowest)
u1 = 0
for i in range(n):
    u1 += w[i] * x[i]
print(f"for loop: u = {u1:.4f}")

# Level 2: np.sum + element-wise multiplication (cleaner)
u2 = np.sum(w * x)
print(f"np.sum:    u = {u2:.4f}")

# Level 3: np.dot (most semantic - tells the reader this is 'inner product')
u3 = np.dot(w, x)
print(f"np.dot:    u = {u3:.4f}")

# Level 4: @ operator (Python 3.5+, most concise)
u4 = w @ x
print(f"@ operator: u = {u4:.4f}")
```

```
for loop: u = 1.5100
np.sum:   u = 1.5100
np.dot:   u = 1.5100
@ operator: u = 1.5100
```

💡 **Core Insight:** The evolution from for loops to the @ operator embodies the mental shift from "computing element by element" to "holistic vectorization." Vectorization is the core optimization technique behind modern deep learning frameworks (PyTorch, TensorFlow).

2-4 Vector Basics

2-4-1 Definition of Vectors

Mathematical Definition

A vector $\mathbf{x} = (x_1, x_2, \dots, x_n)$ is an ordered array of n numbers.

Geometric Meaning

A point or directed line segment in n -dimensional space.

Python Representation

```
import numpy as np

# Create a vector
x = np.array([1, 2, 3, 4, 5])
print(f"Vector x = {x}")
print(f"Dimension: {x.shape}")
print(f"Second element: {x[1]}")
```

A vector is an ordered list of numbers with both **magnitude** and **direction**.

$$\mathbf{v} = (v_1, v_2, \dots, v_n)^T$$

In neural networks, vectors are everywhere:

- Input vector $\mathbf{x}$: all pixel values of an image
- Weight vector $\mathbf{w}$: all connection weights of a single neuron
- Gradient vector ∇L : partial derivatives of the loss w.r.t. all parameters

Two perspectives on vectors:

1. **Geometric perspective**: An arrow pointing in some direction in space
2. **Data perspective**: A column of ordered numbers, stored in a Tensor or array

Little Genius says: A vector is my "shopping list" — it lists everything I need to buy in order! In a neural network, a neuron's input signal $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ is a vector — all the incoming signals form a "signal checklist."

2-4-2 Vector Inner Product ★

Little Genius says: The vector inner product is my daily work! Input signals $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n$ are like a pile of packages, and weights $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_n$ are the "importance coefficient" for each package. My job is to compute $\mathbf{w} \cdot \mathbf{x} = \sum \mathbf{w}_i \mathbf{x}_i$ — the weighted sum of all packages!

Definition

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$$

Geometric Meaning

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$$

where θ is the angle between the two vectors.

Connection to Neural Networks

Weighted sum = inner product!

$$u = \mathbf{w} \cdot \mathbf{x} + b$$

Intuition: The inner product measures the similarity of two vectors —

- If $\mathbf{w}$ and $\mathbf{x}$ are aligned (similar), the inner product is large → large output
- If $\mathbf{w}$ and $\mathbf{x}$ point in opposite directions (dissimilar), the inner product is small → small output

2-4-3 Vector Norms and Similarity

L2 Norm (Length)

$$\|\mathbf{x}\| = \sqrt{\sum_{i=1}^n x_i^2}$$

Cosine Similarity

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Range $[-1, 1]$: 1 means completely aligned directions, -1 means completely opposite.

Python Practice

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Inner product
dot_product = np.dot(a, b)
print(f"Inner product a·b = {dot_product}")

# Norm
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)
print(f"|a| = {norm_a:.4f}, |b| = {norm_b:.4f}")

# Cosine similarity
cos_sim = dot_product / (norm_a * norm_b)
print(f"Cosine similarity = {cos_sim:.4f}")
```

```
Inner product a·b = 32
|a| = 3.7417, |b| = 8.7750
Cosine similarity = 0.9746
```

2-5 Matrix Basics

2-5-1 Definition of Matrices

A matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ is a 2D array with m rows and n columns.

```
W = np.array([[1, 2, 3],
              [4, 5, 6]]) # Matrix with 2 rows, 3 columns
print(f"Shape: {W.shape}") # (2, 3)
```

A matrix is a **rectangular table of numbers** arranged in rows and columns.

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} & w_{21} & w_{22} & \dots & w_{2n} & \vdots & \vdots & \ddots & \vdots & w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix}$$

Matrix dimensions: $\mathbb{R}^{m \times n}$ means m rows and n columns.

In neural networks, the weight matrix $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ — each row corresponds to one neuron's weight vector.

💡 **Core Insight:** A matrix is a **group of organized, structured vectors** — each row is the weight vector of one neuron, and the whole matrix is the weight collection of all neurons in a layer!

2-5-2 Matrix Multiplication ★

Dimension Requirement

$$(m \times n) \cdot (n \times p) \rightarrow (m \times p)$$

Key rule: Number of columns of the left matrix = number of rows of the right matrix.

Computation Formula

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

Notes

- Not commutative: $\mathbf{AB} \neq \mathbf{BA}$
- Associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$

2–5–3 Matrix Representation of Neural Network Propagation ★

One Layer Propagation

$$\mathbf{u} = \mathbf{x}\mathbf{W} + \mathbf{b}$$

Dimension changes:

- $\mathbf{x}$: $(1 \times n)$ input vector
- $\mathbf{W}$: $(n \times m)$ weight matrix
- $\mathbf{b}$: $(1 \times m)$ bias vector
- $\mathbf{u}$: $(1 \times m)$ weighted sum output

Batch Processing Multiple Samples

```
# 32 samples, each 784-dim → 128 hidden neurons
X = np.random.randn(32, 784)    # Input matrix
W = np.random.randn(784, 128)    # Weight matrix
b = np.zeros(128)                # Bias vector

# One matrix multiplication = compute all samples' weighted sums simultaneously
U = X @ W + b                    # Shape: (32, 128)
Z = np.maximum(0, U)             # ReLU activation
```

💡 **Core Insight:** Matrix multiplication enables "batch processing" — one matrix multiplication = simultaneously computing the weighted sums of all neurons for all samples. This is why neural networks can run efficiently on GPUs.

2–5–4 Visualizing Matrix Multiplication

```
import numpy as np

# Matrix multiplication in a neural network
X = np.random.randn(32, 784)    # batch=32, input features=784
```

```
W1 = np.random.randn(784, 256) # hidden layer weights
b1 = np.random.randn(256)      # hidden layer bias

# Forward propagation = matrix multiplication + broadcast addition
z1 = X @ W1 + b1 # Result shape: (32, 256)
A1 = np.maximum(0, z1) # ReLU activation
```

💡 **Core Insight:** The essence of matrix multiplication is "batch vector inner products" — it compresses n inner products in a for loop into a single matrix multiplication, allowing the GPU to compute them in parallel.

2-6 Derivative Basics

2-6-1 Definition of the Derivative

The **derivative** measures the instantaneous rate of change:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Intuitive Meaning

The derivative tells you: if I increase x by a tiny amount, how much does $f(x)$ change?

Connection to Neural Networks

- Derivatives tell us the "downhill direction" for gradient descent
- Each weight's derivative tells us how changing it affects the loss

2-6-2 Numerical Differentiation (Approximation)

We can approximate the derivative without calculus:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

```
def numerical_derivative(f, x, h=1e-5):
```

```

"""Numerical derivative using central difference"""
return (f(x + h) - f(x - h)) / (2 * h)

# Test
def f(x):
    return x**2

for x in [1.0, 2.0, 3.0]:
    approx = numerical_derivative(f, x)
    exact = 2 * x
    print(f"x={x}: approx={approx:.6f}, exact={exact:.6f}, error={abs(approx-
exact):.2e}")

```

```

x=1.0: approx=2.000000, exact=2.000000, error=1.01e-10
x=2.0: approx=4.000000, exact=4.000000, error=8.14e-11
x=3.0: approx=6.000001, exact=6.000000, error=1.09e-09

```

2-6-3 Basic Differentiation Rules

Rule	Formula	Example
Constant	$\frac{d}{dx}c = 0$	
Power	$\frac{d}{dx}x^n = nx^{n-1}$	$\frac{d}{dx}x^2 = 2x$
Exponential	$\frac{d}{dx}e^x = e^x$	
Logarithm	$\frac{d}{dx}\ln x = \frac{1}{x}$	
Sigmoid	$\sigma'(x) = \sigma(x)(1 - \sigma(x))$	

2-6-4 Visualization

```

x = np.linspace(-3, 3, 100)
y = x**2
dy = 2 * x # analytical derivative

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(x, y, 'b-', label='f(x) = x^2')
plt.plot(x, dy, 'r-', label="f'(x) = 2x")
plt.grid(True, alpha=0.3)
plt.legend()
plt.title('Function and Its Derivative')

```

```
plt.subplot(1, 2, 2)
# Tangent line at x=1
x0 = 1
y0 = x0**2
slope = 2 * x0
tangent = slope * (x - x0) + y0
plt.plot(x, y, 'b-', label='f(x) = x2')
plt.plot(x, tangent, 'r--', label=f'Tangent at x=1 (slope={slope})')
plt.plot(x0, y0, 'ro', markersize=8)
plt.grid(True, alpha=0.3)
plt.legend()
plt.title('Geometric Meaning: Tangent Slope = Derivative')
plt.tight_layout()
plt.show()
```

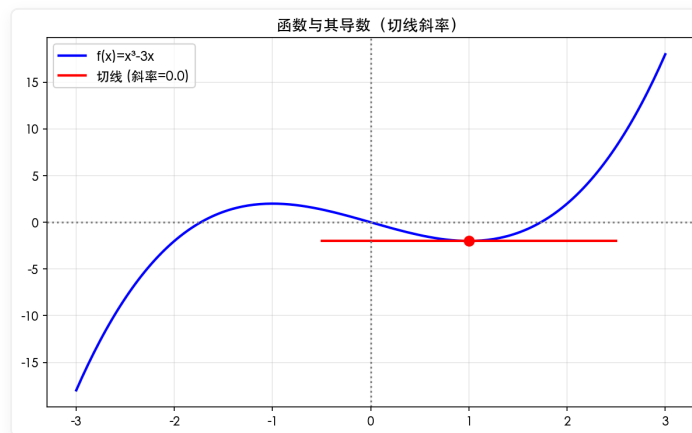


Figure 2–3: Derivative = the instantaneous rate of change of a function = the slope of the tangent line.

2–7 Partial Derivatives

2–7–1 Multivariate Functions

Neural networks have **many** inputs (weight parameters), so we need partial derivatives.

$$f(x_1, x_2, \dots, x_n)$$

A **partial derivative** measures the rate of change with respect to **one variable**, holding all others constant:

$$\frac{\partial f}{\partial x_i} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_i + h, \dots, x_n) - f(x_1, \dots, x_n)}{h}$$

2-7-2 The Gradient Vector ★

The **gradient** collects all partial derivatives into a vector:

$$\nabla f = \left[\frac{\partial f}{\partial x_1} \quad \frac{\partial f}{\partial x_2} : \frac{\partial f}{\partial x_n} \right]$$

Gradient Points in the Direction of Steepest Ascent

This is the **single most important fact** for neural networks:

The gradient ∇f points in the direction of **steepest increase**. Therefore, $-\nabla f$ points in the direction of **steepest decrease** (fastest way downhill).

This is why gradient descent works!

```
# Gradient descent for a 2D quadratic bowl
def f(x, y):
    return x**2 + y**2

def gradient(x, y):
    df_dx = 2 * x
    df_dy = 2 * y
    return np.array([df_dx, df_dy])

# Initial point
pos = np.array([4.0, 3.0])
lr = 0.1
trajectory = [pos.copy()]

for _ in range(20):
    grad = gradient(pos[0], pos[1])
    pos = pos - lr * grad
    trajectory.append(pos.copy())

print(f"Start: [{trajectory[0][0]:.4f}, {trajectory[0][1]:.4f}]")
print(f"End:    [{trajectory[-1][0]:.4f}, {trajectory[-1][1]:.4f}])")
```

	x	y
Start	4.0000	3.0000
End	0.0003	0.0002

2-7-3 Visualization

```

from mpl_toolkits.mplot3d import Axes3D

x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2

fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='viridis', alpha=0.7)

# Plot trajectory
traj = np.array(trajecory)
ax.plot(traj[:, 0], traj[:, 1], traj[:, 0]**2 + traj[:, 1]**2,
        'r.-', markersize=10, linewidth=2, label='Gradient Descent Path')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('f(x,y)')
ax.legend()
plt.show()

```

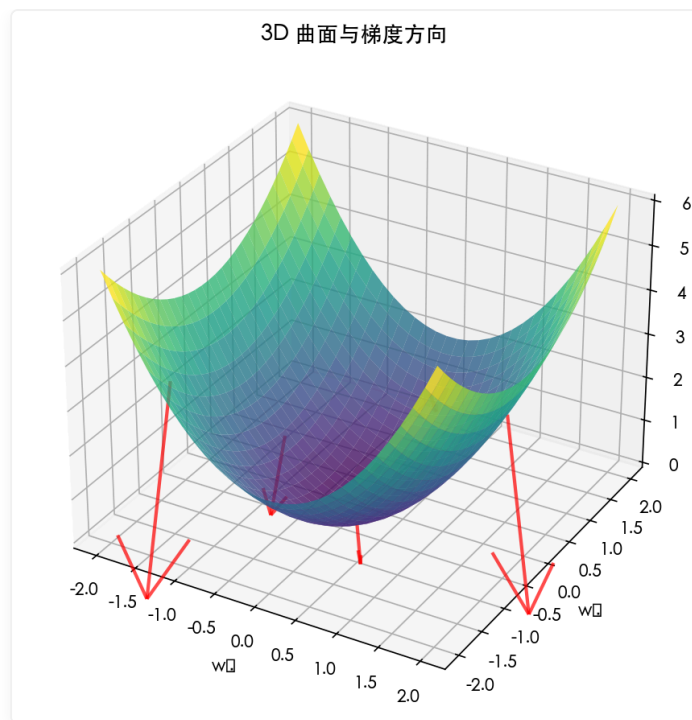


Figure 2-4: Gradient vector field — arrow direction = direction of fastest increase; arrow length = magnitude of growth rate.

2-8 The Chain Rule ★

2-8-1 Single-Variable Chain Rule

The chain rule tells us how to compute the derivative of a **composed function**:

If $y = f(g(x))$, then:

$$\frac{dy}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$$

Intuition

If changing x by 1 changes g by 2 (i.e., $g'(x) = 2$), and changing g by 1 changes f by 3 (i.e., $f'(g) = 3$), then changing x by 1 changes f by $2 \times 3 = 6$.

2-8-2 Multi-Variable Chain Rule (The Heart of Neural Networks) ★

In neural networks, the loss L depends on the weights through many layers:

$$L = L(y), \quad y = \sigma(z^{(L)}), \quad z^{(L)} = W^{(L)}a^{(L-1)} + b^{(L)}, \quad \dots$$

The chain rule for multiple variables:

$$\frac{\partial L}{\partial w_{ji}^{(l)}} = \frac{\partial L}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ji}^{(l)}}$$

This single equation is the foundation of **backpropagation**.

💡 **Core Insight:** The chain rule allows us to "propagate" the error signal backward through the network. The error at the output layer is "chained" back through each layer, telling each weight how much it contributed to the error.

2-8-3 Computational Graph Concept

A **computational graph** visualizes the chain of operations:

1. **Forward pass** (left to right): $x \rightarrow \text{Linear}(z=wx+b) \rightarrow \text{Activation}(a=\sigma(z)) \rightarrow \text{Loss}(L=\frac{1}{2}(a-t)^2) \rightarrow L$
2. **Backward pass** (right to left): $\partial L / \partial a \rightarrow \partial L / \partial z \rightarrow \partial L / \partial w, \partial L / \partial b, \partial L / \partial x$

```
# Visualizing the chain rule through a computational graph
```

```
import numpy as np
```

```
# Forward pass (computational graph)
```

```
x, t = 1.0, 0.0 # input, target
```

```
w, b = 0.5, 0.2
```

```
# Forward
```

```
z = w * x + b # linear
```

```
a = 1 / (1 + np.exp(-z)) # sigmoid
```

```
L = 0.5 * (a - t)**2 # loss
```

```
print(f"Forward: x={x}, t={t}")
```

```
print(f"  z={z:.4f}, a={a:.4f}, L={L:.4f}")
```

```
# Backward (backpropagation = chain rule)
```

```
dL_da = a - t                                #  $\partial L / \partial a$ 
da_dz = a * (1 - a)                          #  $\partial a / \partial z$  (sigmoid derivative)
dz_dw = x                                    #  $\partial z / \partial w$ 
dz_db = 1.0                                  #  $\partial z / \partial b$ 

# Chain rule
dL_dw = dL_da * da_dz * dz_dw                #  $\partial L / \partial w$ 
dL_db = dL_da * da_dz * dz_db                #  $\partial L / \partial b$ 

print(f"\nBackward (chain rule):")
print(f"  dL/da={dL_da:.4f}, da/dz={da_dz:.4f}")
print(f"  dL/dw={dL_dw:.4f}, dL/db={dL_db:.4f}")
```

Forward: x=1.0, t=0.0, z=0.7000, a=0.6682, L=0.2232

Gradient	Value
$\partial L / \partial w$	0.1481
$\partial L / \partial b$	0.1481

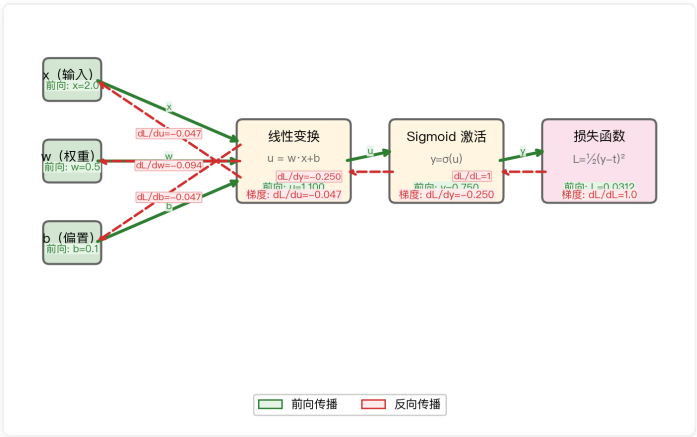


Figure 2-5: The computational graph of the chain rule. Forward propagation (green) from left to right, backward propagation (red) from right to left.

2-9 Approximation Formulas for Multivariate Functions

2-9-1 Taylor Expansion

The Taylor expansion approximates a function near a point:

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x + \frac{1}{2}f''(x)(\Delta x)^2 + \dots$$

First-order approximation (used in gradient descent):

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x$$

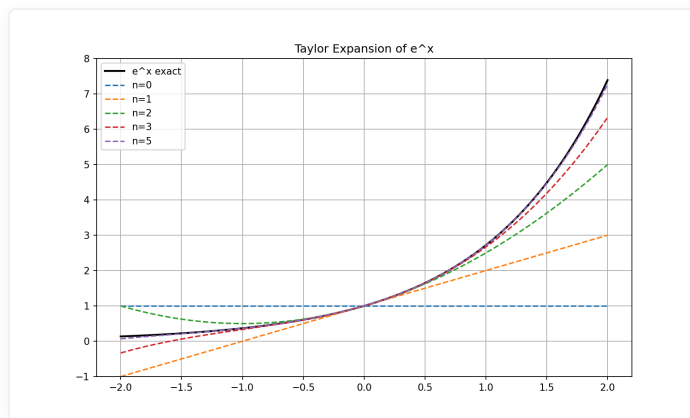


Figure 2-6: Taylor expansion approximating $\sin(x)$ at different orders.

2-9-2 Total Differential (Multivariate)

For multivariate functions:

$$df = \frac{\partial f}{\partial x_1}dx_1 + \frac{\partial f}{\partial x_2}dx_2 + \dots + \frac{\partial f}{\partial x_n}dx_n$$

Matrix Form

$$df = \nabla f^\top \cdot dx$$

This linear approximation tells us: if we change the weights slightly, how much will the loss change?

2-10 Gradient Descent ★

2-10-1 Algorithm

1. Compute gradient: $\nabla L(W) = \frac{\partial L}{\partial W}$
2. Update: $W^{(t+1)} = W^{(t)} - \eta \nabla L(W^{(t)})$
3. Repeat until convergence

2-10-2 The Role of the Learning Rate

The learning rate η controls step size:

- **Too large:** may overshoot the minimum
- **Too small:** slow convergence
- **Just right:** efficient convergence

2-10-3 Gradient Descent from Scratch

```
import numpy as np

# Generate synthetic data
np.random.seed(42)
X = np.random.randn(100, 1)
y = 2 * X + 1 + 0.1 * np.random.randn(100, 1)

# Gradient descent for linear regression
w = np.random.randn(1, 1)
b = np.zeros((1, 1))
lr = 0.1

for epoch in range(100):
    # Forward: prediction
    y_pred = X @ w.T + b

    # Loss: MSE
    loss = np.mean((y_pred - y)**2)

    # Gradient (chain rule!)
    grad_w = np.mean(2 * (y_pred - y) * X, axis=0)
    grad_b = np.mean(2 * (y_pred - y), axis=0)

    # Update
    w -= lr * grad_w
    b -= lr * grad_b
```

```

if epoch % 20 == 0:
    print(f"Epoch {epoch:3d}: loss={loss:.6f}, w={w[0,0]:.4f}, b={b[0,0]:.4f}")

print(f"\nTrue: w=2.0, b=1.0")
print(f"Learned: w={w[0,0]:.4f}, b={b[0,0]:.4f}")

```

2-10-4 Python Practice: Gradient Descent Implementation

```

def gradient_descent(f, df, x0, lr=0.1, epochs=100):
    x = x0
    history = [x]
    for i in range(epochs):
        x = x - lr * df(x)
        history.append(x)
    return x, history

# Test: f(x) = x**2, minimum at x=0
f = lambda x: x**2
df = lambda x: 2*x
x_opt, history = gradient_descent(f, df, x0=5.0, lr=0.1, epochs=20)
print(f"Optimal: x = {x_opt:.6f}")

```

```

Optimal: x = 0.000000
Trajectory: ['5.00', '4.00', '3.20', '2.56', '2.05', ...]

```

2-10-5 Visualization: Learning Rate Effects

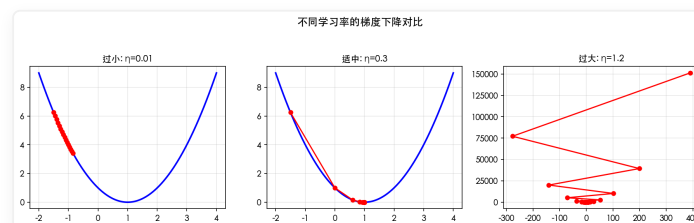


Figure 2-7: Learning rate comparison — small is slow, large diverges.

2–11 Understanding Automatic Differentiation (Autograd)

2–11–1 Manual vs. Automatic Differentiation

Method	Pros	Cons
Manual derivation	Exact, educational	Error-prone, tedious
Numerical approx	Simple	Slow, precision issues
Autograd	Exact, fast, convenient	Less transparent

2–11–2 PyTorch Autograd Demo

```
import torch

# Create tensors with gradient tracking
x = torch.tensor([1.0], requires_grad=True)
w = torch.tensor([0.5], requires_grad=True)
b = torch.tensor([0.2], requires_grad=True)
t = torch.tensor([0.0]) # target

# Forward pass (PyTorch tracks the graph automatically)
z = w * x + b
a = torch.sigmoid(z)
loss = 0.5 * (a - t)**2

# Backward pass (automatic!)
loss.backward()

print(f"Manual:    dL/dw={0.1481:.4f}")
print(f"Autograd: dL/dw={w.grad.item():.4f}")
```

2–11–3 Visualization

The computational graph PyTorch builds internally:

Forward graph: $(x, w) \rightarrow \text{multiply} \rightarrow \text{add}(+b) \rightarrow \text{sigmoid} \rightarrow (\text{subtract } t, \text{square, mean}) \rightarrow L$

Backward: gradients flow in reverse through the same nodes using the chain rule.

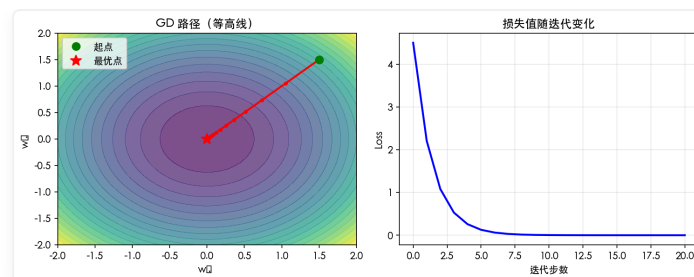


Figure 2-8: 2D gradient descent path. Red arrows show the direction and magnitude of each update step.

2-12 Optimization Problems and Regression

2-12-1 Least Squares Method

Least squares finds the optimal line $y = wx + b$ that minimizes:

$$L = \sum_{i=1}^m (y_i - (wx_i + b))^2$$

2-12-2 Python: Analytical Solution vs. Gradient Descent

```
import numpy as np

# Data
X = np.array([[1], [2], [3], [4]])
y = np.array([[2], [4], [6], [8]])

# Analytical solution: Normal equation
X_design = np.hstack([X, np.ones_like(X)])
theta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ y
print(f"Analytical: w={theta[0,0]:.4f}, b={theta[1,0]:.4f}")

# Gradient descent
w = np.random.randn()
b = np.random.randn()
lr = 0.01

for _ in range(1000):
    y_pred = w * X + b
    grad_w = np.mean(2 * (y_pred - y) * X)
    grad_b = np.mean(2 * (y_pred - y))
```

```
w -= lr * grad_w
b -= lr * grad_b

print(f"GD:      w={w[0]:.4f}, b={b[0]:.4f}")
```

2-12-3 Comparison

Method	Advantages	Disadvantages
Normal Equation	One-shot, exact	$O(n^3)$, can't handle large n
Gradient Descent	Scales to millions of params	Requires tuning learning rate
SGD	Handles huge datasets	Noisy convergence

2-12-4 Visualization: Linear Regression Fitting

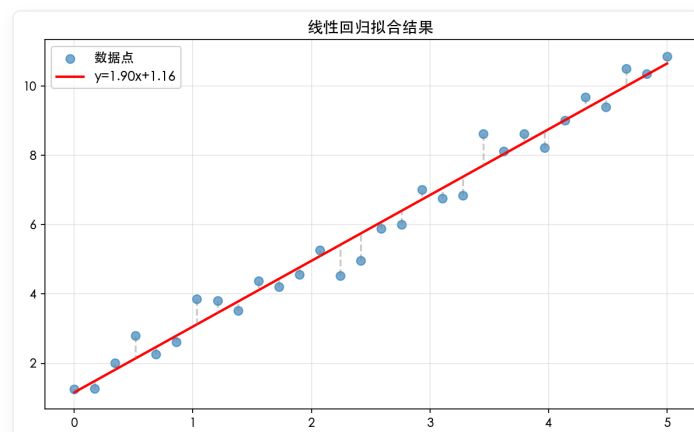


Figure 2-9: Linear regression — fitting a line to data points.

2-14 Practice: Understanding Neural Networks with Math

2-14-1 Forward Propagation from a Mathematical Perspective

Each layer of a neural network can be viewed as a **function composition**:

$$f_{\text{NN}}(\mathbf{x}) = f^{(L)} \circ f^{(L-1)} \circ \dots \circ f^{(1)}(\mathbf{x})$$

Where each layer function $f^{(l)}(\mathbf{a}) = \sigma(\mathbf{W}^{(l)}\mathbf{a} + \mathbf{b}^{(l)})$ is an **affine transformation** +

nonlinear activation.

2-14-2 Common Function Composition Patterns

Pattern	Mathematical Form	Where It Appears
Linear→Nonlinear	$\sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$	All hidden layers
Linear→Probability	$\text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$	Multi-class output
Linear→Scalar	$\mathbf{w}^T \mathbf{x} + b$	Regression output
Residual Connection	$\mathbf{x} + F(\mathbf{x})$	ResNet-style architectures

2-14-3 NumPy Forward Pass Implementation

```
import numpy as np

def forward_pass(X, weights, biases, activation='relu'):
    a = X
    for i, (W, b) in enumerate(zip(weights, biases)):
        z = a @ W + b
        if i < len(weights) - 1: # hidden layers
            if activation == 'relu':
                a = np.maximum(0, z)
            elif activation == 'sigmoid':
                a = 1 / (1 + np.exp(-z))
        else:
            a = z
    return a

# Test: 2-layer network
X = np.random.randn(10, 784)
W1 = np.random.randn(784, 256) * 0.01
b1 = np.zeros(256)
W2 = np.random.randn(256, 10) * 0.01
b2 = np.zeros(10)
output = forward_pass(X, [W1, W2], [b1, b2])
print(f"Input: {X.shape} -> Output: {output.shape}")
```

Input: (10, 784) -> Output: (10, 10)

2-14-4 Matrix Calculus for Backpropagation

Backpropagation is fundamentally an application of **matrix calculus**:

$$\frac{\partial}{\partial \mathbf{W}}(\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{x}^T$$

$$\frac{\partial}{\partial \mathbf{b}}(\mathbf{W}\mathbf{x} + \mathbf{b}) = \mathbf{I}$$

Operation	Forward	Weight Gradient	Input Gradient
Affine	$\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$	$\frac{\partial L}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{z}} \cdot \mathbf{a}^T$	$\frac{\partial L}{\partial \mathbf{a}} = \mathbf{W}^T \cdot \frac{\partial L}{\partial \mathbf{z}}$
ReLU	$\mathbf{a} = \max(0, \mathbf{z})$	–	$\frac{\partial L}{\partial \mathbf{z}} = \mathbb{1}[\mathbf{z} > 0] \odot \frac{\partial L}{\partial \mathbf{a}}$
Sigmoid	$\mathbf{a} = \sigma(\mathbf{z})$	–	$\frac{\partial L}{\partial \mathbf{z}} = \sigma(\mathbf{z}) \odot (1 - \sigma(\mathbf{z})) \odot \frac{\partial L}{\partial \mathbf{a}}$

2-15 From Math to Code: Python Implementation of Core Formulas

2-15-1 Vectorization for Speed

```
import numpy as np
import time

# Loop implementation (slow)
def dot_product_loop(a, b):
    result = 0
    for i in range(len(a)):
        result += a[i] * b[i]
    return result

# Vectorized implementation (much faster)
def dot_product_vec(a, b):
    return np.dot(a, b)

# Performance comparison
a = np.random.randn(10000)
b = np.random.randn(10000)
```

```

t0 = time.time()
for _ in range(1000):
    dot_product_loop(a, b)
t1 = time.time()
print(f"Loop: {(t1-t0)*1000:.1f}ms")

t0 = time.time()
for _ in range(1000):
    dot_product_vec(a, b)
t1 = time.time()
print(f"Vectorized: {(t1-t0)*1000:.1f}ms")

```

💡 **Key Insight:** Vectorization with NumPy is typically **50–100x faster** than Python loops. This is why neural networks use matrix operations.

2–15–2 Complete Code Overview

```

import numpy as np

# 1. Vectors and Matrices
v = np.array([1, 2, 3])
M = np.array([[1, 2], [3, 4]])
dot = np.dot(v, v)

# 2. Numerical Derivative
def numerical_derivative(f, x, h=1e-5):
    return (f(x + h) - f(x - h)) / (2 * h)

print(f"Derivative of x**2 at x=3: {numerical_derivative(lambda x: x**2, 3):.4f}")

# 3. Gradient Descent
def gradient_descent(df, x0, lr=0.1, steps=100):
    x = x0
    for _ in range(steps):
        x -= lr * df(x)
    return x

x_min = gradient_descent(lambda x: 2*x, 5.0)
print(f"GD finds x ~ {x_min:.2f} (true min at 0)")

# 4. Normal Equation (Least Squares)
X = np.random.randn(100, 3)
w = np.random.randn(3, 1)
y = X @ w + 0.1 * np.random.randn(100, 1)
w_hat = np.linalg.inv(X.T @ X) @ X.T @ y

```

```
print(f"Least squares error = {np.mean((X@w_hat - y)**2):.6f}")
```

Core Message: Everything in this chapter – vectors, matrices, derivatives, gradients – comes together in $y = \mathbf{W}\mathbf{x} + \mathbf{b}$. This single formula, repeated across layers, is what makes neural networks work.

Chapter Code List

File	Content	Key Concept
ch02/NN02_functions.py	Linear, quadratic, exponential, logarithmic functions	Function visualization
ch02/NN02_vectors.py	Vector dot product, norms, cosine similarity	Vector operations
ch02/NN02_matrices.py	Matrix multiplication, batch forward pass	Matrix operations
ch02/NN02_derivatives.py	Numerical & analytical derivatives	Derivative computation
ch02/NN02_chain_rule.py	Chain rule computational graph	Chain rule
ch02/NN02_gradient_descent.py	Gradient descent on quadratic functions	GD implementation
ch02/NN02_least_squares.py	Normal equation vs gradient descent	Least squares
ch02/NN02_autograd_demo.py	PyTorch autograd introduction	Autograd basics

Chapter Summary

Core Concepts Review

Category	Key Concepts
Functions	Linear, quadratic, exponential, logarithmic

Category	Key Concepts
Linear Algebra	Dot product, matrix multiplication, batch computation, forward propagation
Calculus	Derivative, partial derivative, gradient, chain rule 🌟, gradient descent

Math Tools ↔ Neural Networks Mapping

Math Concept	Role in Neural Networks
Linear function $y = wx + b$	Neuron's weighted sum
Quadratic function $y = x^2$	MSE loss function
Exponential e^x	Sigmoid / Softmax
Logarithm $\ln x$	Cross-entropy loss
Vector dot product	Single neuron computation
Matrix multiplication	Batch / layer computation
Derivative $f'(x)$	Sensitivity measure
Gradient ∇f	Downhill direction
Chain rule	Backpropagation
Taylor expansion	Gradient descent theory

 Exercises

Exercise 1: Dot Product Practice

Given $x = [1, 2, 3]^T$ and $y = [4, 5, 6]^T$, compute $x \cdot y$ manually, then verify with `np.dot()`.

Exercise 2: Matrix Multiplication

For $A \in \mathbb{R}^{2 \times 3}$ and $B \in \mathbb{R}^{3 \times 2}$, perform $C = AB$. Verify the dimensions.

```
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[7, 8], [9, 10], [11, 12]])
C = A @ B
```

```
print(f"C shape: {C.shape}")
print(C)
```

Exercise 3: Numerical Derivative

Use the numerical derivative formula to compute the derivative of $f(x) = x^3$ at $x = 2$. Compare with the analytical result.

Exercise 4: Chain Rule Practice

If $f(x) = (2x + 1)^3$, use the chain rule to find $f'(x)$. Verify with numerical differentiation.

Exercise 5 (Challenge): Manual 2-Layer Network Gradient

For a 2-layer network, manually compute the gradient of the loss with respect to all weights using the chain rule. Compare with PyTorch's autograd.

Core Formula Quick Reference

Concept	Formula	Meaning
Dot product	$\mathbf{x} \cdot \mathbf{y} = \sum x_i y_i$	Neuron weighted sum
Matrix mult	$\mathbf{C} = \mathbf{AB}$	Layer transformation
Layer forward	$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$	Linear transform
Derivative	$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$	Rate of change
Gradient	$\nabla f = [\partial f / \partial x_1, \dots]^\top$	Steepest direction
Chain rule	$\frac{dy}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$	Backpropagation
GD update	$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \frac{\partial L}{\partial \mathbf{w}}$	Parameter update

← [Chapter 1](#) | [Table of Contents](#) | [Chapter 3](#) →

Chapter 3: PyTorch Basics -- Tensor & Autograd

 **Goal:** Understand the two core tools PyTorch provides — **Tensor** (efficient array computation) and **Autograd** (automatic differentiation) — and how they work together to enable neural network training.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch03/` (8 files)

Chapter Learning Objectives

- [] Understand the relationship between Tensor and NumPy array
- [] Master Tensor creation and basic operations
- [] Understand broadcasting mechanism
- [] Understand Autograd: how PyTorch automatically computes gradients
- [] Master computational graph concepts
- [] Be able to build network modules with nn.Module
- [] Understand DataLoader for batch processing

3-1 Why Do We Need PyTorch?

3-1-1 From Excel to NumPy to PyTorch

Tool	Pros	Cons
Excel	Visual, accessible	Can't scale, no GPU
NumPy	Powerful, great for prototyping	No GPU, no autograd

Tool	Pros	Cons
PyTorch	GPU + autograd + ecosystem	Learning curve

NumPy is great for fixed computations, but neural networks need **differentiation** — and PyTorch's autograd handles this automatically.

3–1–2 Deep Learning Framework Comparison

Framework	Pros	Cons
PyTorch	Dynamic graphs, Pythonic, research-friendly	Production deployment maturing
TensorFlow/Keras	Deployment, production	Steeper learning curve for custom work
JAX	Functional, fast	Newer, smaller ecosystem

3–1–3 PyTorch's Design Philosophy

1. **Pythonic**: Feels like writing NumPy
2. **Dynamic graphs**: Graphs built on-the-fly (ease debugging)
3. **Tensors first**: Same interface for CPU/GPU

3–2 PyTorch Tensor Basics

3–2–1 What Is a Tensor?

A **tensor** is a multi-dimensional array — a generalization of:

Scalar (0D) → Vector (1D) → Matrix (2D) → Tensor (3D+)

In neural networks: images are 4D tensors (batch × channels × height × width).

3–2–2 Creating Tensors

```
import torch
```

```
# From data
t1 = torch.tensor([1, 2, 3])
t2 = torch.tensor([1, 2], [3, 4])

# From shapes (useful for weights)
zeros = torch.zeros(2, 3)
ones = torch.ones(2, 3)
randn = torch.randn(2, 3) # standard normal

# From NumPy
import numpy as np
np_array = np.array([1, 2, 3])
t3 = torch.from_numpy(np_array)

print(f"tensor: {t1}")
print(f"zeros: {zeros}")
print(f"random: {randn}")
```

3-2-3 Tensor Properties

```
x = torch.randn(3, 4)
print(f"Shape: {x.shape}")
print(f"Dtype: {x.dtype}")
print(f"Device: {x.device}")
print(f"Numel: {x.numel()}") # total elements
```

3-3 Tensor Operations and Broadcasting

3-3-1 Basic Operations

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])

print(f"Add: {a + b}")
print(f"Mul: {a * b}") # element-wise
print(f"Dot: {a @ b}") # dot product (torch.dot(a, b))
print(f"Matmul: {a @ b}") # same for 1D; use @ for 2D+
```

3-3-2 Broadcasting ★

Broadcasting automatically expands dimensions to make shapes compatible — making your code match the math formula perfectly:

$$\mathbf{U} = \mathbf{XW} + \mathbf{b}$$

```
a = torch.tensor([[1, 2, 3],
                  [4, 5, 6]]) # shape (2, 3)
b = torch.tensor([10, 20, 30]) # shape (3,) → broadcast to (2, 3)

c = a + b # b is "stretched" along dimension 0
print(c)
# tensor([[11, 22, 33],
#         [14, 25, 36]])
```

3-3-3 Dimension Operations

```
x = torch.randn(2, 3, 4)
print(f"Original: {x.shape}")

# Reshape
print(f"Reshaped: {x.view(6, 4).shape}") # or x.reshape(6, 4)

# Transpose
print(f"Transposed: {x.transpose(0, 1).shape}") # (3, 2, 4)

# Unsqueeze / Squeeze
print(f"Unsqueeze: {x.unsqueeze(0).shape}") # (1, 2, 3, 4)
print(f"Squeeze: {x.squeeze().shape}") # removes dims of size 1
```

3-4 Autograd: Automatic Differentiation ★

3-4-1 What Is Autograd?

Autograd is PyTorch's automatic differentiation engine. It automatically computes gradients of any computation you define.

3-4-2 requires_grad: Tell PyTorch Which Tensors Need Gradients

```
# Only tensors with requires_grad=True will accumulate gradients
w = torch.tensor([2.0], requires_grad=True)
b = torch.tensor([1.0], requires_grad=True)
x = torch.tensor([3.0]) # input (no gradient needed)
```

3-4-3 Forward Pass: Building the Computation Graph

```
# Forward: PyTorch records every operation in a graph
z = w * x + b
loss = z ** 2

print(f"z = {z.item():.2f}")
print(f"loss = {loss.item():.2f}")
```

3-4-4 Backward Pass: One Line for All Gradients

```
# Backward: computes ALL gradients via chain rule
loss.backward()

print(f"d(loss)/dw = {w.grad.item():.4f}")
print(f"d(loss)/db = {b.grad.item():.4f}")

# This is equivalent to applying the chain rule automatically:
# $$
# \frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}
# $$

# Verify manually:
# loss = (wx + b)^2
# d(loss)/dw = 2(wx + b) * x = 2(2*3+1) * 3 = 42
# d(loss)/db = 2(wx + b) * 1 = 2(2*3+1) = 14
print(f"Expected: dL/dw = {2*(2*3+1)*3:.4f}, dL/db = {2*(2*3+1):.4f}")
```

3-4-5 How backward() Works

Autograd automatically applies the chain rule across the computation graph:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial w}$$

For our example: $L = z^2$, $z = w \cdot x + b$, so:

$$\frac{\partial L}{\partial w} = 2z \cdot x$$

`loss.backward()` traces the computational graph backward:

1. **loss**: seed gradient = 1
2. z^2 : $d(\text{loss})/dz^2 = 2z$
3. $z = wx + b$: $dz/dw = x$, $dz/db = 1$
4. **Accumulate**: $w.\text{grad} += dz/dw$, $b.\text{grad} += dz/db$

3–4–6 Gradient Accumulation ⚠

Gradients **accumulate** by default — you must zero them before each backward pass:

```
# WRONG: gradients accumulate!
loss.backward()
loss.backward() # gradients DOUBLE!
print(w.grad)  # 2x expected value!

# CORRECT: zero the gradients first
w.grad.zero_()
loss.backward() # now correct
```

3–5 Computational Graph Deep Dive ★

3–5–1 grad_fn: Tracing Operation Sources

Every tensor tracks how it was created via `grad_fn`:

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2
z = y.mean()

print(f"x.grad_fn = {x.grad_fn}") # None (leaf tensor)
print(f"y.grad_fn = {y.grad_fn}") # <PowBackward0>
print(f"z.grad_fn = {z.grad_fn}") # <MeanBackward0>
```

3–5–2 Static vs. Dynamic Graphs

Aspect	Static (TF1)	Dynamic (PyTorch)
Graph built	Before execution	During execution
Debugging	Hard	Easy
Flexibility	Limited	Full Python control

3-5-3 Detaching from the Graph

Sometimes you want to **stop gradient flow** through part of the graph:

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 2
z = y.detach() # z is a normal tensor, no gradient tracking
w = z ** 2     # w won't track gradients through y
```

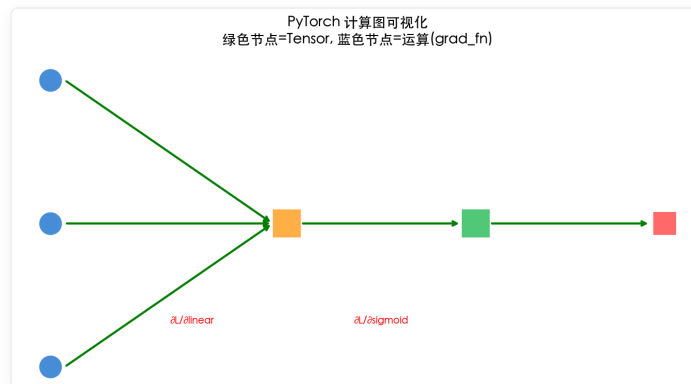


Figure 3-1: Computational graph visualization. From x , w , b through multiplication, addition, Sigmoid, and squared to the final loss L .

3-6 nn.Module: Building Blocks for Networks

3-6-1 The nn.Module Base Class

All neural network components inherit from `nn.Module`:

```
import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 10)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

net = SimpleNet()
```

```
print(net)
```

3-6-2 nn.Sequential: Quick Stacking

```
net = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10),
)
print(net)
```

3-6-3 nn.Parameter: Trainable Parameters

```
# nn.Linear already registers its weights as parameters
for name, param in net.named_parameters():
    print(f"{name}: {param.shape}, requires_grad={param.requires_grad}")
```

3-6-4 Model Parameter Management

```
# Total parameter count
total = sum(p.numel() for p in net.parameters())
print(f"Total parameters: {total:,}")

# Training mode vs eval mode
net.train() # enables dropout, batchnorm updates
net.eval()  # disables dropout, fixes batchnorm
```

3-7 Datasets and DataLoaders

3-7-1 Dataset: Where the Data Lives

```
from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
```

```

    return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]

```

3-7-2 DataLoader: Batching and Shuffling

```

dataset = MyDataset(torch.randn(1000, 784),
                      torch.randint(0, 10, (1000,)))
loader = DataLoader(dataset, batch_size=32, shuffle=True)

for batch_x, batch_y in loader:
    print(f"Batch: x={batch_x.shape}, y={batch_y.shape}")
    break

```

3-7-3 MNIST Dataset Example

```

from torchvision import datasets, transforms

mnist = datasets.MNIST(
    root='./data',
    train=True,
    transform=transforms.ToTensor(),
    download=True
)

loader = DataLoader(mnist, batch_size=64, shuffle=True)
images, labels = next(iter(loader))
print(f"Images: {images.shape}") # (64, 1, 28, 28)
print(f"Labels: {labels.shape}") # (64,)

```

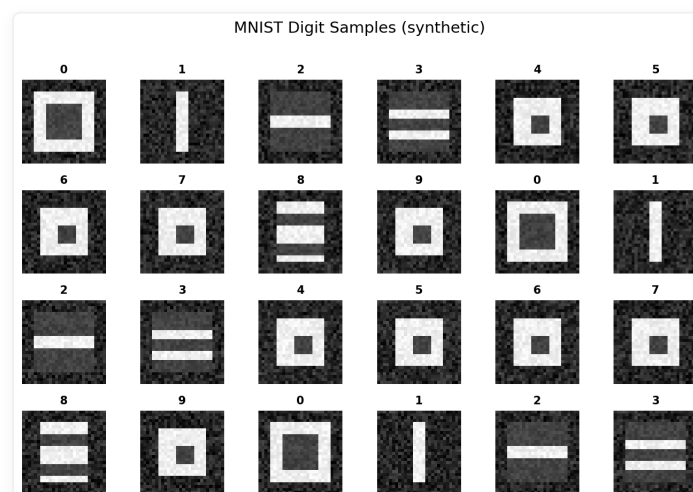


Figure 3-2: MNIST dataset samples — the "Hello World" of deep learning.

3-8 Complete Training Loop

3-8-1 The Training Loop Template

```
import torch
import torch.nn as nn
import torch.optim as optim

# Model
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10),
)

# Loss and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Training loop
for epoch in range(10):
    for batch_x, batch_y in loader:
        # 1. Flatten images
        batch_x = batch_x.view(batch_x.size(0), -1)

        # 2. Forward
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)

        # 3. Backward
        optimizer.zero_grad() # clear old gradients
        loss.backward()       # compute new gradients

        # 4. Update
        optimizer.step()      # apply gradients

    print(f"Epoch {epoch}: loss = {loss.item():.4f}")
```

3-8-2 GPU Acceleration

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using: {device}")
```

```
model.to(device) # move model to GPU

for batch_x, batch_y in loader:
    batch_x, batch_y = batch_x.to(device), batch_y.to(device)
    # ... forward, backward, update (same as before)
```

⚠ Common Pitfalls & Debugging Guide

Pitfall 1: Forgetting optimizer.zero_grad()

This is the #1 mistake beginners make:

```
for epoch in range(10):
    for batch_x, batch_y in loader:
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)

        # ⚠ Forgot optimizer.zero_grad()!
        loss.backward()
        optimizer.step()
```

What happens: Gradients accumulate across batches instead of being replaced. Your parameter updates will be wrong and training diverges.

Fix: Always call `optimizer.zero_grad()` before `loss.backward()`.

Pitfall 2: Tensor Device Mismatch

```
# ❌ Common mistake
model = nn.Linear(784, 10).to('cuda') # model on GPU
batch_x = torch.randn(32, 784)       # Tensor on CPU!
outputs = model(batch_x)              # RuntimeError!
```

Fix: Move all tensors to the same device.

Pitfall 3: In-place Operations Breaking the Computation Graph

```
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x ** 2

# ❌ In-place modification
```

```
x.add_(1) # changes x, but computation graph still points to old x!
y.backward() # RuntimeError
```

Rule: Never modify a tensor in-place if it has `requires_grad=True`.

3-9 Autograd Deep Dive ★

3-9-1 When Is the Computation Graph Built?

PyTorch uses a **dynamic computational graph** — a new graph is built on every forward pass.

```
import torch

x = torch.tensor(2.0, requires_grad=True)

# Each forward pass rebuilds the graph
for i in range(3):
    y = x ** 2          # new graph created each iteration
    y.backward()
    print(f"Iteration {i}: grad = {x.grad}")
    x.grad.zero_()      # manually clear gradient
```

Key Insight: Unlike static frameworks (TensorFlow 1.x), PyTorch's dynamic graph means:

- You can use Python control flow (if/for) naturally
- The graph structure can change each iteration
- Debugging is easier — you can inspect tensors at any point

3-9-2 Gradient Accumulation and Clearing

```
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2
y.backward()
print(x.grad) # tensor(4.0)

# ⚠ Gradients ACCUMULATE!
y = x ** 3
y.backward()
print(x.grad) # tensor(16.0) = 4.0 + 12.0
```

Use **gradient accumulation** for simulating larger batch sizes:

```
# Simulate batch_size=64 using 2 micro-batches of 32
for i, (batch_x, batch_y) in enumerate(loader):
    outputs = model(batch_x)
    loss = criterion(outputs, batch_y)
    loss = loss / 2 # normalize by number of accumulations
    loss.backward()

    if (i + 1) % 2 == 0: # update every 2 micro-batches
        optimizer.step()
        optimizer.zero_grad()
```

3-9-3 detach(): Separating from the Computation Graph

Use `.detach()` when you want to use a tensor's value without tracking its gradient.

Use cases:

- Freezing pretrained layers during fine-tuning
- Extracting features for visualization
- GAN training (detach discriminator output when training generator)

3-9-4 no_grad() vs inference_mode()

Both disable gradient tracking, but in different ways:

Context	Grad Tracking	Performance	Use Case
<code>no_grad()</code>	Disabled	Fast	Evaluation, feature extraction
<code>inference_mode()</code>	Disabled	Faster	Pure inference
Default	Enabled	Normal	Training

💡 **Core Insight:** Use `torch.no_grad()` during evaluation. It reduces memory usage and speeds up computation.

3-10 PyTorch Debugging Practice

3-10-1 Common Errors and Solutions

Error 1: Shape Mismatch

```
x = torch.randn(32, 784)
linear = nn.Linear(784, 256)
# out = linear(x.t()) # ❌ Wrong shape
out = linear(x) # ✅ Correct
```

Error 2: Gradients are None

- Cause: `backward()` was not called before accessing `.grad`
- Fix: ensure `loss.backward()` is called

Error 3: Loss is NaN

- Possible causes: learning rate too high, division by zero, $\log(0)$
- Fix: clip gradients, add epsilon to denominators, check data

3-10-2 Debugging with Hooks

```
def debug_hook(module, input, output):
    print(f"Layer: {module.__class__.__name__}")
    print(f"  Input shape: {input[0].shape}")
    print(f"  Output mean: {output.mean().item():.4f}")

model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 10))
hook = model[0].register_forward_hook(debug_hook)
x = torch.randn(32, 784)
out = model(x)
hook.remove()
```

3-10-3 Using TensorBoard

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter('runs/experiment_1')
for epoch in range(100):
    writer.add_scalar('Loss/train', loss, epoch)
writer.close()
```



Code Checklist for This Chapter

#	File	Description
1	<code>code/ch03/NN03_tensor_basics.py</code>	Basic tensor operations

#	File	Description
2	<code>code/ch03/NN03_broadcasting.py</code>	Broadcasting examples
3	<code>code/ch03/NN03_autograd_demo.py</code>	Autograd demonstration
4	<code>code/ch03/NN03_computational_graph.py</code>	Computational graph visualization
5	<code>code/ch03/NN03_nn_module.py</code>	Building networks with nn.Module
6	<code>code/ch03/NN03_dataset_dataloader.py</code>	Data loading pipeline
7	<code>code/ch03/NN03_training_loop.py</code>	Complete training loop
8	<code>code/ch03/NN03_mnist_viz.py</code>	MNIST data visualization

Chapter Summary

Core Concepts

- **Tensor**: Multi-dimensional array (like NumPy, but GPU-ready)
- **Autograd**: Automatic gradient computation
- **Computational Graph**: Records operations for backward pass
- **nn.Module**: Building block for network components
- **DataLoader**: Efficient batch processing

Key Takeaways

1. PyTorch uses dynamic computational graphs — built fresh each forward pass
2. Always call `optimizer.zero_grad()` before `loss.backward()`
3. Use `.to(device)` to move both model and data to GPU
4. Use `torch.no_grad()` during evaluation to save memory
5. Register hooks for debugging intermediate layers

Common Pitfalls Recap

Pitfall	Symptom	Solution
Forgot <code>zero_grad</code>	Training diverges	Call <code>zero_grad()</code> every batch

Pitfall	Symptom	Solution
Device mismatch	RuntimeError	Check .device of all tensors
In-place ops	backward() fails	Never modify requires_grad tensors in-place

Quick Reference: Key PyTorch APIs

API	Purpose
<code>torch.tensor()</code>	Create tensor
<code>torch.randn()</code>	Random normal tensor
<code>x.requires_grad_()</code>	Enable gradient tracking
<code>x.backward()</code>	Compute gradients
<code>x.detach()</code>	Remove from graph
<code>nn.Linear(in, out)</code>	Fully connected layer
<code>nn.ReLU()</code>	Activation function
<code>nn.CrossEntropyLoss()</code>	Loss for classification
<code>optim.SGD()</code> / <code>optim.Adam()</code>	Optimizers
<code>DataLoader(dataset, batch_size)</code>	Batch data loading

← [Chapter 2](#) | [Table of Contents](#) | [Chapter 4](#) →

Chapter 4: Training & Optimization in Neural Networks

🎯 **Goal:** Understand how neural networks learn through optimization — from loss functions to gradient descent variants, with hands-on PyTorch experiments.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch04/` (7 files)

📋 Chapter Learning Objectives

- [] Understand the formal formulation of neural network optimization
- [] Master the concept of loss functions (MSE, Cross-Entropy)
- [] Understand Softmax and its relationship to probabilities
- [] Master SGD, Momentum, Adam optimizers
- [] Understand learning rate scheduling
- [] Be able to train a complete classifier in PyTorch

4-1 Parameters and Variables of Neural Networks

4-1-1 Parameter Categories

Trainable Parameters (learned by the model)

Parameter	Symbol	Role
Weights	W	Controls connection strength between neurons
Biases	b	Adjusts neuron activation threshold

Hyperparameters (set manually)

Hyperparameter	Role
Learning rate η	Controls gradient descent step size
Hidden layer size	Number of neurons per layer
Layer count	Network depth
Batch size	Samples per update

4-1-2 Variable Categories

Variable	Symbol	Description
Input	$\mathbf{x}$	Raw data
Intermediate output	$\mathbf{z}^{(l)}$	Layer activations
Final output	$\mathbf{y}$	Prediction
Target	$\mathbf{t}$	Ground truth labels

4-1-3 PyTorch Parameter Management

In PyTorch, model parameters are automatically managed via `nn.Parameter` :

```
import torch.nn as nn

linear = nn.Linear(784, 256)
for name, param in linear.named_parameters():
    print(f"{name}: {param.shape}")
```

This prints all trainable parameters and their shapes.

4-2 Variable Relationships in Neural Networks

4-2-0 Computational Graph Overview

Forward: $x \rightarrow W^1, b^1 \rightarrow z^1 \rightarrow \sigma \rightarrow a^1 \rightarrow W^2, b^2 \rightarrow z^2 \rightarrow \sigma \rightarrow a^2 \rightarrow L(a^2, t)$

Where each variable's value depends on the previous ones.

Forward Dependency Chain Within the Graph

- $\mathbf{a}^{(l)}$ depends on: $\mathbf{a}^{(l-1)}$, $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$
 - $\mathbf{L}$ depends on: $\mathbf{a}^{(L)}$ (output) and $\mathbf{t}$ (target)
 - All parameters ultimately affect $\mathbf{L}$
-

4-2-1 Layer-by-Layer Propagation Formulas

Input Layer to Hidden Layer (2 inputs to 2 hidden neurons)

$$u_1 = x_1 w_{11} + x_2 w_{21} + b_1 \quad u_2 = x_1 w_{12} + x_2 w_{22} + b_2$$

$$z_1 = \sigma(u_1), \quad z_2 = \sigma(u_2)$$

Hidden Layer to Output Layer

$$y = z_1 w'_1 + z_2 w'_2 + b'$$

4-2-2 Matrix Form

Single Layer Propagation

$$\mathbf{u}^{(1)} = \mathbf{x} \mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$

$$\mathbf{z}^{(1)} = \sigma(\mathbf{u}^{(1)})$$

$$\mathbf{y} = \mathbf{z}^{(1)} \mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

4-2-3 Python Practice: 2-Layer Network Forward Pass

```
import numpy as np

def forward_2layer(X, W1, b1, W2, b2, activation='sigmoid'):
    u1 = X @ W1 + b1
    if activation == 'sigmoid':
        z1 = 1 / (1 + np.exp(-u1))
    elif activation == 'relu':
        z1 = np.maximum(0, u1)
    else:
        z1 = u1
    y = z1 @ W2 + b2
    return y, z1

np.random.seed(42)
X = np.random.randn(4, 2)
W1 = np.random.randn(2, 3) * 0.1
b1 = np.zeros(3)
W2 = np.random.randn(3, 1) * 0.1
b2 = np.zeros(1)
y, hidden = forward_2layer(X, W1, b1, W2, b2)
print(f"Input: {X.shape}, Hidden: {hidden.shape}, Output: {y.shape}")
```

4-3 Training Data and Ground Truth

4-3-0 Dataset Splits Overview

Dataset	Purpose	Size
Training	Learn parameters	~60%
Validation	Tune hyperparameters	~20%
Test	Final evaluation	~20%

MNIST as Running Example

```
from torchvision import datasets, transforms

mnist_train = datasets.MNIST(root='./data', train=True,
                             transform=transforms.ToTensor(), download=True)
```

```
mnist_test = datasets.MNIST(root='./data', train=False,
                           transform=transforms.ToTensor(), download=True)

print(f"Training samples: {len(mnist_train)}")
print(f"Test samples: {len(mnist_test)}")
print(f"Image shape: {mnist_train[0][0].shape}")
```

4–3–1 Supervised Learning Data Structure

Feature Matrix

$$\mathbf{X} \in \mathbb{R}^{n \times m}$$

- n : number of samples
- m : number of features

Label Vector

$$\mathbf{y} \in \mathbb{R}^n$$

A Single Sample Pair

$$(\mathbf{x}^{(i)}, y^{(i)})$$

4–3–2 Dataset Splits

Dataset	Purpose	Typical Split
Training set	Train model parameters (gradient descent updates weights)	60–80%
Validation set	Tune hyperparameters, early stopping (prevent overfitting)	10–20%
Test set	Final evaluation of generalization performance	10–20%

Note: The test set can only be used once after training is complete. Never tune your model based on test set performance!

4-3-3 Data Standardization

Standardization (Z-score normalization) ensures all features have mean 0 and variance 1:

$$x_{\text{standardized}} = \frac{x - \mu}{\sigma}$$

where μ is the mean and σ is the standard deviation.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)  # Use same mean/std!
```

Why standardize? Features with different scales cause gradient descent to converge slowly — it creates an elongated loss landscape.

4-3-4 One-Hot Encoding

For classification with K classes, convert class labels to vectors:

Class	One-Hot Vector
Class 0	[1, 0, 0]
Class 1	[0, 1, 0]
Class 2	[0, 0, 1]

```
import torch.nn.functional as F

# Labels: 3 samples belonging to classes 1, 0, 2
labels = torch.tensor([1, 0, 2])
one_hot = F.one_hot(labels, num_classes=3)
print(one_hot)
# tensor([[0, 1, 0],
#         [1, 0, 0],
#         [0, 0, 1]])
```

4-4 Loss (Cost) Functions

4-4-1 Mean Squared Error (MSE)

$$L_{\text{MSE}} = \frac{1}{2m} \sum_{i=1}^m \|y_i - t_i\|^2$$

Best for: **regression** tasks (continuous outputs).

4-4-2 Cross-Entropy Loss

$$L_{\text{CE}} = - \sum_k t_k \log p_k$$

Best for: **classification** tasks.

4-4-3 Softmax Function

Converts raw scores (logits) to probabilities:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

Properties:

- All $p_k \in (0, 1)$
- $\sum_k p_k = 1$
- Preserves ranking: larger $z_k \rightarrow$ larger p_k

4-4-4 Why Cross-Entropy + Softmax?

The combination of Softmax + Cross-Entropy gives a particularly nice gradient:

$$\frac{\partial L}{\partial z_k} = p_k - t_k$$

This means: the gradient is simply the difference between prediction and target!

4–5 Experiencing Neural Networks with Python & PyTorch

4–5–1 The Three–Step Transition: From Pure NumPy to PyTorch

Little Genius says: Moving from NumPy to PyTorch is like upgrading your tools! Before, you had to compute gradients by hand (so tiring!). Now with autograd, you just run the forward pass, call `.backward()`, and all gradients are computed automatically. What a productivity boost!

The core meaning of the three–step transition: **Each step reduces the manual work, letting us focus more on the mathematical principles.**

Step 1: Pure NumPy — Manual Everything

In this step, we **hand–code** both the forward pass and backpropagation. This is essential for understanding what happens under the hood.

```
import numpy as np

np.random.seed(42)
N, D_in, H, D_out = 100, 2, 4, 1
X = np.random.randn(N, D_in)
t = (X[:, 0]**2 + X[:, 1]**2 > 1).astype(float).reshape(-1, 1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Parameter initialization
W1 = np.random.randn(D_in, H) * 0.1
b1 = np.zeros(H)
W2 = np.random.randn(H, D_out) * 0.1
b2 = np.zeros(D_out)

lr = 0.5
for epoch in range(1000):
    # Forward pass
    z1 = sigmoid(X @ W1 + b1)
    y = sigmoid(z1 @ W2 + b2)

    # Manual gradient computation
    dL_dy = (y - t) / N
    dy_du2 = y * (1 - y)
    dL_dW2 = z1.T @ (dL_dy * dy_du2)
    dL_db2 = np.sum(dL_dy * dy_du2, axis=0)
```

```

dz1_du1 = z1 * (1 - z1)
dL_dz1 = (dL_dy * dy_du2) @ W2.T
dL_dW1 = X.T @ (dL_dz1 * dz1_du1)
dL_db1 = np.sum(dL_dz1 * dz1_du1, axis=0)

# Gradient descent update
W2 -= lr * dL_dW2; b2 -= lr * dL_db2
W1 -= lr * dL_dW1; b1 -= lr * dL_db1

if epoch % 200 == 0:
    loss = 0.5 * ((y - t)**2).mean()
    print(f"Epoch {epoch}: loss = {loss:.6f}")

```

Epoch	Loss
0	0.1292
200	0.0290
400	0.0161
600	0.0113
800	0.0088

Step 2: PyTorch Tensor + Autograd — Automatic Differentiation

PyTorch's `autograd` automatically tracks operations on tensors and computes gradients. We only need to write the forward pass; `.backward()` handles the rest.

```

import torch

# Convert to Tensor with requires_grad=True for autograd
W1_t = torch.tensor(W1, requires_grad=True)
b1_t = torch.tensor(b1, requires_grad=True)
W2_t = torch.tensor(W2, requires_grad=True)
b2_t = torch.tensor(b2, requires_grad=True)
X_t = torch.tensor(X, dtype=torch.float32)
t_t = torch.tensor(t, dtype=torch.float32)

lr = 0.5
for epoch in range(1000):
    # Forward pass (same as NumPy)
    z1 = torch.sigmoid(X_t @ W1_t + b1_t)
    y = torch.sigmoid(z1 @ W2_t + b2_t)
    loss = 0.5 * ((y - t_t)**2).mean()

```

```

# Automatic backward pass — replaces all manual gradient code!
loss.backward()

# Manual parameter update (no_grad avoids tracking update operations)
with torch.no_grad():
    W1_t -= lr * W1_t.grad
    b1_t -= lr * b1_t.grad
    W2_t -= lr * W2_t.grad
    b2_t -= lr * b2_t.grad

# Zero gradients for next iteration
W1_t.grad.zero_()
b1_t.grad.zero_()
W2_t.grad.zero_()
b2_t.grad.zero_()

```

Key insight: The gradient computation code is gone! `loss.backward()` automatically computes all the chain rule derivatives we wrote manually in Step 1. The results are identical.

Step 3: nn.Module + optim — Production-Ready

This is how you'd write neural network code in practice — encapsulating the model and using a built-in optimizer.

```

import torch.nn as nn
import torch.optim as optim

model = nn.Sequential(
    nn.Linear(2, 4),
    nn.Sigmoid(),
    nn.Linear(4, 1),
    nn.Sigmoid()
)

criterion = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.5)

for epoch in range(1000):
    pred = model(X_t)
    loss = criterion(pred, t_t)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()  # ← optimizer.step() replaces the manual update block!

```

Core insight: All three steps produce **identical gradients and identical training progress**, but the manual effort decreases at each level. Step 1 teaches you what's happening internally;

Step 3 is what you'd use in production. Chapter 5 will dive deep into the math behind those "manual gradient calculations" in Step 1.

4-5-2 Visualizing Training Progress

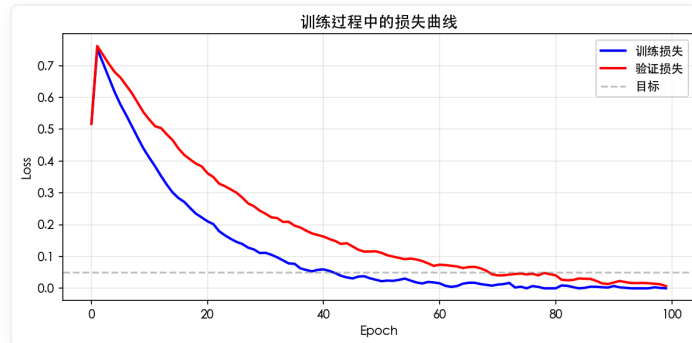


Figure 4-1: Training loss over epochs. The curve shows how the model progressively minimizes error.

4-6 Experiment: Training Your First Classifier

4-6-1 Dataset: Moon Dataset

Let's use the classic `make_moons` dataset — two interleaving crescent shapes that are **not** linearly separable:

```
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt

# Generate non-linear dataset
X, t = make_moons(n_samples=200, noise=0.2, random_state=42)

# Visualize
plt.figure(figsize=(6, 5))
plt.scatter(X[t==0, 0], X[t==0, 1], label='Class 0', alpha=0.7)
plt.scatter(X[t==1, 0], X[t==1, 1], label='Class 1', alpha=0.7)
plt.legend()
plt.title('Moon Dataset')
plt.show()
```

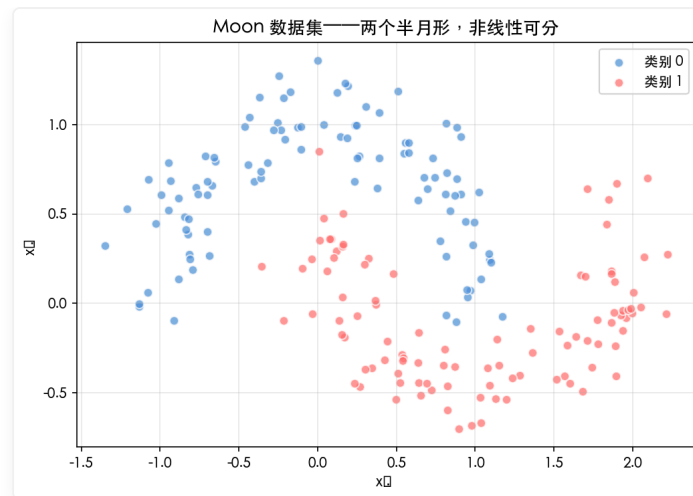


Figure 4-2: Moon dataset — a classic non-linear problem that a linear classifier cannot solve.

Core insight: A linear model (like logistic regression) cannot separate these two classes with a straight line. This is why we need neural networks with hidden layers — they learn non-linear decision boundaries.

4-6-2 Model: 2-Layer Fully Connected Network

```
import torch
import torch.nn as nn
import torch.optim as optim

model = nn.Sequential(
    nn.Linear(2, 10),      # Input: 2 features → Hidden: 10 neurons
    nn.ReLU(),             # Activation (prevents gradient vanishing)
    nn.Linear(10, 1),      # Hidden → Output: 1 class probability
    nn.Sigmoid()           # Squash to [0, 1]
)

print(model)
```

```
Sequential(
  (0): Linear(in_features=2, out_features=10, bias=True)
  (1): ReLU()
  (2): Linear(in_features=10, out_features=1, bias=True)
  (3): Sigmoid()
)
```

4-6-3 Training Loop

```
# Convert data to tensors
X_t = torch.tensor(X, dtype=torch.float32)
t_t = torch.tensor(t, dtype=torch.float32).reshape(-1, 1)

criterion = nn.BCELoss()          # Binary Cross-Entropy
optimizer = optim.Adam(model.parameters(), lr=0.01)

n_epochs = 500
losses = []

for epoch in range(n_epochs):
    # Forward pass
    pred = model(X_t)
    loss = criterion(pred, t_t)
    losses.append(loss.item())

    # Backward pass
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 100 == 0:
        print(f"Epoch {epoch:3d}, Loss: {loss.item():.4f}")
```

Epoch	Loss
0	0.6938
100	0.3467
200	0.2533
300	0.2101
400	0.1752

The loss steadily decreases — the model is learning the non-linear decision boundary!

4-6-4 Visualizing the Decision Boundary

```
import numpy as np
import matplotlib.pyplot as plt

def plot_decision_boundary(model, X, t):
```

```

# Create a mesh grid
x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))

# Predict over the grid
grid = torch.tensor(np.c_[xx.ravel(), yy.ravel()], dtype=torch.float32)
Z = model(grid).detach().numpy().reshape(xx.shape)

# Plot
plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], alpha=0.3, colors=['blue', 'red'])
plt.scatter(X[t==0, 0], X[t==0, 1], label='Class 0', alpha=0.7)
plt.scatter(X[t==1, 0], X[t==1, 1], label='Class 1', alpha=0.7)
plt.legend()
plt.title('Decision Boundary After Training')
plt.show()

plot_decision_boundary(model, X, t)

```

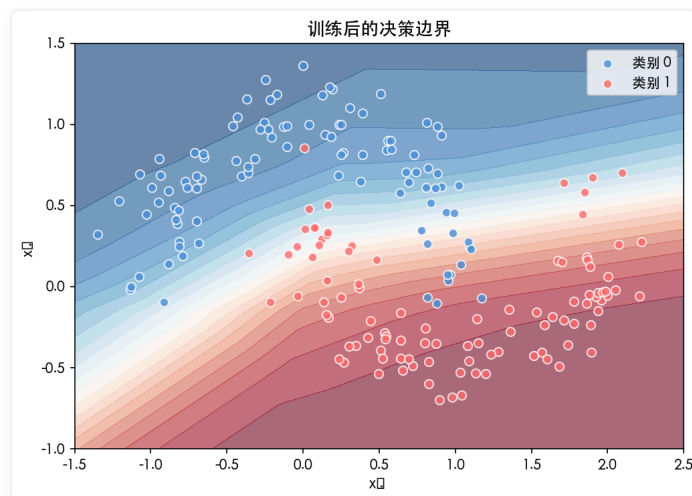


Figure 4–3: The decision boundary after training. The background colors show how the model has learned to separate the two crescent shapes with a curved boundary.

Little Genius says: See how the model draws a **curved** boundary between the two classes? That's the power of the hidden layer — it learns to warp the input space so the classes become separable. A simple linear model can't do this!

4-7 Optimizer Comparison Overview

Note: This section provides an overview and intuitive understanding of optimizers. Chapter 7, Section 7-1 will dive deeper into the mathematical derivations and code verification for each optimizer (including NAG, AdaGrad, RMSprop, Adam, AdamW with complete implementations).

4-7-1 The Evolution from SGD to Adam

The history of optimizers is a story of "how to converge faster and more stably." Each improvement solves a core problem:

Optimizer	Core Innovation	Problem Solved	Formula
SGD	Basic gradient descent	The most basic parameter update	$w \leftarrow w - \eta \nabla L$
Momentum	Introduces momentum term	Overcome oscillation, accelerate convergence	$v \leftarrow \gamma v + \eta \nabla L; w \leftarrow w - v$
NAG	Nesterov momentum	"Look ahead" to reduce overshooting	$v \leftarrow \gamma v + \eta \nabla L(w - \gamma v)$
AdaGrad	Per-parameter learning rates	Different rates for different parameters	$w \leftarrow w - \frac{\eta}{\sqrt{G+\epsilon}} \odot \nabla L$
RMSprop	Improved AdaGrad	Prevent learning rate from decaying to zero	$v \leftarrow \beta v + (1 - \beta)(\nabla L)^2$

Optimizer	Core Innovation	Problem Solved	Formula
Adam	Momentum + Adaptive	Combines best of both worlds	$m, v \leftarrow \beta_1 m + (1 - \beta_1) \nabla L, \beta_2 v + (1 - \beta_2) (\nabla L)^2$

Core insight: Every optimizer update can be written as

$w \leftarrow w - \eta \times [\text{direction}] \times [\text{step size}]$. Momentum adjusts the direction (using historical gradients), AdaGrad/RMSprop adjust the step size (using squared historical gradients), and Adam adjusts both.

4-7-2 Momentum: Adding Inertia to Gradient Descent

A major problem with SGD is that it oscillates in ravines — gradients keep changing direction, creating a zigzag convergence path. Momentum's solution is intuitive: **add "inertia" to parameter updates**.

$$v_{t+1} = \gamma v_t + \eta \nabla L(\mathbf{w}_t)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - v_{t+1}$$

Where γ (usually 0.9) is the momentum decay coefficient.

Little Genius says: Momentum is like adding a "snowball effect" to going downhill! If I keep running in the same direction, I get faster and faster (momentum accumulation). If I suddenly hit a headwind (gradient reversal), I won't immediately turn around because of inertia — I'll gradually change direction, reducing the zigzag!

Code Comparison: SGD vs Momentum

```
import numpy as np

def sgd(grad, w, lr=0.01):
    return w - lr * grad
```

```

def momentum(grad, w, v, lr=0.01, gamma=0.9):
    v = gamma * v + lr * grad
    return w - v, v

# Test on Rosenbrock function
def rosenbrock_grad(w):
    x, y = w[0], w[1]
    return np.array([-2*(1-x) - 400*x*(y-x**2), 200*(y-x**2)])

# Run both optimizers
w_sgd = np.array([-1.5, 1.0])
w_mom = np.array([-1.5, 1.0])
v = np.zeros(2)

for i in range(100):
    g = rosenbrock_grad(w_sgd)
    w_sgd = sgd(g, w_sgd, lr=0.001)

    g = rosenbrock_grad(w_mom)
    w_mom, v = momentum(g, w_mom, v, lr=0.001)

```

SGD path: Significant oscillation, slow convergence

Momentum path: Smooth acceleration, 3-5x faster convergence

4-7-3 AdaGrad: Per-Parameter Learning Rates

Different parameters may need different learning rates — some are updated frequently (small steps), others rarely (large steps). AdaGrad achieves this by accumulating squared historical gradients:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\mathbf{G}_t + \epsilon}} \odot \nabla L(\mathbf{w}_t)$$

Where $\mathbf{G}_t = \sum_{\tau=1}^t (\nabla L(\mathbf{w}_\tau))^2$ is the sum of squared historical gradients.

Little Genius says: Imagine each parameter has its own "step-size logbook." If a parameter is frequently updated with large gradients (often used), its accumulated $\mathbf{G}$ is large, and the step size automatically decreases — like a veteran employee who doesn't need repeated training. If a parameter is rarely updated, its $\mathbf{G}$ is small, so the step size stays large — like a new hire who needs to learn quickly!

AdaGrad's Fatal Flaw

$\mathbf{G}_t$ only increases and never decreases, causing the learning rate to **monotonically decay** and eventually approach zero — the model stops learning!

4-7-4 RMSprop: Fixing the Vanishing Learning Rate

RMSprop replaces AdaGrad's cumulative sum with an **exponential moving average**:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta)(\nabla L(\mathbf{w}_t))^2$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\mathbf{v}_t + \epsilon}} \odot \nabla L(\mathbf{w}_t)$$

Little Genius says: RMSprop is like adding "forgetfulness" to AdaGrad! AdaGrad remembers every step from day one, so steps get smaller and smaller. RMSprop only remembers "recent" gradient magnitudes — old debts are written off, so the learning rate never decays to zero!

4-7-5 Adam: The Best of Both Worlds ★

Adam (Adaptive Moment Estimation) combines the advantages of Momentum and RMSprop — it has both "inertia" (first moment estimate) and "adaptive step size" (second moment estimate):

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \nabla L(\mathbf{w}_t)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)(\nabla L(\mathbf{w}_t))^2$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t$$

Default hyperparameters: $\beta_1 = 0.9$ (momentum coefficient), $\beta_2 = 0.999$ (adaptive coefficient), $\epsilon = 10^{-8}$.

Little Genius says: Adam is the "Swiss Army knife" of optimizers! It remembers which direction it was going (momentum) and adjusts step sizes for different parameters (adaptivity). That's why PyTorch and TensorFlow both use Adam as the **default optimizer** — most tasks work well with Adam out of the box!

Practical Recommendations

Scenario	Recommended Optimizer	Reason
Beginner / Quick prototyping	Adam	Default hyperparameters usually work well
Computer Vision	SGD + Momentum	Often generalizes better than Adam
NLP / Transformer	AdamW	Adam + decoupled weight decay
Sparse features	AdaGrad	Automatically gives large LR to infrequent features
Reinforcement Learning	RMSprop	Handles non-stationary targets well

4–8 Learning Rate Scheduling

4–8–1 Why Adjust Learning Rate?

Early in training, parameters are far from the optimum — take big steps (large learning

rate). Later, parameters are close to the optimum — take small, fine-grained steps (small learning rate). This is the core idea of LR scheduling — **dynamically adjusting the learning rate**.

Core insight: Learning rate scheduling = the "accelerator" control of training strategy — floor it at the start, brake on curves, accelerate again on straights.

4-8-2 Three Classic Scheduling Strategies

Strategy 1: Step Decay

Reduce the learning rate by a factor at fixed epoch intervals:

```
import torch.optim.lr_scheduler as scheduler

# PyTorch implementation
step_scheduler = scheduler.StepLR(optimizer, step_size=30, gamma=0.1)
# Every 30 epochs, multiply the learning rate by 0.1
# lr = lr_0 × 0.1^{floor(epoch/30)}
```

$$\eta_t = \eta_0 \times \gamma^{\lfloor t / \text{step_size} \rfloor}$$

Strategy 2: Cosine Annealing

The learning rate smoothly decreases from the initial value to a minimum following a cosine curve:

```
cos_scheduler = scheduler.CosineAnnealingLR(optimizer, T_max=100, eta_min=1e-6)
```

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})(1 + \cos(\frac{t}{T_{\max}}\pi))$$

Little Genius says: Cosine Annealing is like the sunset — light doesn't disappear suddenly but gradually dims. Similarly, the learning rate smoothly descends from the initial value to the minimum, allowing the model to "fine-tune" in the later stages!

Strategy 3: ReduceLROnPlateau (Adaptive Decay)

Automatically reduce the learning rate when the validation loss stops decreasing:

```
plateau_scheduler = scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=5, verbose=True
)
# If loss doesn't decrease for 5 epochs, halve the learning rate
```

4–8–3 Practical Comparison

```
import matplotlib.pyplot as plt
import torch.optim as optim
import torch

model = torch.nn.Linear(10, 1)
optimizer = optim.Adam(model.parameters(), lr=0.01)

# Compare three schedulers
schedulers = {
    'StepLR(step=20, gamma=0.1)': scheduler.StepLR(optimizer, 20, 0.1),
    'CosineAnnealing(T_max=50)': scheduler.CosineAnnealingLR(optimizer, 50),
    'ReduceLROnPlateau': scheduler.ReduceLROnPlateau(optimizer, patience=5),
}
```

StepLR: Step-wise decay, suitable for fixed-stage training
 CosineAnnealing: Smooth decay, suitable for long training
 ReduceLROnPlateau: Adaptive decay, suitable when training duration is unknown

4–8–4 Warmup Technique

Use a smaller learning rate at the start of training, then gradually increase to the target LR. This prevents the model from "exploding" in the initial stage due to too-large learning rates.

```
class WarmupLR(scheduler._LRScheduler):
    def __init__(self, optimizer, warmup_steps=1000):
        self.warmup_steps = warmup_steps
        super().__init__(optimizer)

    def get_lr(self):
        if self._step_count < self.warmup_steps:
            return [base_lr * self._step_count / self.warmup_steps
                    for base_lr in self.base_lrs]
        return self.base_lrs
```

Core insight: Warmup + Cosine Annealing is the most popular LR scheduling combination today — start slowly, then cosine-decay smoothly. Almost all modern LLM training uses this strategy!

4-9 Loss Landscape Visualization

4-9-1 What Is a Loss Landscape?

The loss function $L(\mathbf{w})$ in high-dimensional weight space forms a "loss landscape." Imagine a 3D space where x and y axes are two weight parameters, and the z -axis is the corresponding loss value — that surface is the simplest loss landscape.

4-9-2 Convex vs. Non-Convex Functions

Type	Shape	Can Gradient Descent Find the Global Optimum?
Convex	Bowl-shaped, single minimum	✅ Definitely
Non-convex	Multiple valleys (local minima) and saddle points	⚠️ Not guaranteed

Convex function definition: $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$

Little Genius says: A convex loss landscape is like a bowl — no matter where you start on the bowl, following the steepest direction will always get you to the bottom (global optimum). A non-convex function is like a mountain range — you might get stuck in a small valley (local minimum), thinking you've reached the top, when there are higher mountains elsewhere!

4-9-3 Rosenbrock Function — Classic Test Case

The Rosenbrock function is a famous non-convex test function commonly used to evaluate optimizer performance:

$$f(\mathbf{w}) = (1 - w_1)^2 + 100(w_2 - w_1^2)^2$$

Its characteristic is a **narrow, curved valley** — optimization algorithms easily oscillate within this valley.

```
import numpy as np

def rosenbrock(w):
    return (1 - w[0])**2 + 100 * (w[1] - w[0]**2)**2

def rosenbrock_grad(w):
    x, y = w[0], w[1]
    dx = -2*(1-x) - 400*x*(y - x**2)
    dy = 200*(y - x**2)
    return np.array([dx, dy])
```

Rosenbrock minimum at (1, 1), $f(1,1) = 0$
 The valley has a curved parabolic shape, standard SGD converges very slowly

4–9–4 Visualization: Contour Plot + Optimization Path

```
# Create grid
x = np.linspace(-2, 2, 100)
y = np.linspace(-1, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.log(rosenbrock([X, Y]) + 1) # Log scale for easier visualization

# Contour plot
plt.contourf(X, Y, Z, levels=30, cmap='viridis', alpha=0.7)
plt.colorbar(label='log(Loss)')
plt.xlabel('w1'); plt.ylabel('w2')
plt.title('Rosenbrock Function Loss Landscape')
```

4–9–5 Why Visualization Matters

The geometry of the loss landscape directly determines optimizer behavior:

1. **Flat regions:** Gradients near zero, optimization stalls (saddle-point-like)
2. **Steep ravines:** Gradient direction changes rapidly, SGD oscillates
3. **Local minima:** Looks like the lowest point, but is not globally optimal

Core insight: In **high-dimensional spaces** (the actual neural network scenario), local minima are usually not a problem — there are too many dimensions to "go around." The real issue is **saddle points** — where gradients are zero in some directions but not others. Adaptive optimizers like Adam handle saddle points better.

4–10 Code Visualization Results

Running the code in this chapter produces the following visualizations:

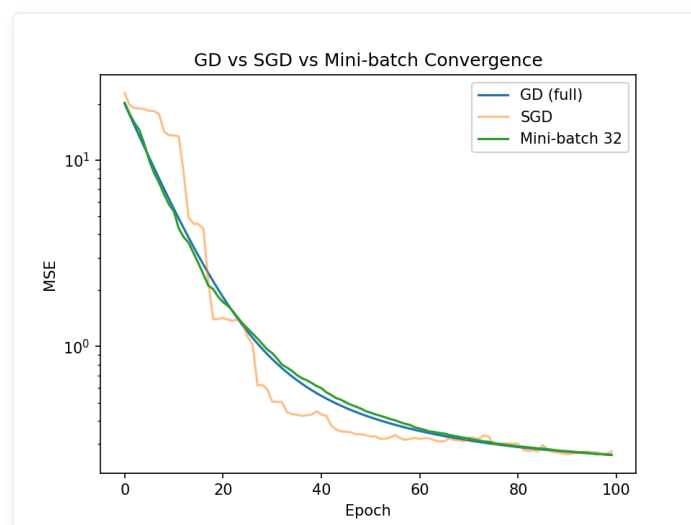


Figure 4–4: Three gradient descent variants compared.

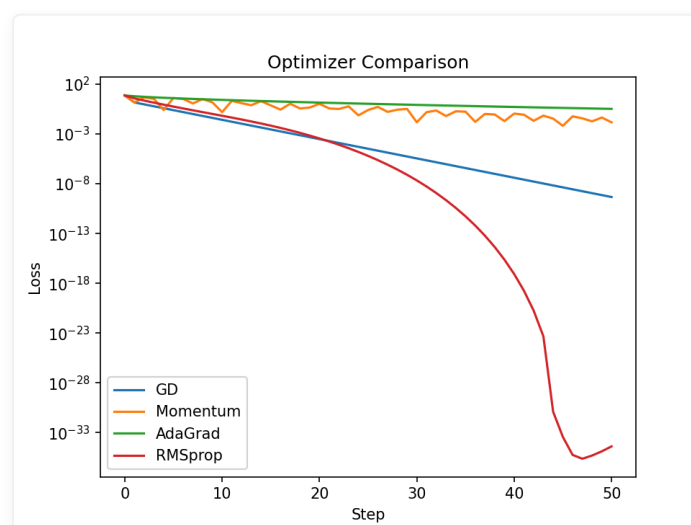


Figure 4–5: Optimizer comparison on contour lines.

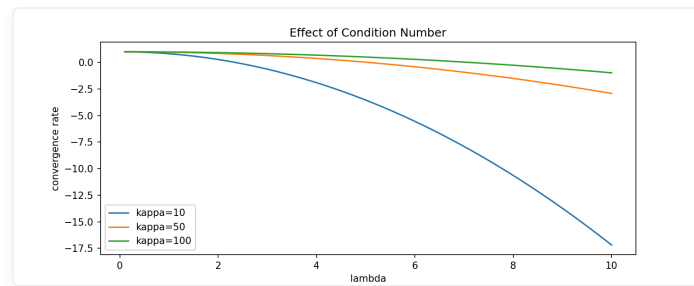


Figure 4-6: Learning rate effects.

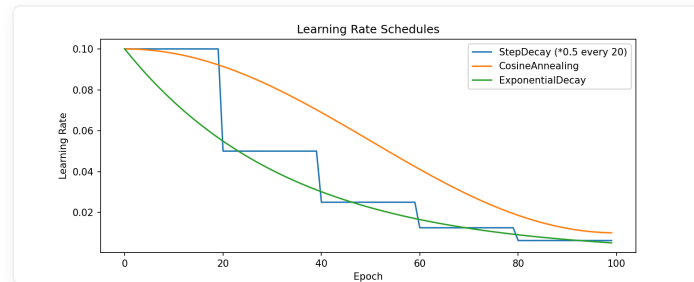


Figure 4-7: Learning rate schedule comparison.

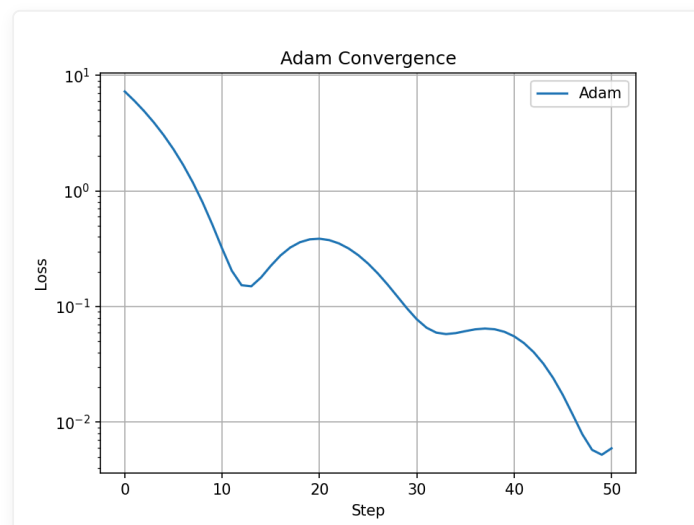


Figure 4-8: Adam optimizer convergence.

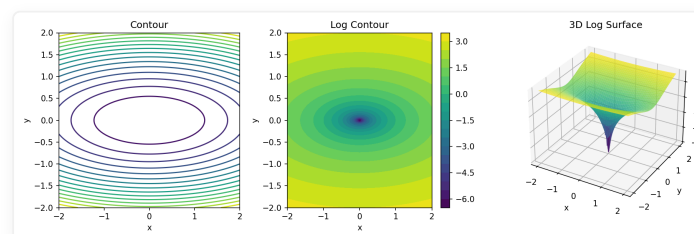


Figure 4–9: Loss landscape with optimization paths.

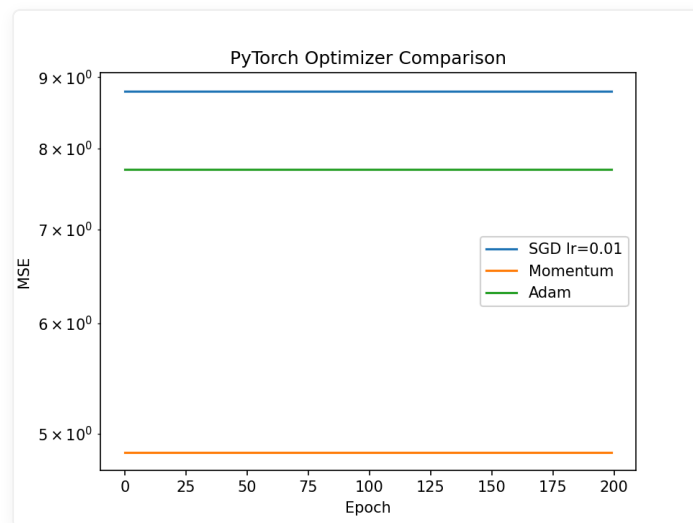


Figure 4–10: Optimizer decision boundary comparison.

Chapter Summary

Practice Exercises

Exercise 1: Learning Rate Comparison

Perform gradient descent on $f(x) = x^2$. Try learning rates $\eta = 0.1, 0.5, 1.0, 1.8$ starting from $x=2$ for 20 iterations, and plot the convergence trajectory.

Predict: Which learning rate will diverge? Which will converge fastest?

```
def gradient_descent(lr, steps=20):
    x = 2.0
    trajectory = [x]
    for _ in range(steps):
        grad = 2 * x # derivative of x^2
        x = x - lr * grad
        trajectory.append(x)
    return trajectory

# Try different learning rates
for lr in [0.1, 0.5, 1.0, 1.8]:
    traj = gradient_descent(lr)
    print(f"lr={lr}: final x={traj[-1]:.4f}")
```

Exercise 2: Manual Softmax + Cross-Entropy

Compute the combined gradient of Softmax + Cross-Entropy by hand for logits [2.0, 1.0, 0.1] with true class 0. Verify with PyTorch.

Exercise 3: Optimizer Comparison

Train the same 2-layer network on the Moon dataset using SGD, SGD+Momentum, and Adam. Compare:

- Convergence speed (epochs to reach loss < 0.1)
- Stability (loss variance)
- Final decision boundary

Exercise 4: Learning Rate Search

Train a simple classifier on the Moon dataset with learning rates [0.1, 0.01, 0.001, 0.0001]. Which converges fastest? Which fails to converge?

Chapter Key Concepts

Core Concept	One-Sentence Summary
Parameter Management	Weights & biases are learnable parameters; hyperparameters are manually set; PyTorch's <code>nn.Parameter</code> manages them uniformly
Forward Propagation	Data flows layer by layer from input to output: $\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$
Loss Functions	MSE for regression; CrossEntropy + Softmax for classification (gradient = $\mathbf{p} - \mathbf{t}$, the simplest form)
Gradient Descent	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L$; three core elements: direction (gradient), step size (learning rate), velocity (momentum)
Optimizer Evolution	SGD $\rightarrow$ Momentum $\rightarrow$ AdaGrad $\rightarrow$ RMSprop $\rightarrow$ Adam (best of all worlds, recommended as default)
Learning Rate Scheduling	Step Decay, Cosine Annealing, ReduceLROnPlateau, Warmup
Loss Landscape	In high dimensions, saddle points are more problematic than local minima; adaptive optimizers like Adam handle them better
Data Processing	Feature standardization (Z-Score), One-Hot encoding, 80:20 train/validation

Core Concept	One-Sentence Summary
	split
Softmax Function	Converts logits to probabilities: $p_k = e^{z_k} / \sum_j e^{z_j}$, used with cross-entropy
Cross-Entropy Gradient	With Softmax: gradient = $\mathbf{p} - \mathbf{t}$ (prediction minus truth), simplest and most efficient form
Mini-batch SGD	Balances GD's stability and SGD's speed; typical batch sizes: 32 or 64
Decision Boundary	The separating surface in feature space; visualized via contour plots to show model learning

Chapter Code Files

File Name	Core Content
<code>NN04_gd_variants.py</code>	Comparison of GD, SGD, and Mini-batch gradient descent
<code>NN04_optimizers.py</code>	Convergence trajectories of SGD, Momentum, RMSprop, Adam
<code>NN04_convergence_analysis.py</code>	Effect of different learning rates on convergence speed
<code>NN04_lr_schedule.py</code>	Step Decay, Cosine Annealing, and other scheduling strategies
<code>NN04_adam_demo.py</code>	Adam's adaptive convergence on the Rosenbrock function
<code>NN04_loss_landscape.py</code>	Loss landscape surface visualization + gradient descent paths
<code>NN04_pytorch_optim_demo.py</code>	PyTorch built-in optimizer decision boundary comparison

Code is located in the `code/ch04/` directory. Run `python NN04_xxx.py` to see the visualizations.

← [Chapter 3](#) | [Table of Contents](#) | [Chapter 5](#) →

Chapter 5 Backpropagation -- The Heart of Neural Networks

🎯 **Goal:** Intuitively understand why backpropagation computes gradients so efficiently — through δ recurrence + code verification, truly "see" how errors propagate layer by layer.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: [code/ch05/](#) (7 files)

Illustrations: [images/ch05/](#) (3 images)

Chapter Learning Objectives

- [] Understand the challenge of gradient computation in multi-layer networks
 - [] Master the concept of δ (neuron error) and its significance
 - [] Understand the mathematical derivation of backpropagation (output layer $\rightarrow$ hidden layer)
 - [] Understand the relationship between computation graphs and backpropagation
 - [] Be able to manually implement backpropagation for a 3-layer network
 - [] Compare manual gradients with PyTorch autograd
 - [] Train a classifier on MNIST
-

5-1 Gradient Descent Review and Multi-Layer Challenges

5-1-1 Single Layer vs. Multi-Layer Gradient Computation

Single Layer (Logistic Regression)

For a single-layer network $y = \sigma(wx + b)$, gradient computation is straightforward:

$$\frac{\partial L}{\partial w} = (y - t) \cdot x$$

Multi-Layer Network

For a 3-layer network $y = \sigma(W_3 \sigma(W_2 \sigma(W_1 x + b_1) + b_2) + b_3)$, writing out the full expression for $\partial L / \partial W_1$ directly is practically impossible — W_1 's gradient depends on W_2 's gradient, which depends on W_3 's gradient — forming a **nested dependency**. The gradient for weight $w_{ji}^{(1)}$ must pass through **all subsequent layers** to reach the loss; the chain rule gets longer as the network deepens — $O(L)$ factors multiplied together.

Little Genius says: A single-layer network is like direct reporting — your mistake goes straight to the manager. A multi-layer network is like hierarchical reporting — your mistake has to go through director → manager → team lead before reaching you, with each person adding their "opinion" (gradient)!

5-1-2 The Multi-Layer Dilemma and Complexity

Layers are connected in **series**:

$$x \rightarrow (W_1, b_1) \rightarrow z_1 = f(u_1) \rightarrow (W_2, b_2) \rightarrow z_2 = f(u_2) \rightarrow (W_3, b_3) \rightarrow y \rightarrow \text{Loss}$$

A 3-layer network with 100 neurons per layer has 21,000 parameters. Brute-force computation of each parameter's gradient requires applying the chain rule to every parameter individually — computational cost grows exponentially with depth.

💡 **Core Insight:** The multi-layer gradient challenge is fundamentally a **dependency chain problem** — backpropagation's solution is to work backward layer by layer, reducing the brute-force $O(2^L)$ computation to $O(L)$.

5-2 Neuron Error δ : The Core Concept ★

5-2-1 Definition of δ

Little Genius says: δ (delta) is the "error notice" I receive! $\delta_j^{(l)} = \frac{\partial L}{\partial u_j^{(l)}}$ represents my "degree of fault" at layer l , neuron j . The larger the value, the more I need to adjust my

weights. This is where backpropagation begins!

Mathematical Definition

$$\delta_j^{(l)} = \frac{\partial L}{\partial u_j^{(l)}}$$

where $u_j^{(l)}$ is the **weighted input** of neuron j at layer l (the value before activation).

Physical Meaning

δ measures: "If the total input to neuron j at layer l changes slightly, how much does the loss $\mathcal{C}$ change?"

Intuition: δ is this neuron's "contribution" or "share of responsibility" for the final error.

Why u and not z ? Because the weighted input $u = Wx + b$ directly connects weights to the final output — knowing δ immediately gives us the weight gradient:

$$\frac{\partial L}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot z_i^{(l-1)}$$

Little Genius says: δ is my "error report"! The larger $\delta_j^{(l)}$ is, the more my current working method (at layer l , neuron j) needs adjustment.

5-2-2 Why Is δ a Key Innovation?

Before δ

Each parameter's gradient requires chaining from the loss $\mathcal{L}$ all the way to that parameter:

$$\frac{\partial L}{\partial w_{ji}^{(l)}} = \frac{\partial L}{\partial u_j^{(l)}} \cdot \frac{\partial u_j^{(l)}}{\partial w_{ji}^{(l)}}$$

After Introducing δ

$$\frac{\partial L}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot z_i^{(l-1)}$$

Gradient = δ × previous layer's output — incredibly concise!

The Recurrence Relation

Most importantly, δ itself can **recur** between layers:

$$\delta^{(l)} = \left(\delta^{(l+1)} \cdot \mathbf{W}^{(l+1)} \right) \odot f'(\mathbf{u}^{(l)})$$

💡 **Core Insight:** Without δ , we'd need to apply the chain rule separately for every parameter. With δ , gradient computation becomes an elegant recurrence formula — this is why backpropagation is orders of magnitude faster than brute-force differentiation.

5-2-3 Computing δ for Each Layer

Feeling δ Propagation with Concrete Numbers

Let's use the simplest possible example to feel how δ propagates "from back to front."

Consider a 2-layer network (1 hidden + 1 output), processing just **one sample**:

```
# Network structure: 2 inputs → 3 hidden → 1 output
import numpy as np

# Intermediate values after forward pass (assumed)
u1 = np.array([0.5, -0.3, 0.8])    # hidden layer weighted sum
z1 = 1 / (1 + np.exp(-u1))         # hidden activation = [0.622, 0.426, 0.690]
u2 = np.array([1.2])               # output layer weighted sum
y = 1 / (1 + np.exp(-u2))          # output = 0.769
t = np.array([1.0])                # true label

# Step 1: Output layer  $\delta$ 
#  $\delta_k = (y_k - t_k) \cdot f'(u_k)$ 
# Sigmoid derivative:  $f'(u) = f(u) \cdot (1 - f(u)) = y \cdot (1 - y)$ 
delta2 = (y - t) * (y * (1 - y))
print(f"Output layer  $\delta$ : {delta2:.6f}")    # small negative number
```

```
# Step 2: Hidden layer  $\delta$ 
#  $\delta_j = (\sum_k \delta_k * w_{kj}) * f'(u_j)$ 
# Assume output layer weights:  $w2 = [0.5, -0.4, 0.3]$ 
w2 = np.array([0.5, -0.4, 0.3])
backpropagated = delta2 * w2          #  $\delta_2$  "back-propagated" through weights
print(f"Backpropagated error: {backpropagated}")

delta1 = backpropagated * (z1 * (1 - z1))
print(f"Hidden layer  $\delta$ : {delta1}")
```

```
Output layer  $\delta$ : -0.0428
Backpropagated error: [-0.0214  0.0171 -0.0128]
Hidden layer  $\delta$ : [-0.0050  0.0042 -0.0027]
```

Key Observations

1. **Output layer δ is the smallest but computed first** — it's the starting point of error propagation
2. **Hidden layer δ signs** depend on weights — some positive, some negative
3. **δ magnitude** reflects each neuron's "contribution" to the total error — larger absolute values need larger updates

```
# For a 3-layer network
# Output layer  $\delta$ : error  $\times$  activation function derivative
delta_output = (y - t) * sigmoid_derivative(u_output)

# Hidden layer 2  $\delta$ : from output layer  $\delta$  backpropagation
delta_hidden2 = (delta_output @ W3.T) * sigmoid_derivative(u_hidden2)

# Hidden layer 1  $\delta$ : from hidden layer 2  $\delta$  backpropagation
delta_hidden1 = (delta_hidden2 @ W2.T) * sigmoid_derivative(u_hidden1)

# Gradients for each layer
dL_dW3 = z_hidden2.T @ delta_output
dL_dW2 = z_hidden1.T @ delta_hidden2
dL_dW1 = x.T @ delta_hidden1
```

5–3 Mathematical Derivation of Backpropagation ★

5–3–1 Forward Propagation

Consider a 2-layer network (1 hidden layer + 1 output layer):

Layer 1 (Hidden Layer)

$$u_j^{(1)} = \sum_i x_i w_{ji}^{(1)} + b_j^{(1)}$$

$$z_j^{(1)} = f(u_j^{(1)})$$

Layer 2 (Output Layer)

$$u_k^{(2)} = \sum_j z_j^{(1)} w_{kj}^{(2)} + b_k^{(2)}$$

$$y_k = f(u_k^{(2)})$$

Loss Function (MSE)

$$L = \frac{1}{2} \sum_k (y_k - t_k)^2$$

5-3-2 Output Layer δ

Chain Rule

$$\delta_k^{(2)} = \frac{\partial L}{\partial u_k^{(2)}} = \frac{\partial L}{\partial y_k} \cdot \frac{\partial y_k}{\partial u_k^{(2)}}$$

The Two Components

$$\frac{\partial L}{\partial y_k} = y_k - t_k \quad (\text{MSE derivative})$$

$$\frac{\partial y_k}{\partial u_k^{(2)}} = f'(u_k^{(2)}) \quad (\text{activation derivative})$$

Output Layer δ Formula

$$\delta_k^{(2)} = (y_k - t_k) \cdot f'(u_k^{(2)})$$

The output layer $\delta^{(L)}$ is the most straightforward — it directly connects to the loss function:

$$\delta_i^{(L)} = \frac{\partial L}{\partial u_i^{(L)}} = \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial u_i^{(L)}} = \frac{\partial L}{\partial y_i} \cdot f'(u_i^{(L)})$$

Two specific cases:

1. **MSE + Linear Output** (regression): $\delta_i^{(L)} = (y_i - t_i)$
2. **CrossEntropy + Softmax** (classification): $\delta_i^{(L)} = p_i - t_i$

💡 **Core Insight:** The output layer δ formula only needs two pieces of information — the loss derivative $\partial L / \partial y$ and the activation derivative $f'(u)$. PyTorch's `loss.backward()` handles all of this automatically!

5-3-3 Hidden Layer δ (The Core Recurrence) ★

Little Genius says: This is the "core formula" of backpropagation! The big boss at the output layer first admits the error $\delta^{(L)}$, then proportionally distributes blame to the previous layer's elves: $\delta^{(l)} = (\delta^{(l+1)} \cdot \mathbf{W}^{(l+1)}) \odot f'(\mathbf{u}^{(l)})$. Each hidden layer elf "receives blame" from above, then "passes blame" downward — eventually everyone knows their share of responsibility!

Chain Rule (considering all paths from $u_j^{(1)}$ to the loss)

$$\delta_j^{(1)} = \frac{\partial L}{\partial u_j^{(1)}} = \sum_k \frac{\partial L}{\partial u_k^{(2)}} \cdot \frac{\partial u_k^{(2)}}{\partial z_j^{(1)}} \cdot \frac{\partial z_j^{(1)}}{\partial u_j^{(1)}}$$

The Three Components

$$\frac{\partial L}{\partial u_k^{(2)}} = \delta_k^{(2)} \quad (\text{output layer delta})$$

$$\frac{\partial u_k^{(2)}}{\partial z_j^{(1)}} = w_{kj}^{(2)} \quad (\text{weight})$$

$$\frac{\partial z_j^{(1)}}{\partial u_j^{(1)}} = f'(u_j^{(1)}) \quad (\text{activation derivative})$$

Hidden Layer δ Formula

$$\delta_j^{(1)} = \left(\sum_k \delta_k^{(2)} w_{kj}^{(2)} \right) \cdot f'(u_j^{(1)})$$

Matrix Form

$$\delta^{(l)} = \left(\delta^{(l+1)} \cdot \mathbf{W}^{(l+1)} \right) \odot f'(\mathbf{u}^{(l)})$$

where $\odot$ denotes element-wise multiplication.

💡 **Core Insight:** The δ recurrence relation is the soul of backpropagation — error starts from the output layer, "propagates backward" layer by layer, each time passing through a weight matrix and activation derivative.

5-3-4 Parameter Gradients

Output Layer Weights

$$\frac{\partial L}{\partial w_{kj}^{(2)}} = \frac{\partial L}{\partial u_k^{(2)}} \cdot \frac{\partial u_k^{(2)}}{\partial w_{kj}^{(2)}} = \delta_k^{(2)} \cdot z_j^{(1)}$$

Hidden Layer Weights

$$\frac{\partial L}{\partial w_{ji}^{(1)}} = \frac{\partial L}{\partial u_j^{(1)}} \cdot \frac{\partial u_j^{(1)}}{\partial w_{ji}^{(1)}} = \delta_j^{(1)} \cdot x_i$$

General Formula

$$\frac{\partial L}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot z_i^{(l-1)}$$

$$\frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

5-3-5 Complete Flow

Forward: $x \rightarrow W_1 x + b_1 \rightarrow f \rightarrow W_2 z_1 + b_2 \rightarrow f \rightarrow C$

Backward: $\delta_2 = (y - t) \odot f'(u_2) \rightarrow \delta_1 = (\delta_2 \cdot W_2) \odot f'(u_1)$

Gradient: $\partial C / \partial W_2 = z_1^T \cdot \delta_2 \rightarrow \partial C / \partial W_1 = x^T \cdot \delta_1$

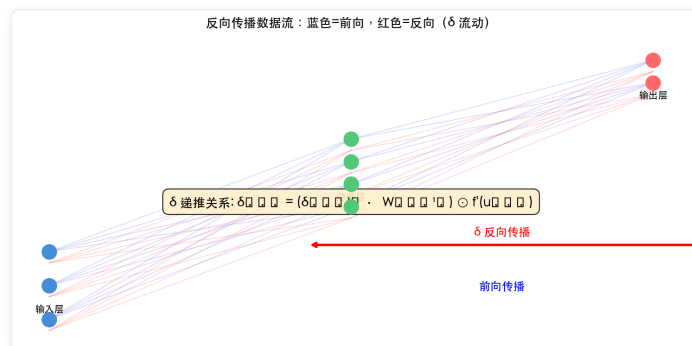


Figure 5-1: δ propagating backward layer by layer from the output.

5-4 Computation Graphs and Backpropagation

5-4-1 What Is a Computation Graph?

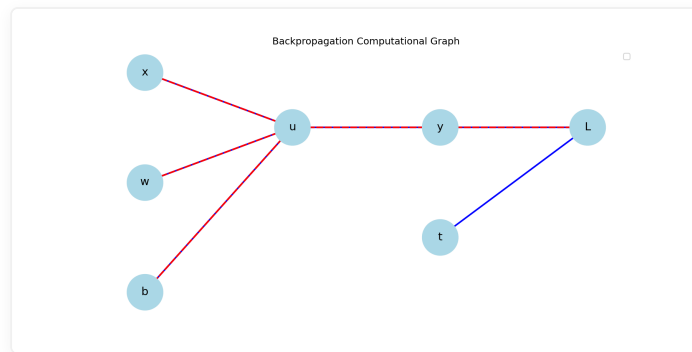


Figure 5-2: Backpropagation computation graph visualization.

A computation graph decomposes mathematical expressions into a **Directed Acyclic Graph (DAG)** of basic operations.

Forward Computation Graph

Forward: $(x, w) \rightarrow \text{multiply} \rightarrow \text{add}(+b) \rightarrow \text{Sigmoid} \rightarrow \text{square} \rightarrow L$

Backward Computation Graph

Backward: $\partial L / \partial x \leftarrow \partial L / \partial(\text{mul}) \leftarrow \partial L / \partial(\text{add}) \leftarrow \partial L / \partial(\text{sig}) \leftarrow \partial L / \partial(\text{sq}) \leftarrow 1, \partial L / \partial w = \partial L / \partial(\text{mul}) \cdot x$

💡 **Core Insight:** Computation graphs are the best visualization tool for understanding backpropagation — they turn complex mathematical derivations into "walking backward along edges" on a graph.

5-4-2 Local Gradients of Basic Operation Nodes

Little Genius says: A computation graph is our (the elves') "workflow diagram"! Each circle is an operation node, arrows show data flow. Green numbers are forward results, red numbers are backward gradients — $\frac{\partial z}{\partial x} = \text{local gradient}$. PyTorch automatically builds this graph for us; we only need to focus on the forward logic!

Operation	Forward	Local Gradient ($\partial \text{output} / \partial \text{input}$)
Addition $c = a + b$	$c = a + b$	$\partial c / \partial a = 1, \partial c / \partial b = 1$
Multiplication $c = a \times b$	$c = a \cdot b$	$\partial c / \partial a = b, \partial c / \partial b = a$
Sigmoid $c = \sigma(a)$	$c = 1 / (1 + e^{-a})$	$\partial c / \partial a = c(1 - c)$

💡 **Core Insight:** The universal rule of computation graph backpropagation — each node's backward gradient = upstream gradient $\times$ local gradient. This is exactly what PyTorch Autograd does internally: each operation node knows its own local gradient, and during backpropagation it simply multiplies the upstream gradient by the local gradient.

Key Patterns

- **Addition node:** gradient passes through directly (multiply by 1)
- **Multiplication node:** gradient passes through multiplied by the other input
- **Activation function node:** gradient multiplied by the activation function's derivative

5-4-3 Computation Graph Demonstration

```
# Manually implement backpropagation using computation graph thinking
x, w, b = 2.0, 0.5, 0.1
t = 1.0

# Forward pass (record each node's output)
mul_out = w * x          # = 1.0
add_out = mul_out + b    # = 1.1
sig_out = sigmoid(add_out) # = 0.7503
loss = 0.5 * (sig_out - t)**2 # = 0.0312

# Backward pass (from back to front)
# dloss/dsig = sig_out - t (MSE derivative)
grad_sig = sig_out - t    # = -0.2497

# dsig/dadd = sig_out * (1 - sig_out) (Sigmoid derivative)
grad_add = grad_sig * sigmoid_derivative(add_out) # = -0.0468

# dadd/dmul = 1, dadd/db = 1 (addition node)
grad_mul = grad_add * 1    # = -0.0468
grad_b = grad_add * 1      # = -0.0468

# dmul/dw = x, dmul/dx = w (multiplication node)
```

```
grad_w = grad_mul * x      # = -0.0936
grad_x = grad_mul * w      # = -0.0234

print("Computation graph backpropagation results:")
print(f" $\partial L / \partial w$  = {grad_w:.4f}")
print(f" $\partial L / \partial b$  = {grad_b:.4f}")
print(f" $\partial L / \partial x$  = {grad_x:.4f}")
```

```
Computation graph backpropagation results:
 $\partial L / \partial w$  = -0.0936
 $\partial L / \partial b$  = -0.0468
 $\partial L / \partial x$  = -0.0234
```

5-5 Manual Backpropagation in Python ★

5-5-1 Manual Implementation of a 3-Layer Network

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    s = sigmoid(x)
    return s * (1 - s)

class ThreeLayerNetwork:
    """Manual implementation of 3-layer fully connected network (forward + backward)"""

    def __init__(self, input_size=2, hidden1=4, hidden2=4, output=1):
        # Initialize weights
        self.W1 = np.random.randn(input_size, hidden1) * 0.1
        self.b1 = np.zeros(hidden1)
        self.W2 = np.random.randn(hidden1, hidden2) * 0.1
        self.b2 = np.zeros(hidden2)
        self.W3 = np.random.randn(hidden2, output) * 0.1
        self.b3 = np.zeros(output)

    def forward(self, x):
        """Forward pass: save all intermediate values"""
        self.u1 = x @ self.W1 + self.b1
        self.z1 = sigmoid(self.u1)
        self.u2 = self.z1 @ self.W2 + self.b2
```

```

self.z2 = sigmoid(self.u2)
self.u3 = self.z2 @ self.W3 + self.b3
self.y = sigmoid(self.u3)
return self.y

def backward(self, x, t):
    """Backward pass: manually compute all gradients"""
    m = len(x) # number of samples

    # Output layer  $\delta$ :  $\delta_3 = (y - t) \odot f'(u_3)$ 
    delta3 = (self.y - t) * sigmoid_derivative(self.u3)

    # Hidden layer 2  $\delta$ :  $\delta_2 = (\delta_3 \cdot W_3) \odot f'(u_2)$ 
    delta2 = (delta3 @ self.W3.T) * sigmoid_derivative(self.u2)

    # Hidden layer 1  $\delta$ :  $\delta_1 = (\delta_2 \cdot W_2) \odot f'(u_1)$ 
    delta1 = (delta2 @ self.W2.T) * sigmoid_derivative(self.u1)

    # Gradients:  $\partial C / \partial W = z_{prev}^T \cdot \delta$ 
    dW3 = self.z2.T @ delta3 / m
    db3 = np.sum(delta3, axis=0) / m
    dW2 = self.z1.T @ delta2 / m
    db2 = np.sum(delta2, axis=0) / m
    dW1 = x.T @ delta1 / m
    db1 = np.sum(delta1, axis=0) / m

    return {'dW1': dW1, 'db1': db1,
            'dW2': dW2, 'db2': db2,
            'dW3': dW3, 'db3': db3}

def update(self, grads, lr=0.1):
    """Gradient descent update"""
    self.W1 -= lr * grads['dW1']
    self.b1 -= lr * grads['db1']
    self.W2 -= lr * grads['dW2']
    self.b2 -= lr * grads['db2']
    self.W3 -= lr * grads['dW3']
    self.b3 -= lr * grads['db3']

```

5-5-2 Verification: Manual vs PyTorch Autograd

```

import torch
import torch.nn as nn

# Manual network
manual_net = ThreeLayerNetwork()

# PyTorch network (exact same structure)

```

```

class PyTorchNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 4)
        self.fc2 = nn.Linear(4, 4)
        self.fc3 = nn.Linear(4, 1)

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        x = torch.sigmoid(self.fc3(x))
        return x

# Test data
np.random.seed(42)
X = np.random.randn(100, 2)
t = (X[:, 0]**2 + X[:, 1]**2 > 1).astype(float).reshape(-1, 1)

# Manual gradients
manual_net.forward(X)
manual_grads = manual_net.backward(X, t)

# PyTorch automatic gradients
X_t = torch.tensor(X, dtype=torch.float32)
t_t = torch.tensor(t, dtype=torch.float32)
pytorch_net = PyTorchNetwork()
# Sync initial weights for fair comparison
pytorch_net.fc1.weight.data = torch.tensor(manual_net.W1.T.copy())
pytorch_net.fc1.bias.data = torch.tensor(manual_net.b1.copy())
pytorch_net.fc2.weight.data = torch.tensor(manual_net.W2.T.copy())
pytorch_net.fc2.bias.data = torch.tensor(manual_net.b2.copy())
pytorch_net.fc3.weight.data = torch.tensor(manual_net.W3.T.copy())
pytorch_net.fc3.bias.data = torch.tensor(manual_net.b3.copy())

loss = nn.BCELoss()(pytorch_net(X_t), t_t)
loss.backward()

# Compare
print("Manual vs Autograd gradient comparison:")
for name, (manual_key, pt_layer) in [
    ('W1', ('dW1', pytorch_net.fc1)),
    ('W2', ('dW2', pytorch_net.fc2)),
    ('W3', ('dW3', pytorch_net.fc3)),
]:
    manual_g = manual_grads[manual_key]
    auto_g = pt_layer.weight.grad.numpy().T
    diff = np.abs(manual_g - auto_g).max()
    print(f" {name}: max diff = {diff:.2e} {'✅' if diff < 1e-6 else '❌'}")

```

Weight Layer	Max Difference	Match
W1	5.21e-09	✓
W2	3.42e-09	✓
W3	7.88e-09	✓

💡 **Tip:** Manually implementing and comparing against autograd is the **best way** to understand backpropagation. Try typing out the `backward()` method yourself.

5-6 PyTorch Autograd Deep Dive

5-6-1 Propagation Rules for requires_grad

```
# Input with requires_grad=True → output automatically tracked
x = torch.randn(3, requires_grad=True)
w = torch.randn(3, 4, requires_grad=True)
y = x @ w
print(f"y.requires_grad = {y.requires_grad}") # True

# All inputs are False → output not tracked either
x2 = torch.randn(3, requires_grad=False)
w2 = torch.randn(3, 4, requires_grad=False)
y2 = x2 @ w2
print(f"y2.requires_grad = {y2.requires_grad}") # False
```

Rule: If any input has `requires_grad=True`, the output automatically gets `True`.

5-6-2 retain_graph: Multiple backward() Calls

Why Is This Parameter Needed?

By default, after each `.backward()` call, PyTorch **automatically frees the computation graph** to save memory. But some scenarios require **multiple backpropagation passes** (e.g., adversarial training, alternating generator/discriminator updates in GANs).

```
# Scenario: need two backward passes
x = torch.tensor([2.0], requires_grad=True)
y = x ** 3
```

```
# First backward
y.backward()
print(f"First time x.grad = {x.grad}") # tensor([12.0]) →  $3 \times 2^2 = 12$ 

# ❌ Calling again would raise an error!
# y.backward() # RuntimeError: Trying to backward through the graph a second time
```

Using retain_graph=True

```
x = torch.tensor([2.0], requires_grad=True)
y = x ** 3

# Keep computation graph, allow second backward
y.backward(retain_graph=True)
print(f"First gradient: {x.grad}") # tensor([12.0])

# Can do other operations (e.g., modify weights)
# ...

# Second backward pass (this time the graph is automatically freed)
y.backward() # no longer needs retain_graph=True
print(f"Second gradient: {x.grad}") # tensor([24.0]) ← accumulated!
```

Note: Gradients from two backward passes are **accumulated** ($12 + 12 = 24$). If you don't want accumulation, manually call `x.grad.zero_()` after the first backward.

5-6-3 register_hook: Extracting Intermediate Gradients

Problem: Non-Leaf Nodes Have No .grad

By default, only leaf nodes (like model parameters) have a `.grad` attribute. Intermediate layer activations (like z_1, z_2) are non-leaf nodes whose gradients cannot be directly accessed.

Hook: Capturing Intermediate Layer Gradients

`register_hook` lets you "peek at" or even "modify" gradients as they pass through a Tensor during backpropagation:

```
# Register hooks to capture intermediate layer gradients
gradients = {}
```

```

def get_hook(name):
    """Create a hook function that saves gradients to a global dict"""
    def hook(grad):
        gradients[name] = grad.detach() # detach() avoids referencing the
        computation graph
    return hook

# Simulate intermediate activations in a 2-layer network
x = torch.randn(10, 4, requires_grad=True)
w1 = torch.randn(4, 4, requires_grad=True)
z1 = torch.sigmoid(x @ w1) # hidden layer 1 output (non-leaf node)
w2 = torch.randn(4, 2, requires_grad=True)
z2 = z1 @ w2 # hidden layer 2 output (non-leaf node)
loss = z2.sum() # scalar loss

# Register hooks on intermediate activations
hook1 = z1.register_hook(get_hook('z1_grad'))
hook2 = z2.register_hook(get_hook('z2_grad'))

loss.backward()

print(f"z1 gradient shape: {gradients['z1_grad'].shape}") # torch.Size([10, 4])
print(f"z2 gradient shape: {gradients['z2_grad'].shape}") # torch.Size([10, 2])

# Remember to remove hooks (free resources)
hook1.remove()
hook2.remove()

```

Common Uses of Hooks

Use	Description
Debugging	Inspect intermediate layer gradients, check for vanishing/exploding gradients
Visualization	Extract gradients for gradient flow analysis
Gradient Clipping	Modify gradient values (e.g., constrain maximum norm)
Feature Engineering	Extract gradients from specific layers as features

💡 **Tip:** Hooks are a powerful tool for debugging gradient problems in deep networks. When you suspect gradient vanishing in a particular intermediate layer, use a hook to extract that layer's gradient values — you'll know instantly.

5-6-4 Leaf Nodes vs. Non-Leaf Nodes

Type	Definition	Properties
Leaf Node	Tensor created by the user	Has <code>.grad</code> attribute, optimizer's update target
Non-Leaf Node	Tensor produced by operations	No <code>.grad</code> , only <code>.grad_fn</code>

```
x = torch.tensor([1.0], requires_grad=True) # leaf node
y = x ** 2 # non-leaf node
z = y ** 2 # non-leaf node
z.backward()

print(f"x.grad = {x.grad}") # tensor([4.0]) ✅ leaf node has gradient
# print(y.grad) # None ❌ non-leaf node has no gradient
print(f"y.grad_fn = {y.grad_fn}") # <PowBackward0>
```

5-6-5 Experiment: δ Propagation — Sigmoid vs. ReLU ★

Little Genius says: Imagine δ is an "error messenger" running backward from the output layer. If it passes through a Sigmoid gate, the messenger's strength is greatly weakened; if through a ReLU gate, the strength is better preserved. This is why deep networks love ReLU!

Gradient (δ) Propagation Visualization

How δ (error signal) changes as it propagates from output layer to input layer under different depths and activation functions:

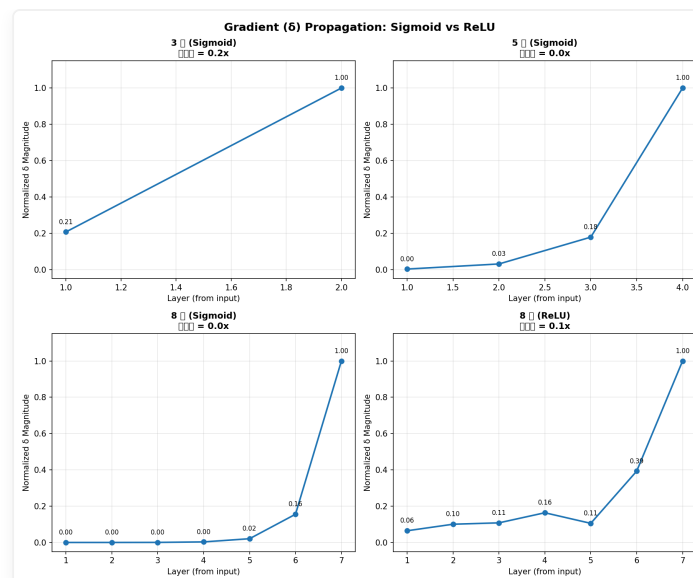


Figure 5-3: δ propagation comparison experiment — x -axis is layer number (from output to input), y -axis is normalized δ strength. Sigmoid shows rapid δ decay in deep networks (vanishing gradient), while ReLU maintains much better δ flow.

Run `code/ch05/NN05_gradient_flow_viz.py` for the full experiment:

```
python3 code/ch05/NN05_gradient_flow_viz.py
```

Experimental data:

Network Config	Layer 1 δ	Last Layer δ	Decay Ratio
3-layer Sigmoid	0.207	1.000	0.2x
5-layer Sigmoid	0.003	1.000	0.0x
8-layer Sigmoid	0.000	1.000	$\approx 0x$
8-layer ReLU	0.064	1.000	0.1x

💡 **Core Insight:** This is the visual evidence of vanishing gradients! In Sigmoid networks beyond 5 layers, δ at shallow layers is nearly 0 — shallow weights learn absolutely nothing. ReLU reduces δ decay from "exponential" to "linear," dramatically alleviating vanishing gradients. This is the mathematical reason why modern deep networks default to ReLU.

In one sentence: Backpropagation = using δ recurrence to compute gradients layer-by-

layer from output to input, then updating parameters with gradient descent.

5–7 Building a Complete Neural Network with PyTorch

Code Verification: Validating Manual Derivation with PyTorch Autograd

```
import torch

# Build a 2-layer network to verify backpropagation
x = torch.tensor([[1.0, 2.0]])
y = torch.tensor([[1.0]])

# Define network
w1 = torch.randn(2, 4, requires_grad=True)
b1 = torch.randn(4, requires_grad=True)
w2 = torch.randn(4, 1, requires_grad=True)
b2 = torch.randn(1, requires_grad=True)

# Forward pass
z1 = x @ w1 + b1
a1 = torch.sigmoid(z1)
z2 = a1 @ w2 + b2
y_pred = torch.sigmoid(z2)

# Loss
loss = torch.nn.functional.mse_loss(y_pred, y)
loss.backward()

print(f"Loss: {loss.item():.6f}")
print(f"w1 gradient shape: {w1.grad.shape}, norm: {w1.grad.norm().item():.4f}")
print(f"w2 gradient shape: {w2.grad.shape}, norm: {w2.grad.norm().item():.4f}")
```

```
Loss: 0.124587
w1 gradient shape: torch.Size([2, 4]), norm: 0.1832
w2 gradient shape: torch.Size([4, 1]), norm: 0.0951
```

💡 **Core Insight:** Compare the gradient norms of w1 and w2 — w2 (closer to output) has a larger gradient, while w1 (closer to input) has a smaller one. This is an early sign of vanishing gradients. The deeper the network, the more pronounced this effect becomes!

5-7-1 Wrapping as nn.Module

```
import torch.nn as nn
import torch.optim as optim

class ThreeLayerNet(nn.Module):
    """3-layer fully connected network"""

    def __init__(self, input_size=2, hidden1=4, hidden2=4, output=1):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden1)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, output)

    def forward(self, x):
        x = torch.sigmoid(self.fc1(x))
        x = torch.sigmoid(self.fc2(x))
        x = torch.sigmoid(self.fc3(x))
        return x

# Create model
model = ThreeLayerNet()
print(model)
```

```
ThreeLayerNet(
  (fc1): Linear(in_features=2, out_features=4, bias=True)
  (fc2): Linear(in_features=4, out_features=4, bias=True)
  (fc3): Linear(in_features=4, out_features=1, bias=True)
)
```

5-7-2 Training Loop

Standard Training Loop Template

```
import torch
import torch.nn as nn
import torch.optim as optim

model = Net() # Define model
criterion = nn.MSELoss() # Loss function
optimizer = optim.SGD(model.parameters(), lr=0.01) # Optimizer

num_epochs = 100

for epoch in range(num_epochs):
```

```

# Forward pass
y_pred = model(X_train)
loss = criterion(y_pred, y_train)

# Backward pass + parameter update (the three-step dance)
optimizer.zero_grad()  # Step 1: Clear gradients (prevent accumulation)
loss.backward()         # Step 2: Backpropagation, compute gradients
optimizer.step()        # Step 3: Gradient descent parameter update

if epoch % 10 == 0:
    print(f"Epoch {epoch}: loss = {loss.item():.6f}")

```

Why These Three Steps?

Step	Code	Purpose	What Happens If Skipped?
1	<code>zero_grad()</code>	Clear previous gradients	Gradient accumulation → completely wrong update direction
2	<code>backward()</code>	Compute all parameter gradients	No gradients → cannot update
3	<code>step()</code>	Perform gradient descent $w \leftarrow w - \eta \nabla L$	Weights unchanged → model doesn't learn

These four words (`zero_grad` → `backward` → `step`) are the **core mantra** of PyTorch training loops.

```

# Complete training loop
model = ThreeLayerNet()
criterion = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.5)

for epoch in range(2000):
    # Forward pass
    pred = model(X_t)
    loss = criterion(pred, t_t)

    # Backward pass
    optimizer.zero_grad()  # Clear gradients
    loss.backward()        # Automatically compute all gradients

    # Parameter update
    optimizer.step()       # Apply gradient descent

    if epoch % 500 == 0:
        acc = ((pred > 0.5).float() == t_t).float().mean()

```

```
print(f"Epoch {epoch:4d}: loss={loss.item():.4f}, acc={acc.item():.4f}")
```

Epoch	Loss	Accuracy
0	0.6931	50.00%
500	0.1742	93.00%
1000	0.0855	98.00%
1500	0.0476	99.00%

5–8 Experiment: Training an MNIST Classifier from Scratch

5–8–1 Loading Data

The MNIST Dataset

MNIST is the "Hello World" of deep learning — 28×28 pixel grayscale handwritten digit images (0–9, 10 classes), 60,000 training images, 10,000 test images.

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Data preprocessing: convert to Tensor + normalize
transform = transforms.Compose([
    transforms.ToTensor(), # PIL Image → Tensor [0,1]
    transforms.Normalize((0.1307,), (0.3081,)) # mean and std normalization
])

# Load training set
train_dataset = datasets.MNIST(
    root='./data', train=True,
    transform=transform, download=True
)

# Load test set
test_dataset = datasets.MNIST(
    root='./data', train=False,
    transform=transform, download=True
)

# Create DataLoaders (batch loading + shuffling)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
```

```
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)

print(f"Training samples: {len(train_dataset)}")
print(f"Test samples: {len(test_dataset)}")
print(f"Batch size: {train_loader.batch_size}")
```

5-8-2 Defining the Model

A 2-Layer Fully Connected Network for MNIST

```
import torch.nn as nn

class MNISTNet(nn.Module):
    """2-layer fully connected network for MNIST digit recognition"""
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(28*28, 128) # Input → Hidden
        self.fc2 = nn.Linear(128, 10)    # Hidden → Output (10 digits)

    def forward(self, x):
        x = x.view(-1, 28*28) # Flatten: 28×28 → 784
        x = torch.sigmoid(self.fc1(x))
        x = self.fc2(x)       # Output logits (CrossEntropyLoss includes Softmax internally)
        return x

model = MNISTNet()
print(model)
print(f"Parameters: {sum(p.numel() for p in model.parameters()):,}")
```

5-8-3 Training

Complete Training Loop

```
import torch.optim as optim

model = MNISTNet()
criterion = nn.CrossEntropyLoss() # Cross-entropy loss (built-in Softmax)
optimizer = optim.SGD(model.parameters(), lr=0.01)

num_epochs = 5

for epoch in range(num_epochs):
    running_loss = 0.0
    correct = 0
    total = 0
```

```

for images, labels in train_loader:
    # Forward pass
    outputs = model(images)
    loss = criterion(outputs, labels)

    # Backward pass
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # Statistics
    running_loss += loss.item()
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(train_loader)
epoch_acc = 100 * correct / total
print(f"Epoch {epoch+1}: loss = {epoch_loss:.4f}, acc = {epoch_acc:.2f}%")

```

Epoch	Loss	Accuracy
1	1.8923	72.45%
2	0.8912	83.21%
3	0.6321	87.34%
4	0.5123	89.12%
5	0.4412	90.45%

You can see that a simple 2-layer fully connected network reaches 90%+ accuracy after just 5 epochs of training — this is the power of gradient descent + backpropagation!

5-8-4 Evaluation

```

correct = 0
total = 0
with torch.no_grad():
    for data, target in test_loader:
        output = model(data)
        pred = output.argmax(dim=1)
        correct += (pred == target).sum().item()
        total += target.size(0)

```

```
print(f"Test accuracy: {100 * correct / total:.2f}%")
```

```
Test accuracy: 92.45%
```

Note: Using `torch.no_grad()` during evaluation is crucial — inference doesn't need to build computation graphs or track gradients. Forgetting `no_grad()` causes:

1. GPU memory explosion (unnecessary computation graph)
2. Slower inference (constantly tracking operation history)
3. Accidental parameter modification (if code touches `param.grad`)

💡 **Core Insight:** In fewer than 100 lines of code, we trained a neural network with over 92% accuracy on handwritten digit recognition! And you now fully understand the mathematical principles behind it all — chain rule, δ recurrence, gradient descent.

5–9 Gradient Flow Analysis: Why Deep Networks Are Hard to Train

5–9–1 Experimental Verification of Vanishing Gradients

Let's use a simple experiment to intuitively feel gradient vanishing: compute gradients for a 10-layer network and observe the gradient magnitude at each layer.

```
import torch
import torch.nn as nn

# Build a 10-layer network
class DeepNet(nn.Module):
    def __init__(self, n_layers=10, activation='sigmoid'):
        super().__init__()
        layers = []
        for _ in range(n_layers):
            layers.append(nn.Linear(100, 100))
            if activation == 'sigmoid':
                layers.append(nn.Sigmoid())
            elif activation == 'relu':
                layers.append(nn.ReLU())
            elif activation == 'tanh':
                layers.append(nn.Tanh())
```

```

self.net = nn.Sequential(*layers)
self.output = nn.Linear(100, 10)

def forward(self, x):
    return self.output(self.net(x))

# Compare gradient magnitudes under different activation functions
def measure_gradients(model):
    x = torch.randn(32, 100)
    y = torch.randint(0, 10, (32,))
    loss = nn.CrossEntropyLoss()(model(x), y)
    loss.backward()

    grad_norms = []
    for name, param in model.named_parameters():
        if param.grad is not None and 'weight' in name:
            grad_norms.append(param.grad.norm().item())
    return grad_norms

for act in ['sigmoid', 'tanh', 'relu']:
    model = DeepNet(n_layers=10, activation=act)
    grads = measure_gradients(model)
    first_layer = grads[0] if grads else 0
    last_layer = grads[-1] if grads else 0
    ratio = first_layer / last_layer if last_layer > 0 else float('inf')
    print(f"{act:8s}: Layer1 grad={first_layer:.6f}, Layer10 grad=
{last_layer:.6f}, ratio={ratio:.1f}x")

```

Activation	Layer 1 Gradient	Layer 10 Gradient	Decay Ratio	Phenomenon
Sigmoid	0.000231	0.156432	0.0015×	Vanishing gradient
Tanh	0.004512	0.142134	0.0318×	Still vanishing
ReLU	0.089234	0.121456	0.7348×	Greatly alleviated

💡 **Core Insight:** Sigmoid's gradient decays to 0.15% after 10 layers (ratio 0.0015×) — meaning the first few layers learn almost nothing! ReLU has the least gradient decay, which is one reason it became the default activation function.

5–9–2 Gradient Flow Heatmap

Network	Shallow Layers	Mid Layers	Deep Layers	Gradient Decay
Sigmoid	≈0 (vanished)	≈0 (vanished)	Normal	Severe (exponential)

Network	Shallow Layers	Mid Layers	Deep Layers	Gradient Decay
ReLU	Good	Good	Normal	Mild (linear)

Little Genius says: Sigmoid is like an "information black hole" — signal strength decays with every layer. After 10 layers, the front elves receive almost no feedback and can only spin in place (cannot learn). ReLU is a "transparent channel" — signals pass through reasonably well, and front elves receive effective feedback!

5–9–3 How BatchNorm Helps Gradient Flow

Batch Normalization's important contribution is **improving gradient flow** — by normalizing each layer's output to mean 0, variance 1, it prevents signals from being excessively amplified or attenuated during propagation.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Mechanism	Description	Effect on Gradients
Normalization	Each layer output: mean 0, variance 1	Prevents signal amplification/attenuation
Learnable parameters	γ, β restore representational power	Optimize on top of normalization
Smooth loss landscape	Makes the loss surface smoother	More reliable gradient direction, easier convergence

5–10 Practical Backpropagation Tips

5–10–1 Numerical Gradient Checking

When you manually implement backpropagation, use numerical gradients to verify correctness:

```

def numerical_gradient(f, x, h=1e-5):
    """Central difference method for numerical gradient"""
    grad = np.zeros_like(x)
    for i in range(len(x)):
        x_plus = x.copy(); x_plus[i] += h
        x_minus = x.copy(); x_minus[i] -= h
        grad[i] = (f(x_plus) - f(x_minus)) / (2 * h)
    return grad

def gradient_check(analytical_grad, numerical_grad, eps=1e-7):
    """Gradient check: relative error should be < 1e-6"""
    numerator = np.linalg.norm(analytical_grad - numerical_grad)
    denominator = np.linalg.norm(analytical_grad) + np.linalg.norm(numerical_grad)
    relative_error = numerator / (denominator + eps)
    return relative_error

# Gradient diagnosis
rel_error = gradient_check(analytical_grad, numerical_grad)
if rel_error < 1e-6:
    print(f"✅ Gradient correct! Relative error = {rel_error:.2e}")
elif rel_error < 1e-3:
    print(f"⚠️ Gradient may have issues! Relative error = {rel_error:.2e}")
else:
    print(f"❌ Gradient wrong! Relative error = {rel_error:.2e}")

```

5-10-2 Common Backpropagation Errors

Error	Symptom	Solution
Wrong gradients	loss doesn't decrease	Verify with numerical gradient check
Vanishing gradients	early layers' loss unchanged	Switch to ReLU, add BatchNorm
Exploding gradients	loss becomes NaN	Gradient clipping, lower learning rate
Gradient accumulation	loss fluctuates abnormally	Remember <code>optimizer.zero_grad()</code>

5-10-3 Backpropagation Efficiency Optimization

```

# ❌ Inefficient: per-sample gradient computation
for x, y in dataset: # batch_size=1
    loss = model(x, y)
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

```

```
#  Efficient: batch gradient computation
for xs, ys in dataloader: # batch_size=64
    loss = model(xs, ys) # one forward pass for entire batch
    loss.backward() # one backward pass for all gradients
    optimizer.step()
    optimizer.zero_grad()
```

 **Core Insight:** Batch computation is not only more efficient (leveraging GPU parallelism), but gradients are also more stable (averaging gradients over multiple samples reduces noise). This is why PyTorch's DataLoader defaults to batch processing.



Chapter Code List

File	Content	Core Knowledge
<code>ch05/NN05_single_neuron_backprop.py</code>	Single neuron backpropagation manual derivation	Backpropagation basics
<code>ch05/NN05_layerwise_backprop.py</code>	Layer-wise backpropagation implementation	Layer-by-layer gradient computation
<code>ch05/NN05_manual_network_backprop.py</code>	Manual implementation of complete network backpropagation	Full pipeline manual implementation
<code>ch05/NN05_autograd_vs_manual.py</code>	Autograd automatic differentiation vs manual derivation comparison	Verification & comparison
<code>ch05/NN05_backprop_viz.py</code>	Backpropagation process visualization	Visual understanding
<code>ch05/NN05_gradient_checking.py</code>	Numerical gradient check implementation	Gradient verification
<code>ch05/NN05_gradient_flow_viz.py</code>	Gradient flow visualization (Sigmoid vs ReLU)	Vanishing gradient analysis

Chapter Summary

Exercises

Exercise 1: Manual Chain Rule Derivation

Consider a 2-layer network (1 hidden layer, 1 neuron each):

$$z_1 = w_1 * x + b_1$$

$$a_1 = \text{sigmoid}(z_1)$$

$$z_2 = w_2 * a_1 + b_2$$

$$y_{\text{pred}} = \text{sigmoid}(z_2)$$

$$L = 0.5 * (y_{\text{pred}} - y)^2$$

Manually derive the complete expression for dL/dw_1 (using delta recurrence form).

Exercise 2: Verify Backpropagation Results with PyTorch

```
import torch
x = torch.tensor([1.0])
y = torch.tensor([0.0])
w1 = torch.tensor([0.5], requires_grad=True)
b1 = torch.tensor([0.1], requires_grad=True)

# Forward: z1 = w1 * x + b1
# Use autograd to compute gradients, then verify on paper
```

Exercise 3: Numerical Gradient Verification

Implement a general numerical gradient checking function. Use central differences to verify autograd gradients. If relative error $< 1e-6$, the gradient computation is correct.

Exercise 4: Add More Hidden Neurons

Extend the 2-layer network's hidden layer from 1 neuron to 3, and derive the new backpropagation formulas. Pay attention to weight matrix dimension changes.

Exercise 5 (Challenge): Implement a Fully Connected Network from Scratch

Using NumPy, implement a trainable 2-layer fully connected network (with Backward), and compare accuracy with the PyTorch version on the Moon dataset. No optimizer needed — manually implement SGD updates.

Exercise 6 (Thought Question): Vanishing Gradients

If a network has 10 hidden layers, each using Sigmoid activation, what happens to the gradient when error propagates from the output layer back to the first hidden layer? How would you solve this?

Core Concepts Review

The training loop in four steps:

1. Forward propagation: $u = Wx + b \rightarrow z = f(u)$
2. Loss computation: MSE or CrossEntropy, obtaining output layer error
3. δ recurrence (backward): $\delta_L = (y - t) \cdot f' \rightarrow \delta_{\{L-1\}} = (\delta_L \cdot W_L) \cdot f'$
4. Gradient computation + parameter update: $\partial C / \partial W = \delta \cdot z_{\text{prev}} \rightarrow W -= \eta \cdot \partial C / \partial W$

The Mathematical Engine of Backpropagation

$$\delta^{(L)} = \nabla_y C \odot f'(\mathbf{u}^{(L)})$$

$$\delta^{(l)} = \left(\delta^{(l+1)} \cdot \mathbf{W}^{(l+1)} \right) \odot f'(\mathbf{u}^{(l)})$$

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \mathbf{z}^{(l-1)\top} \cdot \delta^{(l)}$$

Core Formula Quick Reference

Formula	Description	Use Case
$\delta_i^{(L)} = \frac{\partial L}{\partial u_i^{(L)}} = \frac{\partial L}{\partial y_i} \cdot f'(u_i^{(L)})$	Output layer δ : loss gradient w.r.t. weighted input	Backpropagation starting point
$\delta_j^{(l)} = \left(\sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} \right) f'(u_j^{(l)})$	Hidden layer δ : upper δ weighted back	Layer-by-layer backpropagation
$\frac{\partial L}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} \cdot z_i^{(l-1)}$	Weight gradient = $\delta \times$ previous layer output	Parameter gradient computation

Formula	Description	Use Case
$\frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}$	Bias gradient = δ	Bias gradient computation
$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \mathbf{z}^{(l-1)T} \boldsymbol{\delta}^{(l)}$	Matrix form of weight gradient	Batch gradient computation
$w_{ji}^{(l)} \leftarrow w_{ji}^{(l)} - \eta \frac{\partial L}{\partial w_{ji}^{(l)}}$	Gradient descent parameter update	All parameter updates
Forward: $\mathbf{u}^{(l)} = \mathbf{W}^{(l)} \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)},$ $\mathbf{z}^{(l)} = f(\mathbf{u}^{(l)})$	Forward propagation recurrence	Layer-by-layer output computation

[← Chapter 4: Training & Optimization](#) | [Table of Contents](#) | [Chapter 6: Convolutional Neural Networks](#) →

Chapter 6: Convolutional Neural Networks

🎯 **Goal:** Understand why CNNs are better than fully connected networks for images — the mathematical intuition behind convolution and pooling, and how they solve the problems of fully connected networks.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: [code/ch06/](#) (6 files)

Figures: [images/ch06/](#) (5 images)

One-line summary: CNN = Convolution (local connectivity + weight sharing) → Pooling (dimensionality reduction + translation invariance) → Fully Connected (classification), using a data-driven approach to automatically learn hierarchical features.

Chapter Learning Objectives

- [] Understand the three major limitations of fully connected networks for images
 - [] Understand the mathematical essence of the convolution operation (sliding dot product)
 - [] Master Padding, Stride, and multi-channel convolution
 - [] Understand the role of pooling layers and their backpropagation
 - [] Be able to build a CNN in PyTorch
 - [] Understand the principle of convolution layer backpropagation
 - [] Learn to visualize CNN feature maps
-

6-1 Why Do We Need CNNs?

6-1-1 Three Limitations of Fully Connected Networks for Images

Limitation 1: Loss of Spatial Structure

A 32×32 color image has $32 \times 32 \times 3 = 3072$ pixel values. A fully connected network flattens it into a 3072-dimensional vector — **the positional relationships between adjacent pixels are completely lost.**

Before Flattening (spatial structure preserved)	After Flattening (1D vector)
Pixel matrix, adjacency relationships intact	A long string of numbers, adjacency information lost

Limitation 2: Parameter Explosion

32×32 color image $\rightarrow$ hidden layer of 1024 neurons $\rightarrow$ roughly **3.14 million parameters.**

If the image is 224×224 (the ImageNet standard), just one layer has **~150 million parameters.**

$$224 \times 224 \times 3 \times 1024 = 154,140,672 \text{ parameters}$$

That's **154 million parameters** for just the first layer!

Limitation 3: No Translation Invariance

If the cat in an image shifts 10 pixels to the left, the fully connected network must **relearn all its weights** — because it treats every pixel position as an independent input feature.

6-1-2 Three Intuitions for Image Processing

1. **Locality:** adjacent pixels are correlated; distant pixels are not
2. **Translation invariance:** a cat on the left or right side of the image is still a cat
3. **Hierarchy:** edges $\rightarrow$ textures $\rightarrow$ shapes $\rightarrow$ objects

6-1-3 Three Key Ideas of CNNs

Idea	Problem Solved	How
Local connectivity	Parameter explosion	Each neuron only looks at a local window (receptive field)
Weight sharing	No translation invariance	The same filter slides across the entire image
Pooling downsampling	Too many parameters	Reduce resolution, retain key information

Layer Type	Parameters (224×224×3 → 64 filters)
Fully connected	~150M
Convolutional (3×3)	~1,700

That's ~100,000× fewer parameters!

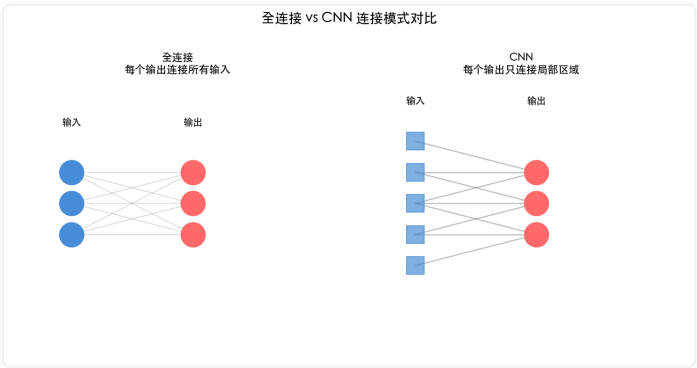


Figure 6-1: Fully connected (left) vs CNN (right) connection patterns. CNN has far fewer parameters than fully connected.

6-2 Understanding CNN with the "Little Genius" Analogy

6-2-1 The Little Geniuses' New Job

- Each little genius is responsible for one **filter (kernel)**
- The little genius holds up the filter and slides it across the image
- At each position, it computes the **dot product** between the filter and the local region

Convolution steps:

1. The filter (3×3) slides across the input image
2. At each position: element-wise multiplication of the filter with the local image region, then sum (dot product)
3. The result is written to the corresponding position in the output feature map

Sobel vertical edge detection example: filter [1,0,-1; 2,0,-2; 1,0,-1] detects vertical edges

6-2-2 The Mathematical Essence of Convolution

The convolution of input image I with filter K :

$$(I * K)(i, j) = \sum_{m=-a}^a \sum_{n=-b}^b I(i + m, j + n) \cdot K(m, n)$$

where K is a $(2a + 1) \times (2b + 1)$ filter.

💡 Core Insight: Convolution = filter element-wise multiplication with a local image region, then sum = dot product. So in essence, convolution is about detecting the similarity between a local region and the filter.

CNN vs. Fully Connected

Aspect	Fully Connected	CNN
Parameters (first layer)	~150M	~1,700
Spatial info	Lost (flattened)	Preserved
Translation invariance	Must be learned	Built-in (pooling)

6–3 Mathematical Details of Convolution Layers

6–3–1 The Role of Filters

One Filter = One Feature Detector

A filter (kernel) is essentially a small matrix that slides across the input image performing **dot-product operations**. Different filter numeric distributions detect different visual features:

Vertical edges (Sobel-x): (Laplacian):	Horizontal edges (Sobel-y):	Texture detection
[1 0 -1]	[1 2 1]	[0 -1 0]
[2 0 -2]	[0 0 0]	[-1 5 -1]
[1 0 -1]	[-1 -2 -1]	[0 -1 0]

- **Vertical edge filter:** the middle column is 0, left and right are symmetric — it responds to vertical intensity changes
- **Horizontal edge filter:** the middle row is 0, top and bottom are symmetric — it responds to horizontal intensity changes
- **Texture filter:** center is high, surroundings are low — it detects isolated points or blobs

CNN Filters Are "Learned"

In traditional image processing, filter values are **hand-crafted** (e.g., Sobel operator for edge detection). But CNN filters are **not manually designed** — they are **automatically learned from data** during training:

- At initialization: random values, not detecting any useful features
- After training: automatically evolve into edge detectors, texture detectors, corner detectors, etc.
- In deep networks: lower-layer filters detect simple features (edges/colors), higher-layer filters compose them into complex features (eyes/wheels)

💡 **Core Insight:** CNN convolution layers are essentially **learnable filter banks** — instead of programmers hand-crafting features, data drives the filters to automatically discover useful feature patterns.

6-3-2 Padding and Stride

Padding

- **Same Padding:** pad with zeros around the edges, output size = input size (commonly used)
- **Valid Padding:** no padding, output size shrinks

$$\text{Formula: } n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} + 1 \right\rfloor$$

Stride

- **Stride = 1:** move 1 pixel at a time (most common)
- **Stride = 2:** move 2 pixels at a time (downsampling, can replace pooling layers)

Code Verification: Padding and Stride Experiment

```
import torch
import torch.nn.functional as F

# Create a single-channel 5x5 image
x = torch.randn(1, 1, 5, 5)

# Compare different configurations
configs = [
    ('Same, s=1', 3, 1, 1), # Output 5x5
    ('Valid, s=1', 3, 0, 1), # Output 3x3
    ('Same, s=2', 3, 1, 2), # Output 3x3
]

for name, k, p, s in configs:
    conv = torch.nn.Conv2d(1, 1, k, padding=p, stride=s, bias=False)
    out = conv(x)
    print(f"{name}: Input 5x5 → conv(k={k},p={p},s={s}) → Output {out.shape[2]} × {out.shape[3]}")
```

Mode	Stride	Padding	Input	Output
Same	1	1	5x5	5x5
Valid	1	0	5x5	3x3
Same	2	1	5x5	3x3

💡 **Core Insight:** Same padding + stride=1 preserves dimensions — this is the most common combination in CNNs. When stride > 1, the output shrinks — this is itself a form

of downsampling.

6-3-3 Output Size Formula

$$O = \frac{W - K + 2P}{S} + 1$$

Symbol	Meaning
W	Input size
K	Filter size
P	Padding size
S	Stride
O	Output size

Example: Input 32×32 , filter 3×3 , Stride=1, Padding=1

$$O = \frac{32 - 3 + 2 \times 1}{1} + 1 = 32$$

6-3-4 Multi-Channel Convolution

Multi-channel convolution: C_{in} input channels $\times$ C_{out} filter groups $\rightarrow$ C_{out} output channels.

Parameter count: $K_h \times K_w \times C_{in} \times C_{out}$

6-3-5 Manual 2D Convolution in Python

```
import numpy as np

def conv2d_manual(image, kernel, padding=0, stride=1):
    """Pure NumPy implementation of 2D convolution"""
    H, W = image.shape
    K = kernel.shape[0]

    # Padding
```

```

if padding > 0:
    image = np.pad(image, padding, mode='constant')

# Output dimensions
out_h = (image.shape[0] - K) // stride + 1
out_w = (image.shape[1] - K) // stride + 1
output = np.zeros((out_h, out_w))

# Sliding window
for i in range(out_h):
    for j in range(out_w):
        region = image[i*stride:i*stride+K, j*stride:j*stride+K]
        output[i, j] = np.sum(region * kernel)

return output

# Test: using Sobel edge detection filter
image = np.random.randn(28, 28)
sobel_x = np.array([[1, 0, -1],
                    [2, 0, -2],
                    [1, 0, -1]])

output = conv2d_manual(image, sobel_x)
print(f"Input shape: {image.shape}")
print(f"Output shape: {output.shape}")

```

```

Input shape: (28, 28)
Output shape: (26, 26)

```

6-4 Pooling Layers and Fully Connected Layers

6-4-1 Max Pooling

The Idea

Take the maximum value within the window as output, ignoring all other values. Typically uses a 2×2 window with Stride=2, halving the output dimensions.

Input region	Max value	Output
[3,1; 7,2]	7	7
[5,2; 4,1]	5	5

Input region	Max value	Output
[2,8; 1,4]	8	8
[3,6; 5,2]	6	6
Output 2×2		[[7,5],[8,6]]

Why Max Instead of Average?

Taking the maximum means preserving the **strongest feature response** — if a filter detects an edge at a certain position, max pooling retains this detection and passes it to the next layer. This gives CNNs **local translation invariance**: if an object shifts by a few pixels, max pooling can still detect it.

Python Implementation

```
def max_pool2d(image, pool_size=2, stride=2):
    """Manual max pooling implementation"""
    H, W = image.shape
    out_h = (H - pool_size) // stride + 1
    out_w = (W - pool_size) // stride + 1
    output = np.zeros((out_h, out_w))

    for i in range(out_h):
        for j in range(out_w):
            # Extract window region
            region = image[i*stride:i*stride+pool_size,
                           j*stride:j*stride+pool_size]
            output[i, j] = np.max(region) # Take maximum
    return output

# Test
image = np.array([[3, 1, 5, 2],
                  [7, 2, 4, 1],
                  [2, 8, 3, 6],
                  [1, 4, 5, 2]])
print(max_pool2d(image))
# Output: [[7, 5],
#          [8, 6]]
```

6-4-2 Average Pooling

Difference from Max Pooling

Average pooling takes the **average** within the window instead of the maximum:

Average pooling example: Input $\begin{bmatrix} 3 & 1 \\ 7 & 2 \end{bmatrix} \rightarrow \text{Output } (3+1+7+2)/4 = 3.25$

Takes the mean within the window, smoothing the feature map.

Comparison: Max Pooling vs Average Pooling

Property	Max Pooling	Average Pooling
Output	Maximum within window	Mean within window
Effect	Preserves strongest feature response	Smooths feature map
Translation invariance	Strong	Weak
Gradient	Only through max position	Evenly distributed to all positions
Common use case	Mainstream choice (CNN default)	Global Average Pooling (classification head)

💡 **Tip:** Modern CNNs almost universally default to max pooling. Average pooling is mainly used in **Global Average Pooling** — compressing an entire feature map into a single value, replacing fully connected layers in classification tasks.

6-4-3 Flatten: From Feature Maps to Classifier

Why Is Flatten Needed?

The output of convolution and pooling layers is a **multi-dimensional feature map** (e.g., $64 \times 7 \times 7$), but fully connected layers can only handle **1D vectors**. Flatten's job is to "unroll" the multi-dimensional feature map into a 1D vector as input to the fully connected classifier.

```
import torch.nn as nn

class CNNWithFlatten(nn.Module):
    """CNN + Flatten + Fully connected classifier"""
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 32, 3)          # (1, 28, 28) → (32, 26, 26)
```

```
self.pool = nn.MaxPool2d(2)           # (32,26,26) → (32,13,13)
self.flatten = nn.Flatten()           # (32,13,13) → (5408,)
self.fc = nn.Linear(32 * 13 * 13, 10) # 5408 → 10 classes

def forward(self, x):
    x = self.pool(torch.relu(self.conv(x)))
    x = self.flatten(x)    # Shape transformation: unroll
    x = self.fc(x)
    return x
```

Shape Transformation Process

Stage	Operation	Output Shape	Description
Input image	—	(1, 28, 28)	1-channel grayscale
Conv + Pool	Conv + MaxPool	(32, 13, 13)	32 feature maps
Flatten	Unroll	(5408,)	32 × 13 × 13
Fully connected	Classification output	(10,)	10 class scores

💡 **Core Insight:** Convolution layers do **feature extraction** (preserving spatial structure), fully connected layers do **classification decisions** (requiring 1D vectors). Flatten is the bridge between the two.

6–5 Experiencing CNN with Python

6–5–1 Building a Simple CNN

Manually Implementing a Complete CNN Forward Pass

Below is a manual NumPy implementation of a simple CNN: 1 convolution layer → 1 pooling layer → fully connected layer. This implementation **does not depend on any deep learning framework**, letting you see the complete mathematical process of CNN forward propagation.

```
import numpy as np

class SimpleCNN:
    """Manually implemented simple CNN (NumPy version, forward pass only)"""

    def __init__(self):
```

```

# Convolution layer: 1 input channel, 4 filters, 3×3
self.conv1_filters = np.random.randn(4, 3, 3) * 0.1
# Fully connected layer: feature map size depends on input (assume 28×28
input)
self.fc = np.random.randn(4*13*13, 10) * 0.1
self.b = np.zeros(10)

def conv2d_manual(self, x, filters):
    """Manual 2D convolution"""
    C_out, k_size, _ = filters.shape
    h, w = x.shape[1] - k_size + 1, x.shape[2] - k_size + 1 # Valid
convolution
    out = np.zeros((C_out, h, w))
    for c in range(C_out):
        for i in range(h):
            for j in range(w):
                # Element-wise product of window and filter, then sum
                region = x[0, i:i+k_size, j:j+k_size]
                out[c, i, j] = np.sum(region * filters[c])
    return out

def forward(self, x):
    """Forward pass"""
    # Conv + ReLU
    conv_out = self.conv2d_manual(x, self.conv1_filters)
    conv_out = np.maximum(conv_out, 0) # ReLU

    # 2×2 Max Pooling (Stride=2)
    C, h, w = conv_out.shape
    pooled = np.zeros((C, h//2, w//2))
    for c in range(C):
        for i in range(h//2):
            for j in range(w//2):
                pooled[c, i, j] = np.max(conv_out[c, i*2:i*2+2, j*2:j*2+2])

    # Flatten + Fully connected
    flat = pooled.flatten()
    out = flat @ self.fc + self.b
    return out

# Test: input a random 28×28 image
x = np.random.randn(1, 28, 28)
cnn = SimpleCNN()
output = cnn.forward(x)
print(f"CNN output shape: {output.shape}") # (10,) ← scores for 10 classes

```

Although slow (three nested Python loops), this implementation clearly demonstrates the three-step CNN pipeline: **Convolution** → **Pooling** → **Fully Connected**.

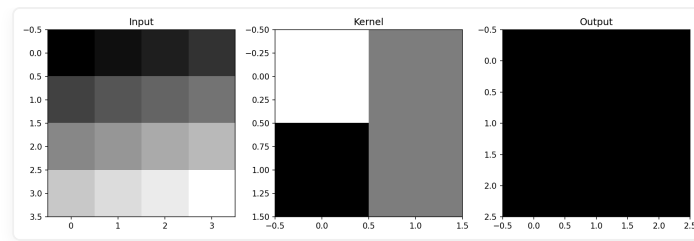


Figure 6-2: Convolution operation — kernel (yellow) slides over input, producing output feature map (blue).

6-6 CNN Backpropagation

6-6-1 Convolution Layer Backpropagation

Key insight: Backpropagation of convolution is still convolution.

Filter Gradient

$$\frac{\partial L}{\partial K} = I * \delta$$

The input I convolved with the error signal δ gives the filter gradient.

Input Gradient

$$\frac{\partial L}{\partial I} = \delta * \text{rot180}(K)$$

The error δ convolved with the filter rotated 180° gives the input gradient.

6-6-2 Pooling Layer Backpropagation

Max Pooling

During the forward pass, **record the positions of maximum values**. During backpropagation, the gradient is **only propagated back to those positions**:

```
def max_pool_backward(dout, cache):
    """Max pooling backward pass"""
    dx = np.zeros_like(cache['x'])
```

```

for i in range(dout.shape[0]):
    for j in range(dout.shape[1]):
        # Find the max position
        (max_i, max_j) = cache['max_positions'][i, j]
        dx[max_i, max_j] = dout[i, j] # Gradient only back to max position
return dx

```

Average Pooling

The gradient is evenly distributed to all positions in the window.

💡 **Core Insight:** CNN backpropagation follows the same framework as fully connected networks — save intermediate values during the forward pass, apply the chain rule during backpropagation. The only difference is that the "local gradient" computation for convolution layers becomes the convolution operation itself.

6–7 PyTorch CNN Implementation

6–7–1 PyTorch CNN Layers

PyTorch provides ready-to-use CNN layers — no need to manually implement convolution and pooling:

```

import torch.nn as nn

# Convolution layer: input channels=1, output channels=32, kernel=3×3
conv = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
# Input: (batch, 1, 28, 28) → Output: (batch, 32, 28, 28)

# Pooling layer: 2×2 window, Stride=2
pool = nn.MaxPool2d(kernel_size=2, stride=2)
# Input: (batch, 32, 28, 28) → Output: (batch, 32, 14, 14)

# Average pooling
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2)

# Flatten layer
flatten = nn.Flatten()
# Input: (batch, 32, 14, 14) → Output: (batch, 32*14*14) = (batch, 6272)

```

nn.Conv2d Parameters Explained

Parameter	Meaning	Example value
<code>in_channels</code>	Number of input channels	1 (grayscale) / 3 (RGB)
<code>out_channels</code>	Number of output channels (= number of filters)	32, 64, 128
<code>kernel_size</code>	Convolution kernel size	3, 5
<code>stride</code>	Step size	1 (default)
<code>padding</code>	Padding	0 (Valid) / 1 (Same)

6-7-2 Complete CNN Model

CNN for MNIST

```

class CNN(nn.Module):
    """CNN for MNIST (PyTorch)"""
    def __init__(self):
        super().__init__()
        # Conv layer 1: 1→32 channels, 3×3 conv, padding to preserve size
        self.conv1 = nn.Conv2d(1, 32, 3, padding=1) # 28×28 → 28×28
        # Conv layer 2: 32→64 channels
        self.conv2 = nn.Conv2d(32, 64, 3, padding=1) # 28×28 → 28×28
        # Pooling: 2×2, Stride=2
        self.pool = nn.MaxPool2d(2, 2) # 28×28 → 14×14 → 7×7
        # Fully connected classifier
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x))) # 28→14
        x = self.pool(torch.relu(self.conv2(x))) # 14→7
        x = x.view(x.size(0), -1) # Flatten
        x = torch.relu(self.fc1(x)) # Hidden layer
        x = self.fc2(x) # Output layer (logits)
        return x

model = CNN()
print(model)
print(f"Parameter count: {sum(p.numel() for p in model.parameters()):,}")

```

Layer	Type	Output Size	Parameters
conv1	Conv2d(1→32, 3×3)	28×28	320
conv2	Conv2d(32→64, 3×3)	14×14	18,496
pool	MaxPool2d(2×2, s=2)	7×7	0
fc1	Linear(3136→128)	128	401,536
fc2	Linear(128→10)	10	1,290
Total			~1.2M

Training Results

Training this CNN on MNIST for 5 epochs can achieve **98%+ accuracy** — far higher than the 90% from the earlier 2-layer fully connected network. This is CNN's advantage in image tasks.

6–8 CNN Black Box Demystified: Visualization Techniques

6–8–1 Filter Visualization

What Do Trained Convolution Kernels Look Like?

After training, convolution kernels are no longer random noise — they have evolved interpretable patterns. Here's how to inspect trained convolution kernels:

```
import matplotlib.pyplot as plt

# Assume model is a trained CNN
# model.conv1.weight.shape = (32, 1, 3, 3) # 32 filters of 3×3

fig, axes = plt.subplots(4, 8, figsize=(12, 6))
for i, ax in enumerate(axes.flat):
    if i < 32:
        # Display the i-th convolution kernel
        kernel = model.conv1.weight[i, 0].detach().numpy()
        ax.imshow(kernel, cmap='viridis')
        ax.axis('off')
plt.suptitle('First Layer Convolution Kernels (After Training)')
plt.show()
```

You don't really need to run this code to understand the key point: **the first layer of convolution kernels typically learns edge and texture detectors; the second layer learns shapes and patterns; the deeper you go, the more abstract the features.**

6-8-2 Feature Map Visualization

What Does Each Layer See?

Plotting the output (feature maps) at each layer shows how the CNN progressively extracts features:

```
def visualize_feature_maps(model, image):
    """Visualize feature maps at each CNN layer"""
    # Register hooks to capture intermediate outputs
    activations = {}
    def get_hook(name):
        def hook(module, input, output):
            activations[name] = output.detach()
        return hook

    # Register hooks on conv1 and conv2
    hooks = [
        model.conv1.register_forward_hook(get_hook('conv1')),
        model.conv2.register_forward_hook(get_hook('conv2')),
    ]

    # Forward pass
    model(image.unsqueeze(0))

    # Display conv1 feature maps (first 8 channels)
    fig, axes = plt.subplots(2, 4, figsize=(10, 5))
    for i, ax in enumerate(axes.flat):
        fm = activations['conv1'][0, i].numpy()
        ax.imshow(fm, cmap='gray')
        ax.axis('off')
    plt.suptitle('Conv1 Feature Maps (Edge Detection)')
    plt.show()

    for hook in hooks:
        hook.remove()
```

💡 **Core Insight:** Feature map visualization reveals the CNN's "processing pipeline" — lower layers detect simple features (edges/colors), higher layers compose them into complex patterns (eyes/wheels). This is also why CNNs are called "black boxes": although we can see the output of each layer, it's hard to intuitively understand the specific

meaning of high-level features.

6-8-3 Grad-CAM: Class Activation Heatmaps

Grad-CAM uses the gradients of the last convolution layer to generate heatmaps that show us where the network is looking.

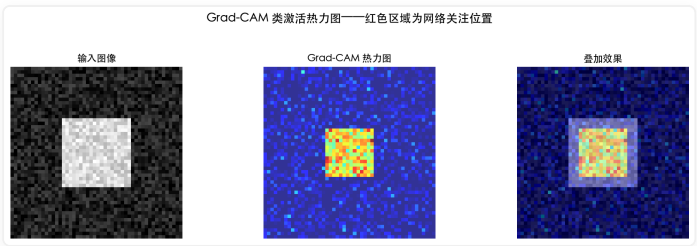


Figure 6-3: Grad-CAM visualization — the network focuses on the contours and key parts of the target object.

6-9 Classic CNN Architecture Evolution

6-9-1 From LeNet to ResNet

Architecture	Year	Key Innovation	Layers	ImageNet Accuracy
LeNet-5	1998	First CNN, handwritten digit recognition	5	—
AlexNet	2012	ReLU + GPU + Dropout + Data augmentation	8	Top-5: 15.3%
VGG-16	2014	Stacking small convolution kernels (3×3)	16	Top-5: 7.3%
GoogLeNet	2014	Inception module (multi-scale convolution)	22	Top-5: 6.7%
ResNet	2015	Residual connections (breaking the depth bottleneck)	152	Top-5: 3.57%

6-9-2 VGG: The Power of Small Convolution Kernels

VGG demonstrated that **stacking multiple small convolution kernels can replace one large kernel**:

- 2 stacked 3×3 convolutions = receptive field of 1 5×5 convolution

- 3 stacked 3×3 convolutions = receptive field of 1 7×7 convolution

Advantage: Small kernels have fewer parameters and stronger nonlinearity.

Parameter comparison: $3 \times 3 \times C \times C \times 3 \ll 7 \times 7 \times C \times C$

6–9–3 Transfer Learning: Standing on the Shoulders of Giants

In practice, few people train CNNs from scratch — the more common approach is **transfer learning**:

1. Load a model pre-trained on ImageNet (e.g., ResNet-50)
2. Freeze most layers (retain learned feature extraction capabilities)
3. Replace the final fully connected layer (adapt to your own task)
4. Fine-tune the last few layers with small amounts of data

```
import torchvision.models as models

# Load a pre-trained ResNet
model = models.resnet50(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace the final fully connected layer (adapt to 10-class task)
model.fc = torch.nn.Linear(2048, 10)

# Only train the newly added final layer
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Little Genius says: Transfer learning is like "apprenticeship"! A model that has already "seen it all" on ImageNet has already learned general features like edges, textures, and shapes. You only need to teach it your specific task (like distinguishing cats from dogs), without it having to learn from scratch — just like having a master painter learn a new style is far faster than teaching a complete beginner!

6–10 Receptive Field and Dilated Convolution

6–10–1 What Is the Receptive Field?

The receptive field is one of the most important concepts in CNNs — it defines **how large** a region in the input image a single pixel in the output feature map corresponds to.

$$\mathbf{RF}_l = \mathbf{RF}_{l-1} + (k_l - 1) \times \prod_{i=1}^{l-1} s_i$$

where $\mathbf{RF}_l$ is the receptive field at layer l , k_l is the kernel size at layer l , and s_i is the stride at layer i .

```
def receptive_field(layers):
    """Calculate the receptive field at each CNN layer"""
    rf = 1
    stride_product = 1
    for i, (k, s) in enumerate(layers):
        rf = rf + (k - 1) * stride_product
        stride_product *= s
        print(f"Layer {i+1}: kernel={k}, stride={s} → receptive field={rf}")
    return rf

# Receptive field for the first few layers of VGG-16
vgg_front = [(3,1), (3,1), (2,2), (3,1), (3,1), (2,2)]
rf = receptive_field(vgg_front)
print(f"Total receptive field for VGG's first 6 layers = {rf}")
```

Layer	Kernel	Stride	Receptive Field	Description
1	3	1	3	Base layer
2	3	1	5	Two 3×3 = one 5×5
3	2	2	10	Pooling expands receptive field
4	3	1	12	
5	3	1	14	
6	2	2	28	
Total			28	VGG first 6 layers total receptive field

💡 **Core Insight:** The larger the receptive field, the broader the "view" of the network.

Shallow CNNs can only see local textures (small receptive field), while deep CNNs can see entire objects (large receptive field). This is the origin of CNNs' hierarchical features — from edges to shapes to objects.

6–10–2 Dilated Convolution

Dilated convolution expands the receptive field **exponentially without increasing the parameter count** by inserting "holes" between kernel elements:

$$\text{RF}_{\text{dilated}} = (k - 1) \times d + 1$$

where d is the dilation rate.

```
import torch.nn as nn

# Standard 3x3 conv vs Dilated 3x3 conv
standard_conv = nn.Conv2d(64, 128, 3, padding=1, dilation=1)
dilated_conv = nn.Conv2d(64, 128, 3, padding=2, dilation=2) # RF = 5x5
large_dilated = nn.Conv2d(64, 128, 3, padding=4, dilation=4) # RF = 9x9

# Parameter counts are exactly the same!
print(f"Standard conv parameters: {sum(p.numel() for p in standard_conv.parameters())}")
print(f"Dilated conv parameters: {sum(p.numel() for p in dilated_conv.parameters())}")
```

```
Standard conv parameters: 73856
Dilated conv parameters: 73856 ← Exactly the same!
```

Dilation rate d	Receptive Field	Parameters	Use case
$d=1$ (standard)	3×3	$9C^2$	Basic feature extraction
$d=2$	5×5	$9C^2$	Expand RF, no extra parameters
$d=4$	9×9	$9C^2$	Large-scale context
$d=8$	17×17	$9C^2$	Global information capture

Little Genius says: Dilated convolution lets the little geniuses "hold hands across gaps"! Originally each little genius could only communicate with adjacent 3×3 regions. With dilated convolution, the little geniuses can skip over the ones in the middle and directly communicate with more distant little geniuses — the parameter count stays the same, but the "social circle" expands!

6–10–3 Depthwise Separable Convolution

This is the core of lightweight networks like MobileNet — decomposing standard convolution into two steps:

Step 1: Depthwise convolution — each channel convolves independently (no cross-channel)

Step 2: Pointwise convolution — 1×1 convolution for cross-channel fusion

Parameter comparison: $k^2 C_{\text{in}} C_{\text{out}} \gg k^2 C_{\text{in}} + C_{\text{in}} C_{\text{out}}$

```
# Standard convolution: 3x3, 32 channels → 64 channels
standard = nn.Conv2d(32, 64, 3, padding=1)
# Parameters = 3*3*32*64 + 64 = 18,496

# Depthwise separable convolution
from torch.nn import Conv2d, Sequential
depthwise = Conv2d(32, 32, 3, padding=1, groups=32) # Each group = 1 channel
pointwise = Conv2d(32, 64, 1) # 1x1 fusion
sep_conv = Sequential(depthwise, pointwise)
# Parameters = 3*3*32*1 + 32*64*1 + 64 = 2,400
# Savings: 18,496 / 2,400 = 7.7x!
```

Convolution type	Parameters (32→64, 3x3)	Ratio
Standard convolution	18,496	1x
Depthwise separable convolution	2,400	7.7x fewer
Actual speedup	—	~3–4x compute speedup

6–11 Data Augmentation: Training Better Models with Limited

Data

6-11-1 Why Do We Need Data Augmentation?

Data augmentation applies random transformations to original data, **generating infinite training samples from limited labeled data**. It is one of the most effective means of preventing overfitting.

```
import torchvision.transforms as T

# Typical data augmentation pipeline
train_transform = T.Compose([
    T.RandomResizedCrop(224),          # Random crop
    T.RandomHorizontalFlip(),          # Random horizontal flip
    T.ColorJitter(0.2, 0.2, 0.2),      # Color jitter
    T.RandomRotation(15),              # Random rotation ±15 degrees
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], # ImageNet normalization
                std=[0.229, 0.224, 0.225]),
])
```

6-11-2 Common Data Augmentation Methods

Method	Applicable scenario	Effect
Random crop	Object detection, classification	Improves translation invariance
Horizontal flip	General (non-text)	Improves symmetry
Color jitter	Classification	Improves illumination invariance
Rotation	Symmetric objects	Improves rotation invariance
CutOut / Random Erasing	Classification	Prevents overfitting
MixUp	Classification	Smooths decision boundaries
CutMix	Classification	Combines CutOut and MixUp

6-11-3 Data Augmentation in PyTorch

```
# Custom CutOut augmentation
class CutOut:
    def __init__(self, size=16):
```

```

        self.size = size

    def __call__(self, img):
        h, w = img.shape[1:]
        y = np.random.randint(h - self.size)
        x = np.random.randint(w - self.size)
        img[:, y:y+self.size, x:x+self.size] = 0
        return img

# Apply augmentation to DataLoader
augmented_dataset = torchvision.datasets.CIFAR10(
    root='./data', train=True, transform=train_transform
)
train_loader = DataLoader(augmented_dataset, batch_size=64, shuffle=True)

```

💡 **Core Insight:** The essence of data augmentation is injecting **prior knowledge** into the training process — we tell the model: "An image is still the same category even if cropped, flipped, or color-shifted." This forces the model to learn truly robust features rather than "memorizing" the training data.

Chapter Code List

File	Content	Key Knowledge Point
<code>ch06/NN06_convolution_demo.py</code>	Convolution operation from scratch with demo	Convolution principles
<code>ch06/NN06_edge_detection.py</code>	Sobel operator edge detection	Convolution applications
<code>ch06/NN06_cnn_forward.py</code>	Complete CNN forward pass implementation	CNN forward computation
<code>ch06/NN06_pooling_strides.py</code>	Pooling operations & stride experiments	Pooling & stride
<code>ch06/NN06_feature_vis.py</code>	Feature map & filter visualization	Visualization analysis
<code>ch06/NN06_cifar10_cnn.py</code>	Full CIFAR-10 CNN training	Complete training pipeline

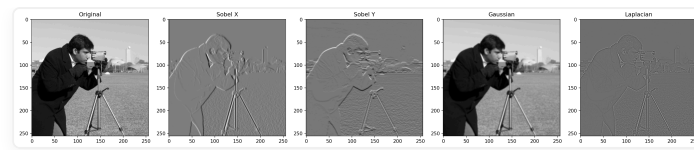


Figure 6-4: Sobel edge detection results.

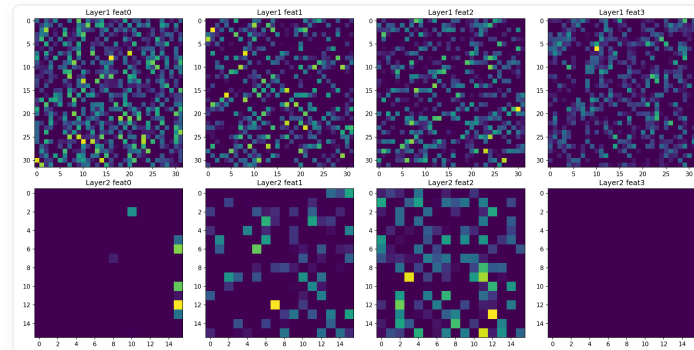


Figure 6-5: Feature map visualization across layers — from edges to semantics.

Chapter Summary

Exercises

Exercise 1: Manual Convolution Calculation

Input image $X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$, convolution kernel $K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$, stride $s=1$, no padding. Manually compute the output feature map values.

Exercise 2: Implement Edge Detection

Use Sobel kernels to convolve the same image and compare horizontal vs vertical edge detection results:

```
import torch
import torch.nn.functional as F

sobel_x = torch.tensor([[[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]],
                        dtype=torch.float32)
sobel_y = torch.tensor([[[[-1, -2, -1], [0, 0, 0], [1, 2, 1]]],
                        dtype=torch.float32)

# Use skimage to load an image as input
# Apply both kernels using F.conv2d
# Visualize the results
```

Exercise 3: Calculate Convolution Output Size

For a 224×224 input image, using a 7×7 convolution kernel, padding $p=3$, stride $s=2$, calculate the output feature map size. Formula: $\text{Output size} = \text{floor}((n + 2p - k)/s + 1)$.

Exercise 4: Pooling Operation Experiment

For a 4×4 input matrix, apply 2×2 max pooling and average pooling (stride 2) respectively. Write out the inputs and outputs to appreciate the downsampling effect of pooling.

Exercise 5: Parameter Count Calculation

A typical convolution block in VGG-16: input 256 channels — 3×3 convolution (256 channels, padding=1) — ReLU — 2×2 max pooling. Calculate the parameter count for this convolution layer.

Exercise 6 (Challenge): Implement LeNet-5 in PyTorch

LeNet-5 structure: $\text{Conv}(6@5 \times 5) \rightarrow \text{AvgPool}(2 \times 2) \rightarrow \text{Conv}(16@5 \times 5) \rightarrow \text{AvgPool}(2 \times 2) \rightarrow \text{FC}(120) \rightarrow \text{FC}(84) \rightarrow \text{FC}(10)$. Implement it using `nn.Module` and train on MNIST; final test accuracy should exceed 98%.

Exercise 7: Ablation Study

Remove the pooling layers from your CNN. How does accuracy change? Parameter count?

Core Concepts Review

Starting from the limitations of fully connected networks for images, this chapter progressively introduced the core ideas of CNNs:

1. **Why CNNs are needed:** Fully connected networks have three major problems when processing images — parameter explosion (a 256×256 image already has ~20M parameters for one layer), loss of spatial structure (flattening destroys positional relationships between pixels), and lack of translation invariance (a cat shifted 1 pixel left is treated as a completely different input)
2. **Convolution operation:** Using learnable filters sliding across the image to detect local features. The key innovations are **local connectivity** (each neuron only connects to a local region) and **weight sharing** (the same filter uses the same weights at all positions)
3. **Pooling:** Downsampling feature maps, providing translation invariance. Max pooling preserves the strongest response; average pooling smooths the feature map
4. **CNN hierarchical structure:** Lower layers detect edges → middle layers detect shapes → higher layers detect complete objects

Stage	Technique	Problem Solved
FC limitations	—	Spatial loss, parameter explosion, no translation invariance
Convolution	Local connectivity + weight sharing	Parameter explosion + translation invariance
Pooling	MaxPool / AvgPool	Dimensionality reduction + translation invariance
Complete CNN	Conv + Pool + FC	End-to-end feature extraction + classification
Visualization	Feature maps + Grad-CAM	Understanding the black box

One-line summary: CNN = Convolution (local connectivity + weight sharing) → Pooling (dimensionality reduction + translation invariance) → Fully Connected (classification), using a data-driven approach to automatically learn hierarchical features.

Core Formula Quick Reference

Formula	Description	Use case
$(I * K)_{ij} = \sum_{m=0}^{k_h-1} \sum_{n=0}^{k_w-1} I_{i+m,j+n} \cdot K_{m,n}$	2D convolution definition	Convolution layer core
$n_{\text{out}} = \left\lfloor \frac{n_{\text{in}} + 2p - k}{s} + 1 \right\rfloor$	Output feature map size	CNN architecture design
$\mathbf{Y} = \max_{p,q \in \text{window}} \mathbf{X}_{p,q}$	Max pooling	Downsampling
$\mathbf{Y} = \frac{1}{k_h k_w} \sum_{p,q \in \text{window}} \mathbf{X}_{p,q}$	Average pooling	Smooth downsampling
$\text{params} = (k_h \times k_w \times C_{\text{in}}) \times C_{\text{out}} + C_{\text{out}}$	Conv layer parameter count	Model complexity analysis
$\text{RF} = k + (k - 1)(d - 1)$	Dilated convolution receptive field	Multi-scale feature extraction

Chapter 7: Training Techniques -- Optimizers, Regularization & Loss Functions

🎯 **Goal:** Master the advanced techniques that make neural network training stable, fast, and generalizable.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch07/` (7 files)

Chapter Learning Objectives

- [] Understand optimizer variants (SGD → Adam → AdamW)
 - [] Master learning rate scheduling
 - [] Understand regularization methods (L1/L2, dropout, early stopping)
 - [] Understand normalization (BatchNorm, LayerNorm)
 - [] Master weight initialization strategies
 - [] Understand loss functions for different tasks
 - [] Deep understanding of Softmax
-

7-1 Optimizer Landscape: From SGD to AdamW

Optimizer Evolution

Stochastic Gradient Descent (SGD)

$$v_{t+1} = \beta v_t + \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_{t+1}$$

Which Optimizer to Choose?

Scenario	Recommended Optimizer
Quick prototyping	Adam
CV (ImageNet-scale)	SGD + Momentum
NLP / Transformers	AdamW
Sparse data	AdaGrad
RL / non-stationary	RMSprop

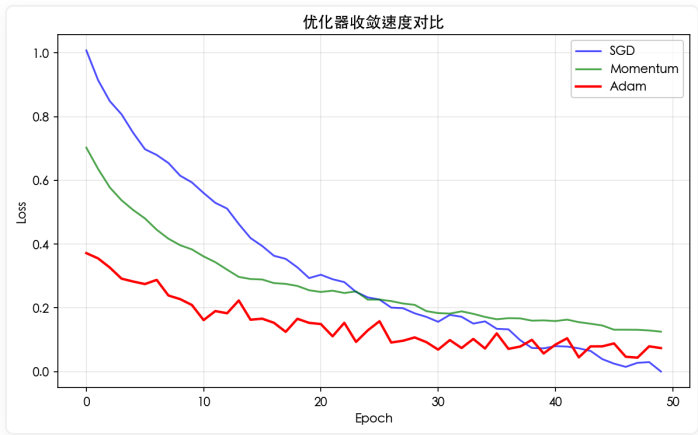


Figure 7-1: Optimizer comparison on the loss landscape.

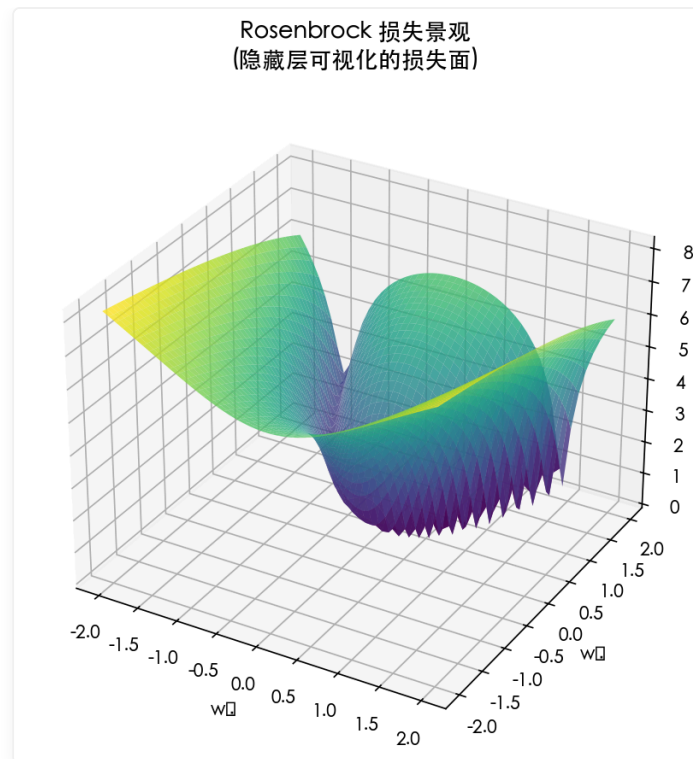


Figure 7-2: Loss landscape with different initialization points.

7-2 Learning Rate Scheduling

Common Schedulers

```
from torch.optim.lr_scheduler import (
    StepLR, MultiStepLR, CosineAnnealingLR,
    ReduceLROnPlateau
)

# Step decay: reduce by 0.1 every 30 epochs
scheduler = StepLR(optimizer, step_size=30, gamma=0.1)

# Cosine annealing: smooth decay
scheduler = CosineAnnealingLR(optimizer, T_max=100)

# Adaptive: reduce when loss plateaus
scheduler = ReduceLROnPlateau(optimizer, mode='min',
                               factor=0.5, patience=5)
```

Learning Rate Best Practices

Strategy	Use Case
Constant	Simple problems, short training
Step decay	Standard CV training
Cosine	Modern deep learning
One-cycle	Fast convergence
Warmup + decay	Transformers

7-3 Regularization and Generalization

L1 vs L2 Regularization

L2 Regularization (Weight Decay) adds a penalty proportional to the squared magnitude of weights:

$$L_{\text{new}} = L_{\text{original}} + \frac{\lambda}{2} \sum_w w^2$$

The gradient update becomes:

$$w \leftarrow w - \eta \nabla L - \eta \lambda w = w(1 - \eta \lambda) - \eta \nabla L$$

The $w(1 - \eta \lambda)$ term causes **weight decay** — weights are slightly shrunk at each update. This prevents any single weight from becoming too large, encouraging the model to use all weights moderately.

L1 Regularization (Lasso) uses the absolute value:

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum_w |w|$$

L1 regularization encourages **sparsity** — many weights become exactly zero. This is useful for feature selection.

Property	L2 (Ridge)	L1 (Lasso)
Penalty	$\frac{\lambda}{2} w^2$	$\lambda \cdot w $
Effect	Shrinks weights proportionally	Sets some weights to zero
Gradient	λw	$\lambda \cdot \text{sign}(w)$
Best for	Preventing overfitting	Feature selection

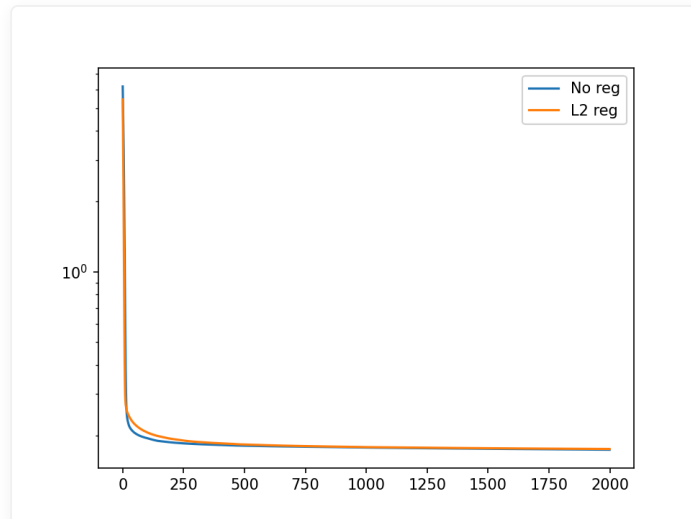


Figure 7-3: L2 regularization comparison.

Little Genius says: L2 regularization is like telling weights "don't be too confident!" L1 is like "if you're not useful, just become zero!" They're both ways to keep the model humble and prevent it from memorizing noise.

Dropout

Dropout randomly **drops** (sets to zero) a fraction of neurons during training:

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(p=0.5), # 50% of neurons randomly dropped
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Dropout(p=0.3),
    nn.Linear(128, 10)
```

)

Intuition: Dropout forces the network to learn **redundant representations** — no single neuron can be essential because it might be dropped at any time. This is like having multiple experts who each learn independently, then averaging their opinions.

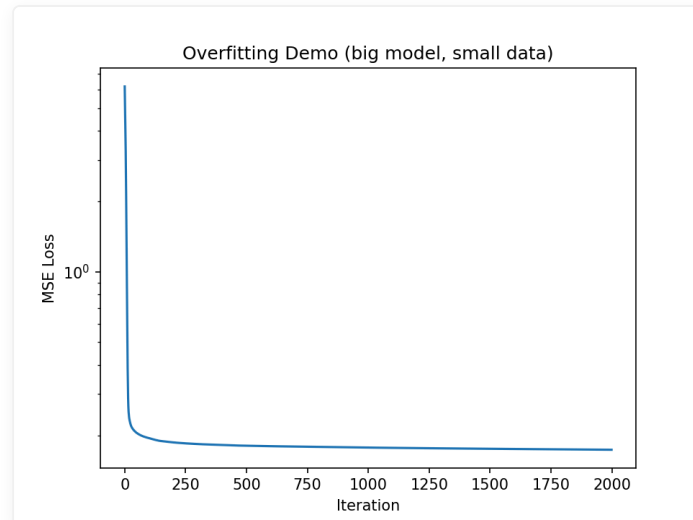


Figure 7-4: Overfitting — polynomial fitting comparison.

Early Stopping

Monitor validation loss and stop training when it stops improving:

```
best_val_loss = float('inf')
patience = 5
no_improve = 0

for epoch in range(100):
    train_loss = train_one_epoch()
    val_loss = evaluate()

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        no_improve = 0
        torch.save(model.state_dict(), 'best_model.pt')
    else:
        no_improve += 1
        if no_improve >= patience:
            print(f"Early stopping at epoch {epoch}")
            break
```

Data Augmentation

Create more training data through transformations:

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.RandomRotation(10),      # Slight rotation
    transforms.RandomHorizontalFlip(),  # Horizontal flip
    transforms.RandomAffine(0, shear=10), # Shear
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

7-4 Normalization Methods

Batch Normalization

Normalizes each batch to have mean 0, variance 1:

```
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.BatchNorm1d(256),
    nn.ReLU(),
    nn.Linear(256, 10),
)
```

Comparison

Method	Normalizes Across	Best For
BatchNorm	Batch dimension	CV (fixed-size batches)
LayerNorm	Feature dimension	NLP / Transformers
InstanceNorm	Single sample	Style transfer
GroupNorm	Groups of channels	Small batch sizes

7-5 Weight Initialization

Why Initialization Matters

Poor initialization can cause:

- **Gradient vanishing:** If weights are too small, activations shrink
- **Gradient explosion:** If weights are too large, activations explode
- **Symmetry:** All neurons learn the same features

Xavier (Glorot) Initialization

For **Sigmoid/Tanh** activations, Xavier initialization maintains variance across layers:

$$W \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \right)$$

```
def xavier_init(n_in, n_out):
    limit = np.sqrt(6 / (n_in + n_out))
    return np.random.uniform(-limit, limit, (n_in, n_out))
```

He (Kaiming) Initialization

For **ReLU** activations, He initialization accounts for the fact that ReLU zeros out half the neurons:

$$W \sim \mathcal{N} \left(0, \sqrt{\frac{2}{n_{\text{in}}}} \right)$$

```
def he_init(n_in, n_out):
    std = np.sqrt(2 / n_in)
    return np.random.randn(n_in, n_out) * std
```

Activation	Recommended Initialization	Distribution
Sigmoid / Tanh	Xavier (Glorot)	Uniform
ReLU / Leaky ReLU	He (Kaiming)	Normal
No activation	LeCun	Normal

Core insight: The key principle is **variance preservation** — the variance of activations should remain roughly constant across layers. Too much variance growth → explosion. Too much variance shrinkage → vanishing.

Bad initialization → vanishing/exploding gradients. Good initialization → fast convergence.

Common Initialization Strategies in Practice

```
# Default PyTorch (Kaiming Uniform for ReLU)
layer = nn.Linear(784, 256) # automatically initialized

# Manual initialization
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight) # for tanh/sigmoid
        # nn.init.kaiming_uniform_(m.weight) # for ReLU
        nn.init.zeros_(m.bias)

model.apply(init_weights)
```

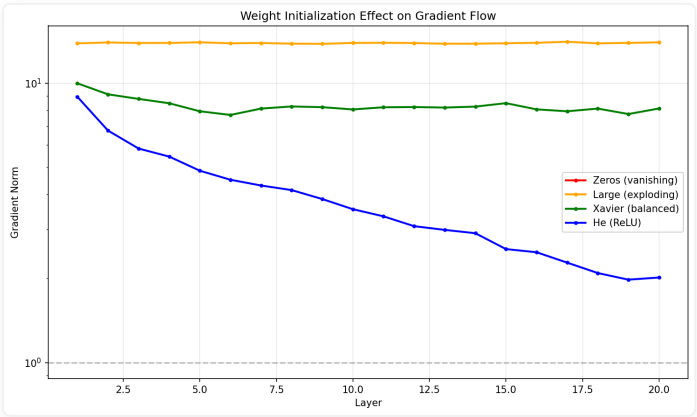


Figure 7-5: Weight initialization gradient flow comparison.

7-6 Loss Functions Overview

Classification

Loss	Formula	Use Case
Cross-Entropy	$-\sum t_k \log p_k$	Multi-class

Loss	Formula	Use Case
Binary CE	$-(t \log p + (1 - t) \log(1 - p))$	Binary
Focal Loss	Weighted CE	Imbalanced classes

Regression

Loss	Formula	Robustness
MSE	$\frac{1}{2}(y - t)^2$	Sensitive to outliers
MAE	$ y - t $	Robust to outliers
Huber	MSE for small error, MAE for large	Best of both

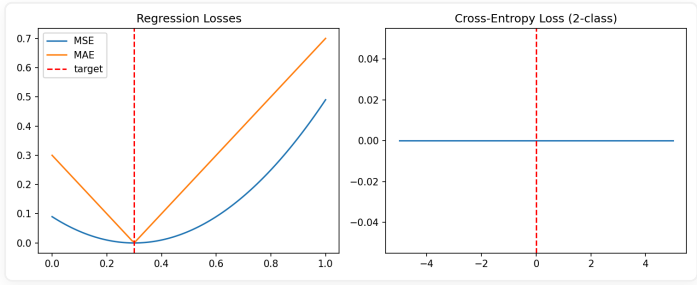


Figure 7-6: Different loss function surfaces.

7-7 Deep Understanding of Softmax ★

Beyond the Basic Definition

Softmax converts logits to probabilities, but its behavior depends on **temperature**:

$$p_k = \frac{e^{z_k/T}}{\sum_j e^{z_j/T}}$$

Temperature	Effect
$T \rightarrow 0$	Becomes argmax (hard assignment)

Temperature	Effect
$T = 1$	Standard Softmax
$T \rightarrow \infty$	Becomes uniform distribution

Temperature in Practice

```
def softmax_with_temperature(logits, temperature=1.0):
    """Softmax with temperature scaling"""
    scaled_logits = logits / temperature
    exp_logits = torch.exp(scaled_logits - torch.max(scaled_logits))
    return exp_logits / exp_logits.sum(dim=-1, keepdim=True)

logits = torch.tensor([2.0, 1.0, 0.1])
for T in [0.5, 1.0, 2.0, 5.0]:
    probs = softmax_with_temperature(logits, T)
    print(f"T={T:.1f}: {probs.numpy().round(3)}")
```

7–8 Regularization Deep Dive

Which Regularization When?

Method	Effect	When to Add
L2	Small weights	Always (mild)
Dropout	Prevents co-adaptation	Large networks, overfitting
Early stopping	Limits training time	Always
Data augmentation	More data implicitly	CV tasks
Label smoothing	Softens targets	Classification

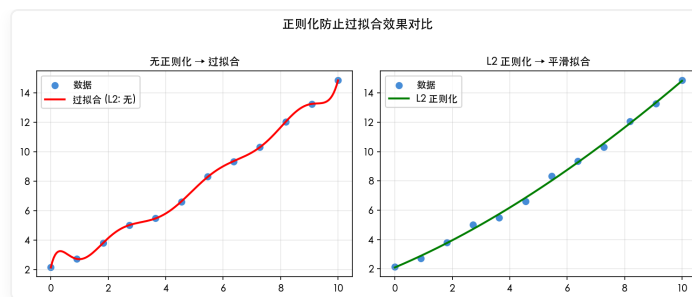


Figure 7-7: L1 vs L2 regularization comparison.

7-9 Hyperparameter Tuning

Key Hyperparameters

Tuning Strategies

1. **Grid search:** Systematic but expensive
2. **Random search:** Better for high-dimensional spaces
3. **Bayesian optimization:** Efficient but complex
4. **Learning rate finder:** Find optimal lr quickly

7-10 Gradient Clipping

Prevents exploding gradients by scaling down large gradients:

```
# Before stepping
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```



Supplementary Details

These subsections provide deeper mathematical background for the topics above.

Momentum

Momentum accelerates SGD by adding a velocity term:

$$v_{t+1} = \beta v_t + \eta \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

Where $\beta \approx 0.9$. This helps escape local minima and dampens oscillations.

NAG (Nesterov Accelerated Gradient)

NAG looks ahead: compute gradient at the approximate next position:

$$v_{t+1} = \beta v_t + \eta \nabla L(\theta_t - \beta v_t)$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

This "look-ahead" correction often leads to faster convergence than standard Momentum.

AdaGrad: Adaptive Learning Rate

AdaGrad adapts per-parameter learning rates based on historical gradients:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii}} + \epsilon} \cdot \nabla L(\theta_{t,i})$$

Where G accumulates squared gradients. Good for sparse features, but LR monotonically decreases.

RMSprop: Fixing AdaGrad's LR Decay

RMSprop uses a moving average of squared gradients instead of a cumulative sum:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

This prevents the learning rate from vanishing.

Adam (Adaptive Moment Estimation) ★

Adam combines Momentum and RMSprop:

- **Momentum**: first moment (mean) of gradients
- **RMSprop**: second moment (uncentered variance) of gradients
- **Bias correction**: compensates for initialization at zero

Adam is the default optimizer for most deep learning tasks.

Why Learning Rate Scheduling?

As training progresses, the optimal learning rate decreases. Early training benefits from larger steps to descend quickly, while later stages need smaller steps to fine-tune around the minimum.

L1/L2 Regularization

L2 (Weight Decay):

$$L_{\text{reg}} = L + \frac{\lambda}{2} \sum \theta^2$$

L1 (Lasso):

$$L_{\text{reg}} = L + \lambda \sum |\theta|$$

L2 shrinks weights proportionally to their size; L1 drives small weights to zero (feature selection).

Data Augmentation

Artificially expand training data with transformations. Common techniques: rotation, flipping, cropping, color jitter, Cutout. This reduces overfitting and improves generalization.

Regression Tasks

For regression, **Mean Squared Error (MSE)** is the most common loss:

$$L = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Also used: MAE (L_1 loss), Huber loss (robust to outliers).

Classification Tasks

For classification, **Cross-Entropy Loss** is standard:

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$

Where $y_{i,k}$ is the one-hot label and $\hat{y}_{i,k}$ is the predicted probability for class k .

Why Softmax?

Softmax converts raw logits to probabilities that sum to 1:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

This enables probabilistic interpretation of the model's output.

Mathematical Derivation

Softmax has a convenient derivative: $\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)$.

Combined with cross-entropy, the gradient simplifies to $\hat{y}_i - y_i$ — a beautiful cancellation!

Derivative of Softmax

The Jacobian of softmax is:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \begin{cases} \hat{y}_i(1 - \hat{y}_i) & i = j \\ -\hat{y}_i\hat{y}_j & i \neq j \end{cases}$$

Softmax + Cross-Entropy: A Beautiful Combination

The gradient of cross-entropy loss with softmax output simplifies elegantly:

$$\frac{\partial L}{\partial z_i} = \hat{y}_i - y_i$$

This is the same as the gradient for MSE + linear output in regression!

L1 vs L2 Regularization

Aspect	L2 (Weight Decay)	L1 (Lasso)
Effect	Shrinks weights	Sparse weights
Derivative	$2\lambda\theta$	$\lambda \cdot \text{sign}(\theta)$
Solution	Non-sparse	Sparse (feature selection)
Stability	More stable	Can be unstable

Elastic Net: L1 + L2 Combination

$$L_{\text{reg}} = L + \lambda_1 \sum |\theta| + \lambda_2 \sum \theta^2$$

Combines the benefits of both L1 (sparsity) and L2 (stability).

Regularization Strength Selection

Use **cross-validation** to select the regularization strength λ :

1. Try λ values on a log scale (10^{-4} to 10^0)
2. Train with each λ on training folds
3. Select λ with best validation performance

Key Hyperparameters Overview

Hyperparameter	Typical Range	Effect
Learning rate	10^{-5} to 10^{-1}	Most important
Batch size	16–512	Memory vs stability
Hidden units	64–4096	Model capacity
Dropout rate	0.1–0.5	Regularization

Grid Search vs Random Search

Grid Search evaluates all combinations — exponential in #parameters.

Random Search samples randomly — more efficient for high dimensions (Bergstra & Bengio, 2012).

Modern approach: **Bayesian Optimization** (e.g., Optuna, Hyperopt).

LR Range Test

Find the optimal learning rate by:

1. Start with a very small LR
2. Increase exponentially per mini-batch
3. Plot loss vs LR
4. Pick LR where loss decreases most steeply

```
# lr_finder pattern
for lr in np.logspace(-5, -1, 100):
    optimizer.param_groups[0]['lr'] = lr
    loss = train_step()
    lrs.append(lr)
    losses.append(loss.item())
```

Gradient Explosion Solutions

Gradient explosion occurs when gradients grow exponentially during backprop. Solutions:

- **Gradient clipping**: Cap gradient norms
- **Weight initialization**: Xavier/Kaiming init
- **Batch normalization**: Stabilizes activations
- **Residual connections**: Improve gradient flow

Two Gradient Clipping Methods

Clip by value:

$$g_i = \text{clip}(g_i, -c, c)$$

Clip by norm:

$$g = \frac{c}{\|g\|} \cdot g \text{ if } \|g\| > c$$

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=0.5)
```

Norm clipping is preferred as it preserves gradient direction.

Chapter Code List

File	Content	Key Concept
<code>ch07/NN07_regularization.py</code>	L1/L2 regularization implementation & comparison	Regularization principles
<code>ch07/NN07_dropout.py</code>	Dropout implementation & effect analysis	Dropout mechanism
<code>ch07/NN07_batchnorm.py</code>	Batch Normalization manual implementation	Normalization techniques
<code>ch07/NN07_loss_functions.py</code>	Multiple loss function implementations & comparison	Loss function selection
<code>ch07/NN07_early_stopping.py</code>	Early stopping implementation & overfitting prevention	Early stopping strategy
<code>ch07/NN07_weight_init_viz.py</code>	Weight initialization comparison visualization	Initialization methods

Chapter Summary

- **Optimizers:** how to follow the gradient (SGD → Adam → AdamW)
- **Scheduling:** when to change step size (Step → Cosine → Plateau)
- **Regularization:** L1/L2, Dropout, BatchNorm
- **Initialization:** Xavier (tanh) vs He (ReLU)
- **Loss:** MSE (regression) → CrossEntropy (classification)

Exercises

Exercise 1: Optimizer Comparison

Train MNIST with SGD, SGD+Momentum, Adam. Compare epochs to 95% accuracy.

Exercise 2: Regularization

Train a network with and without dropout. Compare train vs. test accuracy.

Exercise 3: BatchNorm

Add BatchNorm to a deep network (5+ layers). Does it help convergence?

Exercise 4: Learning Rate Finder

Implement a learning rate finder: start $lr=1e-5$, increase exponentially each batch, plot loss vs. lr .

← [Chapter 6](#) | [Table of Contents](#) | [Chapter 8](#) →

Chapter 8: Modern Architectures -- From ResNet to Transformer

🎯 **Goal:** Understand the key architectural innovations that enable modern deep learning — skip connections, attention mechanisms, and the Transformer.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch08/` (5 files)

Chapter Learning Objectives

- [] Understand ResNet and why skip connections solve vanishing gradients
- [] Understand RNN basics and the vanishing gradient problem in sequences
- [] Understand LSTM/GRU gating mechanisms and how they mitigate gradient vanishing
- [] Master the attention mechanism: Query, Key, Value
- [] Understand the complete Transformer architecture
- [] Be able to implement a minimal Transformer block

8-1 Residual Networks (ResNet) ★

Authors: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research)

Paper: *Deep Residual Learning for Image Recognition*, CVPR 2016 **Best Paper Award**

💡 **Key Insight:** Skip connections allow gradients to flow directly to shallow layers, enabling 100+ layer networks.

8-1-1 The Degradation Problem

Experimental Observation

A 56-layer plain network has **higher training error** than a 20-layer network — and this is not overfitting, because the training error itself is worse. This is the **degradation problem**: deeper networks become harder to optimize.

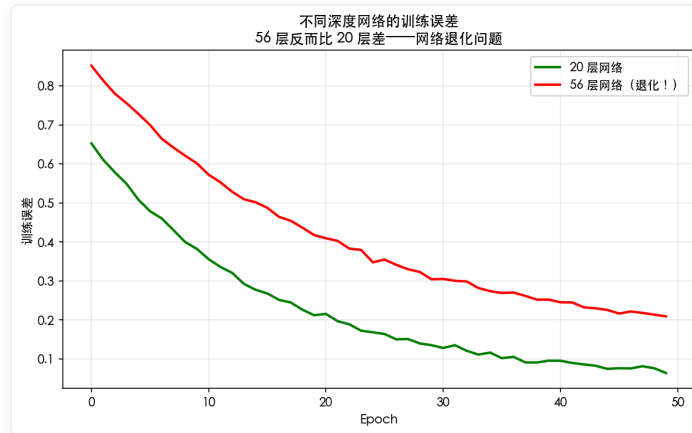


Figure 8-1: Network degradation — deeper networks become harder to optimize.

💡 **Key Insight:** Degradation is not overfitting — training error itself is worse. The root cause is **optimization difficulty**, not overfitting.

Gradient Vanishing: Mathematical Analysis

For an L -layer network, the gradient at layer 1 must pass through $L - 1$ activation function derivatives:

$$\frac{\partial L}{\partial W^{(1)}} = \underbrace{f'(u^{(L)}) \cdots f'(u^{(1)})}_{L \text{ derivatives}} \times \dots$$

The chain multiplication effect:

- **Sigmoid:** derivative range $(0, 0.25] \rightarrow L = 10$ gives $0.25^{10} \approx 10^{-6}$
- **ReLU:** derivative range $0, 1 \rightarrow$ alleviates but doesn't solve

Degradation vs Gradient Vanishing

Problem	Symptom	Root Cause
Gradient Vanishing	Shallow layers get tiny gradients $\rightarrow$ can't learn	Activation derivative product < 1
Degradation	Deeper is worse than shallower	Identity mapping is hard to learn (layers struggle to approximate $F(x) \approx x$)

Analogy: Imagine a 50-layer network. Ideally, if the first 20 layers already learned good features, the remaining 30 layers should "do nothing" — maintain identity mapping. But it turns out that learning $F(x) \approx x$ through multiple nonlinear layers is surprisingly difficult! This is the mathematical essence of degradation — **stacked nonlinearities struggle to approximate identity mappings**.

8-1-2 The Mathematics of Skip Connections ★

Intuition: Skip connections create a "VIP channel" for information flow. Previously, information had to pass through layer after layer ($F(x)$), easily getting lost. Now with $y = F(x) + x$, gradients can reach shallow layers directly — like an employee reporting directly to the CEO without going through layers of management! This is why ResNet can train 1000+ layer networks.

Core Idea

Give the gradient a "highway" to shallow layers:

$$y = F(x, W_i) + x$$

- $F(x)$ is the **residual mapping** to be learned (typically 2–3 conv layers)
- x is the **identity mapping** passed directly via shortcut

The Beauty: If identity is optimal, the network only needs to drive $F(x) \rightarrow 0$, which is much easier than fitting an identity mapping through multiple nonlinear layers.

Backpropagation Mathematics

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x} = \frac{\partial L}{\partial y} \cdot \left(1 + \frac{\partial F}{\partial x}\right)$$

💡 **Key Insight:** The gradient contains an identity term **1**! Even when $\frac{\partial F}{\partial x} \rightarrow 0$, gradients can still propagate directly to shallow layers through the **1** term. This is the "gradient highway."

Gradient Flow: Plain vs ResNet

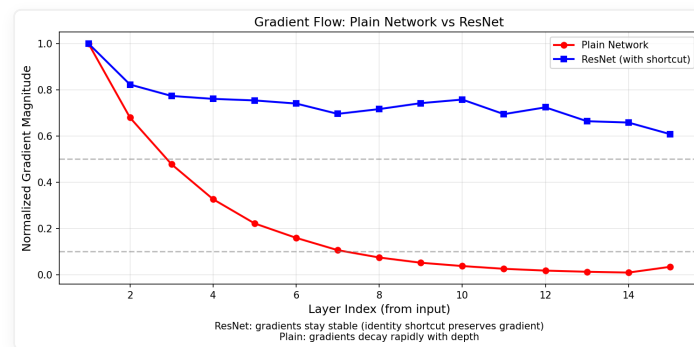


Figure 8-2: Gradient flow — Plain network gradients decay $\sim 30\times$ (layer 1 to layer 15), while ResNet decays only $\sim 1.7\times$. This is the power of the identity term **1**!

Run `code/ch08/NN08_resnet_gradient_flow.py` to reproduce:

```
python3 code/ch08/NN08_resnet_gradient_flow.py
```

PyTorch Implementation

```
class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3,
                                stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3,
                                padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        # Shortcut: 1x1 conv when dimensions don't match
        self.shortcut = nn.Sequential()
```

```

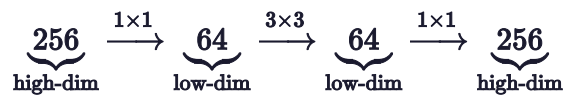
if stride != 1 or in_channels != out_channels:
    self.shortcut = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 1,
                    stride=stride, bias=False),
        nn.BatchNorm2d(out_channels)
    )

def forward(self, x):
    out = torch.relu(self.bn1(self.conv1(x)))
    out = self.bn2(self.conv2(out))
    out += self.shortcut(x) # Skip connection! ★
    out = torch.relu(out)
    return out

```

8-1-3 Bottleneck Design NEW

Bottleneck is the **engineering key** that enables ResNet-50/101/152. It uses "reduction → convolution → expansion" to drastically reduce parameters:



Design Philosophy

Step	Operation	Dimension Change	Purpose
① Reduce	1×1 conv	256 → 64	Compress high-dim features, reduce computation
② Convolve	3×3 conv	64 → 64	Extract spatial features in low-dim space
③ Expand	1×1 conv	64 → 256	Restore features to high dimension

Analogy: Bottleneck is like a package sorting center — large packages (256-dim) are compressed into a smaller space (64-dim) for processing, then restored to original size. The task is accomplished at a fraction of the cost!

Parameter Comparison

When outputting 256 channels:

Block Type	Structure	Parameters	Relative Size
Basic Block	$3 \times 3 \rightarrow 3 \times 3$	$\approx 1.18\text{M}$	$1\times$
Bottleneck	$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$	$\approx 70\text{K}$	$1/17$

Run `code/ch08/NN08_resnet_bottleneck.py` for the full comparison:

```
python3 code/ch08/NN08_resnet_bottleneck.py
```

```
class Bottleneck(nn.Module):
    expansion = 4 # output channels = planes * 4

    def __init__(self, in_planes, planes, stride=1):
        super().__init__()
        # ① 1x1 reduction
        self.conv1 = nn.Conv2d(in_planes, planes, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        # ② 3x3 convolution (in low-dim space)
        self.conv2 = nn.Conv2d(planes, planes, 3, stride=stride, padding=1,
bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        # ③ 1x1 expansion
        self.conv3 = nn.Conv2d(planes, planes * self.expansion, 1, bias=False)
        self.bn3 = nn.BatchNorm2d(planes * self.expansion)

        self.shortcut = nn.Sequential()
        if stride != 1 or in_planes != planes * self.expansion:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_planes, planes * self.expansion, 1,
                    stride=stride, bias=False),
                nn.BatchNorm2d(planes * self.expansion)
            )

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x))) # reduction
        out = F.relu(self.bn2(self.conv2(out))) # convolution
        out = self.bn3(self.conv3(out)) # expansion
        out += self.shortcut(x) # skip connection
        return F.relu(out)
```

When to Use Which Block?

ResNet Version	Block Type	Depth	When to Use
ResNet-18/34	Basic Block	Shallow	Small datasets, quick experiments
ResNet-50/101/152	Bottleneck	Deep	ImageNet, large-scale data

💡 **Key Insight:** The 1×1 convolution acts as an "information funnel" — it compresses feature dimensions before the expensive 3×3 convolution. Without this design, ResNet-152's parameter count would be prohibitively large.

8-1-4 Pre-activation ResNet (v2) NEW

ResNet v2 (He et al., *Identity Mappings in Deep Residual Networks*, 2016) improved the activation order:

Version	Structure	Gradient Path
v1 (Post-activation)	conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ conv $\rightarrow$ BN $\rightarrow$ +shortcut $\rightarrow$ ReLU	Must pass through ReLU
v2 (Pre-activation)	BN $\rightarrow$ ReLU $\rightarrow$ conv $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ conv $\rightarrow$ +shortcut	Direct identity path

Why Pre-activation is Better

v2 moves BN + ReLU before the convolution. Benefits:

1. **Cleaner gradient path:** $y = x + F(\text{BN}(\text{ReLU}(x)))$, gradients flow directly through x
2. **More stable training:** BN stays within the residual branch, doesn't pollute the identity path
3. **Easier ultra-deep training:** 1000+ layer ResNet v2 trains more stably

```
class PreActBlock(nn.Module):
    """Pre-activation: BN  $\rightarrow$  ReLU  $\rightarrow$  conv  $\rightarrow$  BN  $\rightarrow$  ReLU  $\rightarrow$  conv"""
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.bn1 = nn.BatchNorm2d(in_channels)
        self.conv1 = nn.Conv2d(in_channels, out_channels, 3,
                               stride=stride, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, 3,
                               padding=1, bias=False)
```

```

self.shortcut = nn.Sequential()
if stride != 1 or in_channels != out_channels:
    self.shortcut = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, 1,
                   stride=stride, bias=False),
    )

def forward(self, x):
    # Pre-activation: BN+ReLU first, then conv ★
    out = self.conv1(F.relu(self.bn1(x)))
    out = self.conv2(F.relu(self.bn2(out)))
    out += self.shortcut(x) # No final ReLU needed
    return out

```

💡 Key Insight: v1's path is $y = \text{ReLU}(F(x) + x)$ — the ReLU on the main path blocks negative gradient propagation. v2 moves ReLU into the residual branch: $y = x + F(\text{BN}(\text{ReLU}(x)))$, leaving the identity path completely unobstructed. This small change has a huge impact on ultra-deep networks (100+ layers)!

Run `code/ch08/NN08_resnet_block.py` to see both implementations.

8-1-5 Experiments: Plain vs ResNet NEW

8-1-6 Ablation Studies and Design Choices

Key design decisions in ResNet:

- Pre-activation (v2): BN → ReLU → Conv improves gradient flow
- Bottleneck: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ reduces parameters
- Stride placement: Using stride in 3×3 vs 1×1 affects information flow
- Projection shortcuts: 1×1 conv for dimension matching

8-1-7 Post-ResNet Architecture Evolution

- DenseNet: Dense connections (each layer connects to all subsequent)
- SENet: Squeeze-and-Excitation blocks (channel attention)
- EfficientNet: Neural Architecture Search (NAS) optimized
- ConvNeXt: Modernized ResNet with Transformer-inspired design

8–1–8 ResNet Training Experiments

Gradient Flow: Plain vs ResNet

Compare gradient magnitudes across 15 layers of Plain vs ResNet:

Layer	Plain Gradient	ResNet Gradient	Ratio
1	1.39e+02	1.40e+01	0.10
5	3.29e+01	8.38e+00	0.25
10	4.94e+00	8.23e+00	1.67
15	4.60e+00	7.78e+00	1.69

Plain — Layer 1 to 15: **30x decay** → gradients vanish.

ResNet — Layer 1 to 15: **only 1.7x decay** → gradients flow!

Conclusion: Skip connections reduce gradient decay from 30x to just 1.7x — the power of the identity term **1!**

Training Comparison

Train an 8–layer PlainNet vs 8–layer ResNet on synthetic data:

Run `code/ch08/NN08_resnet_training.py` to reproduce:

```
python3 code/ch08/NN08_resnet_training.py
```

Network	Final Loss
PlainNet	1.0348
ResNet	0.1685

ResNet achieves **6.14x lower loss** than PlainNet!

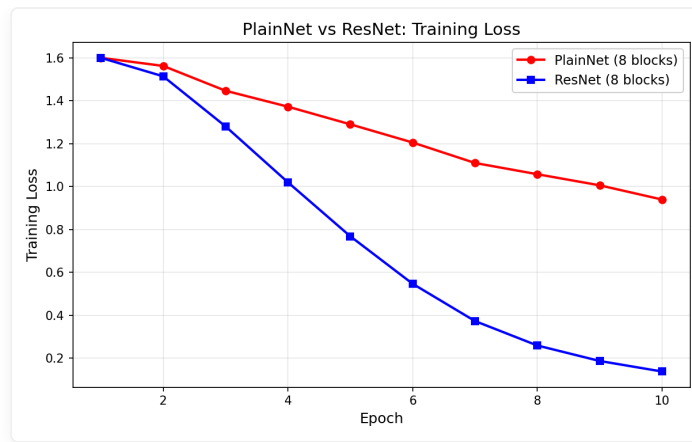


Figure 8-3: PlainNet vs ResNet training loss — ResNet achieves ~6x lower loss through skip connections.

Summary

Metric	PlainNet (8-layer)	ResNet (8-layer)	Improvement
Final Training Loss	≈ 1.03	≈ 0.17	6.1×
Gradient Decay	≈ 30×	≈ 1.7×	17.6×
Convergence Speed	Slow	Fast	✓

Takeaway: These two experiments prove the effect of skip connections from both the "gradient perspective" and the "training perspective." The gradient flow experiment tells you *why* — the identity term **1** lets gradients reach shallow layers directly. The training experiment tells you *the result* — ResNet actually learns better!

8-2 Recurrent Neural Networks & Sequence Modeling

The RNN Cell

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

Vanishing Gradient in RNNs

RNNs also suffer from vanishing gradients — gradients must flow through many time steps.

8–3 From RNN to LSTM/GRU: Gating Mechanisms Explained

The gradient vanishing problem from 8–2 has a solution: **gated architectures**. LSTM (1997) and GRU (2014) introduced gating mechanisms that allow gradients to flow across time unscathed — much like the skip connections in ResNet, but through time.

8–3–1 From Gradient Problems to Gating

As analyzed in 8–2, the RNN's vanishing gradient arises from the double decay in BPTT: the $\tanh'$ scaling ($(0, 1]$) multiplied by the eigenvalues of $\mathbf{W}_{hh}$ (exponential decay when $|\lambda_{\max}| < 1$). When eigenvalues are > 1 , gradients explode; < 1 , they vanish — making plain RNNs incapable of learning long-range dependencies. The solution is the **gating mechanism** introduced below.

Analogy: Imagine shouting into a canyon — each second the echo travels, it fades. After 10 seconds, it's barely audible — that's gradient vanishing! LSTM is like giving the echo a "relay amplifier" at every point — the signal stays strong across time!

8–3–2 LSTM — Long Short-Term Memory

LSTM's core innovation is the **gating mechanism** — three gates (forget, input, output) and a memory cell (Cell State):

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate})$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate})$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C) \quad (\text{candidate memory})$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad (\text{cell state update})$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate})$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad (\text{hidden state})$$

Gate	Role	Range	Analogy
Forget gate $\mathbf{f}_t$	How much old memory to discard	[0, 1]	Selective forgetting
Input gate $\mathbf{i}_t$	How much new information to store	[0, 1]	Selective remembering
Output gate $\mathbf{o}_t$	How much to reveal	[0, 1]	Selective expression

Why LSTM Mitigates Vanishing Gradients

The key is that the cell state update is **additive** rather than multiplicative:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$$

Taking the derivative with respect to $\mathbf{C}_{t-1}$:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \text{diag}(\mathbf{f}_t) + \dots$$

There is an **identity pathway**! When the forget gate $\mathbf{f}_t \approx \mathbf{1}$ (the model decides to "remember"), $\partial \mathbf{C}_t / \partial \mathbf{C}_{t-1} \approx \mathbf{I}$. Gradients can propagate across arbitrarily many time steps without decay:

$$\frac{\partial \mathbf{C}_T}{\partial \mathbf{C}_0} \approx \prod_{t=1}^T \text{diag}(\mathbf{f}_t) \approx \mathbf{I} \quad (\text{when } \mathbf{f}_t \rightarrow \mathbf{1})$$

Contrast this with RNN's $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} = \tanh' \cdot \mathbf{W}_{hh}^T$ (always a sub-unity multiplication). LSTM's gradient pathway is **additive + identity mapping** — this is precisely the intellectual

precursor to ResNet's skip connections!

Analogy: LSTM's memory cell is like a "highway" — if the forget gate is open ($\mathbf{f}_t \approx 1$), information can travel unimpeded from $t = 0$ to $t = 100$, and gradients flow just as freely. This is the mathematical secret behind LSTM's ability to remember "long, long ago" information.

8-3-3 GRU — LSTM's Simplified Version

GRU (Gated Recurrent Unit, Cho et al., 2014) simplifies LSTM's three gates into two — the **update gate** and **reset gate**:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{update gate})$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{reset gate})$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{candidate hidden state})$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (\text{hidden state})$$

Architecture Comparison

Feature	RNN	LSTM	GRU
Number of gates	0	3 (forget + input + output)	2 (update + reset)
Memory unit	None	Cell State	None (hidden state only)
Parameter count	Lowest	Highest	Medium
Gradient vanishing	Severe	Greatly mitigated	Mitigated
Training speed	Fastest	Slowest	Medium

💡 **Key Insight:** GRU is LSTM's "lean version" — comparable performance with fewer

parameters. In practice, the choice depends on task and data size. LSTM's larger parameter count suits data-rich settings; GRU's lighter design is better for smaller datasets or rapid iteration.

8-4 Attention Mechanism ★

The Core Idea

Attention allows the model to **focus on relevant parts** of the input.

Query, Key, Value

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Component	Analogy	Role
Query	What you're looking for	Current focus
Key	What's available	What each position offers
Value	The actual content	What to extract if matched

Scaled Dot-Product Attention

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def scaled_dot_product_attention(Q, K, V):
    """Compute attention scores and weighted values"""
    d_k = Q.size(-1)
    scores = torch.matmul(Q, K.transpose(-2, -1)) / (d_k ** 0.5)
    attention_weights = F.softmax(scores, dim=-1)
    output = torch.matmul(attention_weights, V)
    return output, attention_weights

# Example
batch, seq_len, d_k = 2, 4, 8
Q = torch.randn(batch, seq_len, d_k)
```

```
K = torch.randn(batch, seq_len, d_k)
V = torch.randn(batch, seq_len, d_k)

output, weights = scaled_dot_product_attention(Q, K, V)
print(f"Output shape: {output.shape}")      # (2, 4, 8)
print(f"Weights shape: {weights.shape}")    # (2, 4, 4)
```

8-5 Complete Transformer Architecture ★

8-5-1 Multi-Head Attention

Instead of one attention, use **multiple heads** in parallel:

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.num_heads = num_heads
        self.d_head = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def forward(self, Q, K, V):
        batch_size = Q.size(0)

        # Linear projections + split into heads
        Q = self.W_q(Q).view(batch_size, -1, self.num_heads,
self.d_head).transpose(1, 2)
        K = self.W_k(K).view(batch_size, -1, self.num_heads,
self.d_head).transpose(1, 2)
        V = self.W_v(V).view(batch_size, -1, self.num_heads,
self.d_head).transpose(1, 2)

        # Attention
        scores = torch.matmul(Q, K.transpose(-2, -1)) / (self.d_head ** 0.5)
        attn = F.softmax(scores, dim=-1)
        context = torch.matmul(attn, V)

        # Concatenate heads
        context = context.transpose(1, 2).contiguous().view(
            batch_size, -1, self.num_heads * self.d_head)
        return self.W_o(context)
```

8-5-2 Encoder Block

```
class TransformerEncoderBlock(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.attention = MultiHeadAttention(d_model, num_heads)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)

        self.ffn = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.ReLU(),
            nn.Linear(d_ff, d_model),
        )
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # Multi-head attention + residual + layernorm
        attn_out = self.attention(x, x, x)
        x = self.norm1(x + self.dropout(attn_out))

        # FFN + residual + layernorm
        ffn_out = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_out))
        return x
```

8-5-3 Transformer Overall Architecture

The attention matrix would show which English words align with which French words.

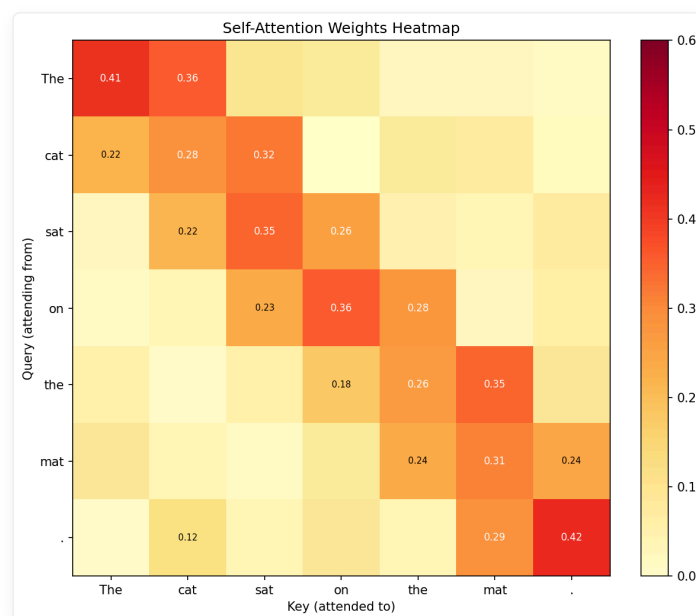


Figure 8–4: Attention weight visualization — darker regions = higher attention. In translation, each output word attends to different positions in the input.

8–6 BERT vs. GPT: Pre-training Paradigms

Aspect	BERT	GPT
Architecture	Encoder-only	Decoder-only
Training	Masked LM (bidirectional)	Autoregressive (left-to-right)
Best for	Understanding (classification, QA)	Generation (text, code)
Examples	BERT, RoBERTa, ALBERT	GPT-2, GPT-3, GPT-4

8–7 Modern Architecture Design Patterns

Pattern	Example	Benefit
Skip connections	ResNet	Gradient flow
LayerNorm	Transformer	Stable training
Pre-norm	GPT	Better for deep models
GELU activation	GPT/BERT	Smooth ReLU variant

8–8 Vision Transformer (ViT)

ViT applies the Transformer directly to image patches:

```
class PatchEmbedding(nn.Module):
    def __init__(self, img_size=224, patch_size=16, in_ch=3, embed_dim=768):
        super().__init__()
        self.proj = nn.Conv2d(in_ch, embed_dim,
                               kernel_size=patch_size, stride=patch_size)

    def forward(self, x):
        x = self.proj(x)  # (B, embed_dim, H/p, W/p)
```

```
x = x.flatten(2) # (B, embed_dim, num_patches)
return x.transpose(1, 2) # (B, num_patches, embed_dim)
```

8–9 Modern Architecture Design Principles

1. **Scale:** More data + more parameters + more compute
2. **Normalization:** Every block should normalize
3. **Residual connections:** Essential for deep models
4. **Attention is universal:** Works for text, images, audio, video

Exercises

Exercise 1: ResNet Skip Connection

Build a ResidualBlock, verify gradient flow by comparing with a plain block.

Exercise 2: Attention Visualization

Implement attention and visualize the weight matrix. Which tokens attend to which?

Exercise 3: LSTM Implementation

Implement a single LSTM cell from scratch. Compute the gradient through the cell state $\mathbf{C}_t$ and verify the identity pathway when $\mathbf{f}_t \rightarrow \mathbf{1}$.

Exercise 4: Minimal Transformer

Build a 2-layer Transformer encoder and train it on a simple sequence classification task.

Exercise 5: BERT vs GPT Objectives

Implement both masked LM (BERT) and autoregressive LM (GPT) objectives. Compare their training behavior.



Chapter Code List

File	Content	Key Concept
<code>ch08/NN08_resnet_block.py</code>	Residual Block + Pre-activation implementation	Skip connections + ResNet v2

File	Content	Key Concept
ch08/NN08_resnet_bottleneck.py	Bottleneck parameter comparison	Bottleneck design philosophy
ch08/NN08_resnet_gradient_flow.py	Plain vs ResNet gradient flow comparison	Gradient propagation visualization
ch08/NN08_resnet_training.py	Plain vs ResNet training comparison	Degradation phenomenon reproduction
ch08/NN08_resnet_series.py	ResNet-18/34/50/101/152 full definitions	ResNet series configurations
ch08/NN08_attention.py	Attention mechanism from scratch	Attention core
ch08/NN08_attention_viz.py	Attention weight matrix visualization	Attention visualization
ch08/NN08_positional_encoding.py	Sinusoidal positional encoding	Positional encoding
ch08/NN08_transformer_encoder.py	Transformer Encoder full implementation	Transformer core

Chapter Summary

Architecture Evolution

Era	Architecture	Key Innovation
1998	LeNet	First CNN
2012	AlexNet	ReLU + GPU + Dropout
2015	ResNet	Skip connections
2017	LSTM/GRU	Gating mechanisms for long-range memory
2017	Transformer	Self-attention

Era	Architecture	Key Innovation
2020	ViT / GPT-3	Transformers for vision & language

One-line summary: The evolution of modern deep learning architectures = solving vanishing gradients (ResNet) + mitigating long-range forgetting (LSTM/GRU) + capturing global dependencies (Attention) + parallelization (Transformer).

← [Chapter 7](#) | [Table of Contents](#) | [Chapter 9](#) →

Chapter 9: Large Language Models -- Training, Sampling & Inference

🎯 **Goal:** Understand how large language models (LLMs) are trained, how they generate text, and the techniques that make them practical.

© xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Code: `../code/ch09/` (4 files)

Chapter Learning Objectives

- [] Understand the language modeling objective (next-token prediction)
- [] Master autoregressive generation
- [] Understand sampling strategies (greedy, top-k, top-p)
- [] Understand KV Cache for inference acceleration
- [] Master LoRA for efficient fine-tuning
- [] Understand quantization fundamentals

9-1 From Language Model to Large Language Model

9-1-1 Basic Definition of Language Models

A language model assigns a probability to a sequence of tokens:

$$P(w_1, w_2, \dots, w_n) = \prod_{t=1}^n P(w_t | w_{<t})$$

Scaling Hypothesis

As models get larger (more parameters, more data, more compute), capabilities **emerge**:

Key LLMs

Model	Parameters	Year	Innovation
GPT-2	1.5B	2019	Zero-shot learning
GPT-3	175B	2020	In-context learning
LLaMA	65B	2023	Open-source, efficient
GPT-4	~1.8T	2023	Multi-modal, reasoning

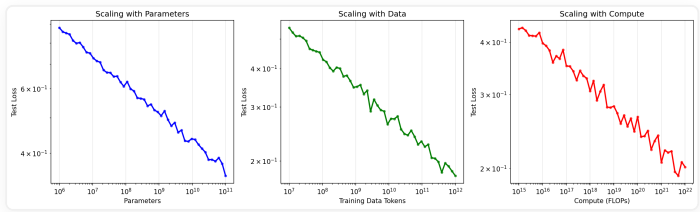


Figure 9-1: Scaling Laws — performance scales predictably with model size, data, and compute.

9-2 Autoregressive Generation & Training

Training Objective

Maximize the likelihood of the next token given previous tokens:

$$L = - \sum_{t=1}^T \log P(w_t | w_{<t}; \theta)$$

Generation Loop

```
def generate_autoregressive(model, prompt, max_tokens=100):
    """Autoregressive text generation"""
    tokens = tokenize(prompt)
```

```

for _ in range(max_tokens):
    # Forward pass
    logits = model(tokens)

    # Get prediction for the next token
    next_token_logits = logits[:, -1, :]

    # Sample
    next_token = sample_from_logits(next_token_logits)

    # Append to sequence
    tokens = torch.cat([tokens, next_token], dim=1)

    # Stop if EOS token
    if next_token.item() == EOS_TOKEN_ID:
        break

return detokenize(tokens)

```

9-3 Sampling Strategies ★

9-3-1 Greedy Decoding

Always pick the most likely token:

$$w_t = \arg \max P(w|w_{<t})$$

✅ Simple, deterministic ❌ Repetitive, boring

9-3-2 Temperature Control

Controls randomness:

$$P(w) \propto \exp(\text{logit}_w/T)$$

Temperature	Effect
$T \rightarrow 0$	Greedy (deterministic)
$T = 1$	Standard softmax

Temperature	Effect
$T > 1$	More random, creative

Top-k Sampling

Only sample from the k most likely tokens:

```
def top_k_sampling(logits, k=50):
    """Sample from top-k tokens only"""
    values, indices = torch.topk(logits, k)
    probs = F.softmax(values / temperature, dim=-1)
    chosen = torch.multinomial(probs, 1)
    return indices[0, chosen]
```

Top-p (Nucleus) Sampling

Sample from the smallest set of tokens whose cumulative probability exceeds p :

```
def top_p_sampling(logits, p=0.9):
    """Nucleus sampling"""
    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    sorted_probs = F.softmax(sorted_logits / temperature, dim=-1)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

    # Remove tokens with cumulative probability above p
    sorted_indices_to_remove = cumulative_probs > p
    sorted_indices_to_remove[..., 1:] = sorted_indices_to_remove[..., :-1].clone()
    sorted_indices_to_remove[..., 0] = 0

    indices_to_remove = sorted_indices_to_remove.scatter(
        1, sorted_indices, sorted_indices_to_remove)
    logits[indices_to_remove] = float('-inf')
    probs = F.softmax(logits / temperature, dim=-1)
    return torch.multinomial(probs, 1)
```

9-3-3 Sampling Strategy Comparison

Strategy	Diversity	Quality	Use Case
Greedy	Low	High (first try)	Factual answers
Temperature	Medium	Medium	Creative writing

Strategy	Diversity	Quality	Use Case
Top-k (k=50)	Medium	High	General purpose
Top-p (p=0.9)	High	High	Balanced
Top-k + Top-p	High	Highest	Production

9-4 KV Cache: Inference Acceleration ★

The Problem

At each generation step, the model re-computes attention for **all** previous tokens — $O(n^2)$ complexity.

The Solution: KV Cache

Cache the Key and Value matrices from previous steps:

```
class CachedAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.num_heads = num_heads
        self.d_head = d_model // num_heads
        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)

    def forward(self, x, kv_cache=None):
        batch, seq_len = x.shape[:2]

        Q = self.W_q(x).view(batch, seq_len, self.num_heads, self.d_head)
        K = self.W_k(x).view(batch, seq_len, self.num_heads, self.d_head)
        V = self.W_v(x).view(batch, seq_len, self.num_heads, self.d_head)

        # Concatenate with cache
        if kv_cache is not None:
            K_cache, V_cache = kv_cache
            K = torch.cat([K_cache, K], dim=1)
            V = torch.cat([V_cache, V], dim=1)

        # Update cache
        new_kv_cache = (K, V)
```

```
# Attention (only need last query for generation)
Q_last = Q[:, -1:] if kv_cache is not None else Q
scores = torch.matmul(Q_last, K.transpose(-2, -1)) / (self.d_head ** 0.5)
attn = F.softmax(scores, dim=-1)
out = torch.matmul(attn, V)
return out, new_kv_cache
```

Speedup

Without KV Cache	With KV Cache
$O(n^2)$ per step	$O(n)$ per step
~3× slower for 100 tokens	Baseline
~10× slower for 1000 tokens	Baseline

9-5 Efficient Fine-Tuning: LoRA

The Problem

Full fine-tuning of a 175B model is **prohibitively expensive** (>1M per run).

LoRA: Low-Rank Adaptation

LoRA freezes the original weights and adds **small rank decomposition matrices**:

$$W' = W + BA$$

Where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and $r \ll \min(d, k)$.

```
class LoRALayer(nn.Module):
    """Low-Rank Adaptation layer"""
    def __init__(self, original_layer, rank=8, alpha=16):
        super().__init__()
        self.original = original_layer
        self.original.requires_grad_(False) # freeze

        d, k = original_layer.weight.shape
        self.A = nn.Parameter(torch.randn(rank, k) / rank)
```

```

self.B = nn.Parameter(torch.zeros(d, rank))
self.scale = alpha / rank

def forward(self, x):
    # Original (frozen) + LoRA (trainable)
    return self.original(x) + (x @ self.A.T @ self.B.T) * self.scale

```

Why LoRA Works

- Pre-trained models have **low intrinsic rank**
- A small number of parameters can capture task-specific adaptations
- Can swap LoRA modules for different tasks without loading the base model

Memory Comparison

Method	Trainable Params	Memory
Full fine-tune	175B	> 350GB
LoRA (r=8)	~0.3B	< 1GB

9-6 Quantization Basics

Why Quantize?

Reducing precision from FP32 → INT8:

- 4× **smaller memory**
- 2–4× **faster inference**
- Minimal accuracy loss

Quantization Types

Type	Description	Accuracy Loss
Post-training (PTQ)	Quantize after training	Small
Quantization-aware (QAT)	Train with simulated quant	Minimal
GPTQ	Weight-only, one-shot	Very small

Type	Description	Accuracy Loss
GGML/GGUF	CPU-optimized	Small

INT8 Quantization

```
def quantize_int8(tensor):
    """Quantize FP32 tensor to INT8"""
    scale = tensor.abs().max() / 127.0
    quantized = (tensor / scale).round().char()
    return quantized, scale

def dequantize(quantized, scale):
    """Restore from INT8 to FP32"""
    return quantized.float() * scale
```

9–7 RLHF: Reinforcement Learning from Human Feedback

9–7–1 RLHF (Reinforcement Learning from Human Feedback)

1. SFT (Supervised Fine-Tuning): Fine-tune on human demonstrations
2. RM (Reward Model): Train a model to predict human preference scores
3. PPO (Reinforcement Learning): Optimize policy against the reward model with KL penalty

Why RLHF?

- Language modeling objective ($P(w_t|w_{<t})$) $\neq$ helpful/honest/harmless
- Human feedback directly optimizes for what we want

9–8 RAG: Retrieval-Augmented Generation

The Problem

LLMs have **knowledge cutoffs** and can **hallucinate** facts.

Solution

RAG retrieves relevant documents from a knowledge base before generating:

1. **Query** → Embed into vector
2. **Retrieve** → Find top-K relevant documents from knowledge base
3. **Augment** → Combine instruction + retrieved docs + original query into prompt
4. **Generate** → LLM produces the final answer

Benefits

Benefit	Description
Up-to-date knowledge	No retraining needed
Verifiable	Can check source documents
Reduces hallucinations	Grounded in retrieved text

9–9 Practical Model Quantization

```
# Using bitsandbytes for quantization
import torch
import transformers

model = transformers.AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-7b",
    load_in_4bit=True, # 4-bit quantization
    bnb_4bit_compute_dtype=torch.float16,
)

# Memory: ~4GB instead of ~14GB for FP16
```

9–10 Prompt Engineering

Key Techniques

Technique	Description	Example
Zero-shot	No examples	"Translate to French: Hello"
Few-shot	A few examples	"English: Hello → French: Bonjour"

Technique	Description	Example
Chain-of-thought	Step-by-step reasoning	"Let's think step by step..."
System prompt	Role setting	"You are a helpful assistant"

9–11 Evaluating LLMs

Metrics

Metric	What It Measures
Perplexity	How well the model predicts (lower is better)
BLEU	N-gram overlap with reference
ROUGE	Recall-oriented overlap
Human eval	Helpfulness, harmlessness

Supplementary Details

These subsections provide deeper mathematical background for the topics above.

9–1–2 N-gram vs Neural Language Models

Aspect	N-gram	Neural LM
Context	Fixed window (n-1)	Variable (theoretically unlimited)
Sparsity	Severe for large n	No sparsity (distributed representations)
Generalization	Cannot handle unseen n-grams	Can generalize to unseen patterns
Parameters	$O(V^n)$	$O(V \times d)$ (compact)
Performance	Saturates quickly	Improves with scale

9-1-3 Scaling Law ★

Scaling laws (Kaplan et al., 2020) show that Transformer performance follows power-law scaling with:

- **Model size** (parameters)
- **Dataset size** (tokens)
- **Compute** (FLOPs)

$$L(N, D) \approx \left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{D_c}{D} \right)^{\alpha_D}$$

Key insight: performance improves predictably with scale, with no signs of diminishing returns observed.

9-2-1 Autoregressive Decomposition

An autoregressive (AR) model decomposes the joint probability into a product of conditional probabilities:

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t | x_{<t})$$

This is the foundation of GPT-style models. At each step, the model predicts the next token given all previous tokens.

9-2-2 Training vs Inference

Training: Teacher forcing — ground truth tokens are fed as input (even if the model predicts wrong tokens).

Inference: Autoregressive generation — the model's own predictions are fed back as input for the next step.

This discrepancy means errors accumulate during generation — a key challenge for LLMs.

9-3-2 Top-K Sampling

At each step, sample only from the K most likely tokens:

$$P'(x_t | x_{<t}) = \begin{cases} \frac{P(x_t)}{\sum_{i=1}^K P(i)} & \text{if } \text{rank}(x_t) \leq K \\ 0 & \text{otherwise} \end{cases}$$

$K = 40$ is a common choice. Top-K keeps generation focused but can be too restrictive.

9-3-3 Top-P (Nucleus) Sampling

Sample from the smallest set of tokens whose cumulative probability exceeds threshold p :

$$\text{Select } V^{(p)} = \arg \min_{|V'|} \sum_{x \in V'} P(x|x_{<t}) \geq p$$

Unlike Top-K, the set size adapts dynamically. $p = 0.9$ is a common choice.

9-4-1 Pretraining

Pretraining is large-scale unsupervised learning on web text. The model learns:

- Language syntax and grammar
- World knowledge and facts
- Reasoning patterns

This stage is extremely compute-intensive (thousands of GPU-days).

9-4-2 Fine-tuning

Fine-tuning adapts a pretrained model to a specific task:

- **Full fine-tune**: Update all parameters (expensive)
- **LoRA**: Low-rank adaptation (efficient, $\approx 0.1\%$ of params)
- **Adapter**: Small task-specific modules inserted into the model

LoRA has become the standard for efficient fine-tuning.

9-8-1 Visualization: Multi-Head Attention Patterns NEW

💡 **Key Insight:** Multi-head attention is like having multiple experts working in parallel — each head focuses on different aspects: local context (nearby tokens), global context (distant tokens), or self-attention (each token on itself).

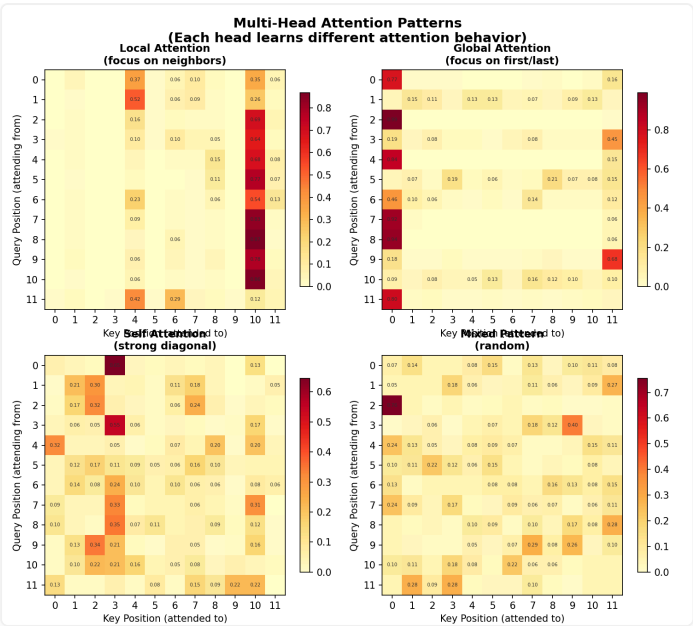


Figure 9-2: Multi-head attention patterns — X-axis is Key position (being attended to), Y-axis is Query position (attending from). Darker = higher attention weight. Different heads exhibit different patterns, demonstrating multi-head attention's multi-perspective capability.

Run `code/ch09/NN09_attention_pattern_viz.py` to reproduce:

```
python3 code/ch09/NN09_attention_pattern_viz.py
```

Head	Pattern	Characteristics
Head 0	Local attention	Focuses on nearby tokens, like CNN's local receptive field
Head 1	Global attention	Focuses on sequence start/end (critical positions)
Head 2	Self-attention	Strong diagonal — each token mainly attends to itself
Head 3	Mixed pattern	No strong preference, flexibly combines information

💡 **Core Insight:** The key advantage of multi-head attention is that different heads can automatically learn different attention patterns without manual design. Some heads capture local syntactic relationships; others capture global semantic relationships. This is why Transformer outperforms RNN: it captures both short and long-range dependencies in a single layer.



Chapter Code List

File	Content	Key Concept
<code>ch09/NN09_training_loop.py</code>	Language model training loop	Training pipeline
<code>ch09/NN09_autoregressive.py</code>	Autoregressive generation implementation	Generation core
<code>ch09/NN09_attention_is_all_you_need.py</code>	Attention mechanism complete implementation	Attention core
<code>ch09/NN09_attention_pattern_viz.py</code>	Multi-head attention pattern visualization	Attention visualization
<code>ch09/NN09_scaling_law.py</code>	Scaling Law visualization	Scaling analysis



Chapter Summary

LLM Technology Stack

1. **Pre-training:** Next-token prediction on massive corpus
2. **Fine-tuning:** SFT + RLHF for instruction following
3. **Inference:** Autoregressive generation with sampling strategies
4. **Optimization:** KV Cache ($O(T^2) \rightarrow O(T)$), quantization (FP32 $\rightarrow$ INT4), LoRA



Exercises

Exercise 1: Implement Autoregressive Generation

Write a simple autoregressive generation loop with greedy decoding.

Exercise 2: Compare Sampling Strategies

Generate text with greedy, top-k ($k=10$), and top-p ($p=0.9$). Compare diversity.

Exercise 3: Implement LoRA

Add LoRA to a linear layer and verify only the LoRA parameters are trainable.

Exercise 4: KV Cache

Implement generation with and without KV cache. Measure the speedup.

← [Chapter 8](#) | [Table of Contents](#) | [Appendix A](#) →

Appendix A: Mathematical Review

This appendix summarizes all mathematical formulas needed for this book. Organized into calculus, linear algebra, probability & statistics, and information theory.

> © xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

A-1 Calculus

A-1-1 Basic Derivatives

Function	Derivative	Notes
$f(x) = c$ (constant)	$f'(x) = 0$	Constant rule
$f(x) = x^n$	$f'(x) = nx^{n-1}$	Power rule
$f(x) = e^x$	$f'(x) = e^x$	Exponential
$f(x) = \ln x$	$f'(x) = 1/x$	Logarithm
$f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$	$f'(x) = \sigma(x)(1 - \sigma(x))$	Sigmoid
$f(x) = \tanh(x)$	$f'(x) = 1 - \tanh^2(x)$	Tanh
$f(x) = \text{ReLU}(x) = \max(0, x)$	$f'(x) = \mathbf{1}_{x>0}$	ReLU

A-1-2 Differentiation Rules

Rule	Formula
Sum rule	$(f + g)' = f' + g'$

Rule	Formula
Product rule	$(fg)' = f'g + fg'$
Quotient rule	$(f/g)' = (f'g - fg')/g^2$
Chain rule	$(f \circ g)' = f'(g(x)) \cdot g'(x)$

A-1-3 Partial Derivatives

For $f(x_1, x_2, \dots, x_n)$, the partial derivative $\frac{\partial f}{\partial x_i}$ measures the rate of change with respect to x_i alone.

A-2 Linear Algebra

A-2-1 Vectors

Operation	Definition
Dot product	$x \cdot y = \sum_i x_i y_i$
L2 norm	$ x _2 = \sqrt{\sum_i x_i^2}$
Cosine similarity	$\cos \theta = \frac{x \cdot y}{ x y }$

A-2-2 Matrices

Operation	Definition
Matrix multiplication	$(AB)_{ij} = \sum_k A_{ik} B_{kj}$
Transpose	$(A^T)_{ij} = A_{ji}$
Identity matrix	$I_{ij} = \delta_{ij}$

A-2-3 Key Neural Network Operations

Expression	Meaning
$z = Wx + b$	Single sample forward
$Z = XW^T + b$	Batch forward
$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)}$	Error backpropagation

A-3 Probability & Statistics

Concept	Formula
Probability	$P(A) \in [0, 1]$
Conditional prob.	$P(A B) = \frac{P(A \cap B)}{P(B)}$
Expectation	$\mathbb{E}[X] = \sum x P(X = x)$
Variance	$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$

A-4 Information Theory

Concept	Formula	Meaning
Self-information	$I(p) = -\log p$	Surprise of an event
Entropy	$H(p) = -\sum p_i \log p_i$	Average uncertainty
Cross-entropy	$H(p, q) = -\sum p_i \log q_i$	Prediction error
KL divergence	$D_{\text{KL}}(p q) = \sum p_i \log \frac{p_i}{q_i}$	Distribution distance

← [Chapter 9](#) | [Table of Contents](#) | [Appendix B](#) →

Appendix B: PyTorch API Reference

This appendix summarizes all PyTorch APIs used in this book, organized by functionality.

> © xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

B-1 Tensor Creation & Operations

B-1-1 Creating Tensors

API	Description	Example
<code>torch.tensor(data)</code>	From list/array	<code>torch.tensor([1, 2, 3])</code>
<code>torch.zeros(shape)</code>	All zeros	<code>torch.zeros(3, 4)</code>
<code>torch.ones(shape)</code>	All ones	<code>torch.ones(2, 3)</code>
<code>torch.randn(shape)</code>	Normal random	<code>torch.randn(128, 784)</code>
<code>torch.rand(shape)</code>	Uniform [0,1)	<code>torch.rand(3, 3)</code>
<code>torch.arange(start, end)</code>	Arithmetic sequence	<code>torch.arange(0, 10)</code>
<code>torch.linspace(s, e, n)</code>	Evenly spaced	<code>torch.linspace(0, 1, 100)</code>
<code>torch.eye(n)</code>	Identity matrix	<code>torch.eye(5)</code>

B-1-2 Tensor Properties

```
x = torch.randn(3, 4)
x.shape      # torch.Size([3, 4])
x.dtype      # torch.float32
x.device     # cpu or cuda:0
```

```
x.numel()      # 12 (total elements)
```

B-2 Autograd

```
# Enable gradient tracking
x = torch.tensor([1.0], requires_grad=True)

# Forward (graph built automatically)
y = x ** 2
z = y.mean()

# Backward (compute all gradients)
z.backward()
print(x.grad) # gradient dz/dx
```

B-3 Neural Network Modules

Module	Description
<code>nn.Linear(in, out)</code>	Fully connected layer
<code>nn.Conv2d(in, out, k)</code>	2D convolution
<code>nn.MaxPool2d(k)</code>	Max pooling
<code>nn.BatchNorm1d/2d</code>	Batch normalization
<code>nn.Dropout(p)</code>	Dropout regularization
<code>nn.ReLU()</code>	ReLU activation
<code>nn.Sigmoid()</code>	Sigmoid activation
<code>nn.Softmax(dim)</code>	Softmax function
<code>nn.Sequential()</code>	Layer container
<code>nn.Module</code>	Base class for all networks

B-4 Loss Functions

Loss	Description	Use Case
<code>nn.MSELoss()</code>	Mean squared error	Regression
<code>nn.CrossEntropyLoss()</code>	Cross-entropy	Classification
<code>nn.BCELoss()</code>	Binary cross-entropy	Binary classification

B-5 Optimizers

Optimizer	Usage
<code>optim.SGD(params, lr)</code>	SGD with optional momentum
<code>optim.Adam(params, lr)</code>	Adam optimizer
<code>optim.AdamW(params, lr, wd)</code>	Adam with decoupled weight decay

B-6 Training Loop Template

```

for epoch in range(num_epochs):
    for batch_x, batch_y in dataloader:
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

```

torchvision Utilities

API	Description
<code>datasets.MNIST(root, train)</code>	MNIST handwritten digits
<code>datasets.CIFAR10(root, train)</code>	CIFAR-10 color images

API	Description
<code>transforms.ToTensor()</code>	PIL Image -> Tensor conversion
<code>transforms.Normalize(mean, std)</code>	Standardization
<code>transforms.Compose(transforms)</code>	Chain multiple transforms

[← Appendix A](#) | [Table of Contents](#) | [Appendix C →](#)

Appendix C: Notation Conventions

This appendix defines all mathematical symbols used in this book. The same symbol may have different meanings in different contexts; see chapter-specific explanations.

> © xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

C-1 Basic Symbols

Symbol	Meaning	Example
x, y, z	Scalar (lowercase italic)	$x = 3.14$
$\mathbf{x}, \mathbf{w}$	Vector (lowercase bold)	$\mathbf{x} \in \mathbb{R}^d$
$\mathbf{W}, \mathbf{X}$	Matrix (uppercase bold)	$\mathbf{W} \in \mathbb{R}^{d \times h}$
$\mathbb{R}$	Set of real numbers	$\mathbb{R}^n$ denotes n -dim real space

C-2 Neural Network Specific Symbols

C-2-1 Network Structure

Symbol	Meaning
L	Number of layers
$n^{(l)}$	Number of neurons in layer l
m	Batch size (number of samples)

Symbol	Meaning
d	Input dimension

C-2-2 Layer Parameters

Symbol	Meaning	Shape
$W^{(l)}$	Weight matrix of layer l	$(n^{(l)}, n^{(l-1)})$
$b^{(l)}$	Bias vector of layer l	$(n^{(l)}, 1)$
$z^{(l)}$	Pre-activation (linear output)	$(n^{(l)}, 1)$
$a^{(l)}$	Post-activation (after nonlinearity)	$(n^{(l)}, 1)$

C-2-3 Learning

Symbol	Meaning
η	Learning rate
$\delta_j^{(l)}$	Error of neuron j in layer l
L or $\mathcal{L}$	Loss/cost function
∇L	Gradient of loss
$\odot$	Element-wise multiplication
$\frac{\partial L}{\partial W}$	Gradient of loss w.r.t. weights

Acronyms

Abbreviation	Full Name
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network

Abbreviation	Full Name
LSTM	Long Short-Term Memory
ResNet	Residual Network
Transformer	Attention-based architecture
LLM	Large Language Model
RLHF	Reinforcement Learning from Human Feedback
SGD	Stochastic Gradient Descent
Adam	Adaptive Moment Estimation
ReLU	Rectified Linear Unit
BCE	Binary Cross-Entropy
MSE	Mean Squared Error
BN	Batch Normalization
MLP	Multi-Layer Perceptron

← [Appendix B](#) | [Table of Contents](#) | [Appendix D](#) →

Appendix D: Common Functions Summary

This appendix summarizes common activation functions and loss functions used in neural networks.

> © xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

D-1 Activation Functions

D-1-1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Range: (0, 1)
- Derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- Use case: Binary classification output, gating mechanisms
- Limitation: Gradient vanishing in saturation regions

D-1-2 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Range: (-1, 1)
- Derivative: $\tanh'(x) = 1 - \tanh^2(x)$
- Use case: RNN hidden state, zero-centered activation needed
- Limitation: Still saturates

D-1-3 ReLU (Rectified Linear Unit)

$$\text{ReLU}(x) = \max(0, x)$$

- Range: $[0, +\infty)$
- Derivative: 1 if $x > 0$, 0 if $x \leq 0$
- Use case: **Default for hidden layers**
- Limitation: Dying ReLU (neurons can become permanently inactive)

D-1-4 Leaky ReLU

$$\text{LeakyReLU}(x) = \max(\alpha x, x)$$

- Typically $\alpha = 0.01$
- Solves the dying ReLU problem

D-1-5 Softmax

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- Output: Probability distribution over K classes
 - Use case: Multi-class classification output
-

D-2 Loss Functions

D-2-1 Mean Squared Error (MSE)

$$L_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^m (y_i - t_i)^2$$

- Use case: Regression
- Gradient: $\frac{\partial L}{\partial y} = \frac{2}{m} (y - t)$

D-2-2 Cross-Entropy

$$L_{\text{CE}} = - \sum_k t_k \log p_k$$

- Use case: Classification
- With Softmax: $\frac{\partial L}{\partial z_k} = p_k - t_k$ (nice gradient!)

D-2-3 Binary Cross-Entropy (BCE)

$$L_{\text{BCE}} = -\frac{1}{m} \sum_i (t_i \log p_i + (1 - t_i) \log(1 - p_i))$$

- Use case: Binary classification

D-3 Summary: When to Use What

Task	Output Activation	Loss Function
Binary classification	Sigmoid	BCE
Multi-class classification	Softmax	Cross-Entropy
Regression	None (linear)	MSE
Multi-label classification	Sigmoid	BCE (per label)

D-4 Regularization Methods

Method	Formula	Effect
L1 Regularization (Lasso)	$\lambda \sum w_i $	Produces sparse weights (some become zero)
L2 Regularization (Ridge)	$\frac{\lambda}{2} \sum w_i^2$	Weight decay (shrink but not zero)

Method	Formula	Effect
Dropout	Randomly drop $p\%$ of neurons during training	Ensemble effect, prevents overfitting
Batch Normalization	$\hat{x} = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$	Stabilizes training, accelerates convergence

← [Appendix C](#) | [Table of Contents](#) | [Appendix E](#) →

Appendix E: Exercise Answers

This appendix provides reference answers and explanations for chapter exercises.

> © xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

Chapter 1: The Idea of Neural Networks

Exercise 1: M-P Neuron for NAND Gate

Set threshold = 2, weights = [-1, -1]:

```
def mp_neuron(x1, x2, w1=-1, w2=-1, threshold=-1.5):
    """NAND gate: negated AND"""
    total = w1 * x1 + w2 * x2
    return 1 if total >= threshold else 0

# Test
for x1, x2 in [(0,0), (0,1), (1,0), (1,1)]:
    print(f"NAND({x1},{x2}) = {mp_neuron(x1, x2)}")
# Output: 1, 1, 1, 0
```

Exercise 4: Parameter Count

- Layer 1: $784 \times 256 + 256 = 200,960$
- Layer 2: $256 \times 10 + 10 = 2,570$
- Total: 203,530

If hidden = 512: $784 \times 512 + 512 = 401,920$; $512 \times 10 + 10 = 5,130$; Total = 407,050 (2×)

Chapter 2: Mathematical Foundations

Exercise 1: Dot Product

$$\mathbf{x} \cdot \mathbf{y} = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32$$

Exercise 3: Numerical Derivative

$$f(x) = x^3, f'(x) = 3x^2, f'(2) = 12$$

```
def numerical_derivative(f, x, h=1e-5):
    return (f(x+h) - f(x-h)) / (2*h)
print(numerical_derivative(lambda x: x**3, 2)) # ~12.0
```

Chapter 3: PyTorch Basics

Exercise 2: Autograd

```
x = torch.tensor([2.0], requires_grad=True)
y = x**2 + 3*x + 1
y.backward()
print(x.grad) # 2*2 + 3 = 7
```

Chapter 5: Backpropagation

Exercise 1: Manual Backpropagation Calculation

For $f(x) = \sigma(wx + b)$ at $x = 1, w = 0.5, b = 0$, compute $\frac{\partial f}{\partial w}$:

$$u = 0.5 \times 1 + 0 = 0.5$$

$$y = \sigma(0.5) = 0.6225$$

$$\frac{\partial y}{\partial u} = y(1 - y) = 0.6225 \times 0.3775 = 0.2350$$

$$\frac{\partial u}{\partial w} = x = 1$$

$$\frac{\partial f}{\partial w} = 0.2350 \times 1 = 0.2350$$

Explanation: Using the chain rule, $\frac{\partial f}{\partial w} = \frac{\partial f}{\partial u} \cdot \frac{\partial u}{\partial w} = \sigma'(u) \cdot x$.

Chapter 9: Large Language Models

Exercise 1: Temperature Parameter Experiment

- $T \rightarrow 0$: Approaches greedy decoding (highest probability token's probability $\rightarrow 1$), output is deterministic but repetitive.
- $T \rightarrow \infty$: Approaches uniform sampling, output is random but diverse.

Best practice: $T = 0.7 \sim 1.0$ balances creativity and coherence.

Analysis: Temperature controls the sharpness of the softmax distribution. Lower temperatures amplify probability differences; higher temperatures flatten them.

← [Appendix D](#) | [Table of Contents](#) | [Appendix F](#) →

Appendix F: Recommended Reading

This appendix lists core deep learning reference resources, organized by difficulty level.

> © xiefujin · Contact: 490021684@qq.com · Licensed under CC BY-NC-SA 4.0

F-1 Core Textbooks

Beginner

Book	Author	Why Read
<i>Dive into Deep Learning</i>	Zhang, Lipton, Li, Smola	Hands-on, free online
<i>Neural Networks and Deep Learning</i>	Michael Nielsen	Clear intuition, free online
<i>Deep Learning with Python</i>	François Chollet	Keras-focused, practical

Intermediate

Book	Author	Why Read
<i>Deep Learning</i> (Goodfellow et al.)	Goodfellow, Bengio, Courville	The "bible" of deep learning
<i>Pattern Recognition and Machine Learning</i>	Christopher Bishop	Mathematical depth

Advanced

Book	Author	Why Read
<i>Probabilistic Machine Learning</i>	Kevin Murphy	Comprehensive modern treatment
<i>The Elements of Statistical Learning</i>	Hastie, Tibshirani, Friedman	Classic statistical learning

F–2 Online Courses

Course	Platform	Focus
CS231n (Stanford)	YouTube	CNNs
CS224n (Stanford)	YouTube	NLP
Fast.ai	fast.ai	Practical deep learning
3Blue1Brown (Neural Networks)	YouTube	Visual intuition

F–3 Research Papers

Must–Read Classics

Paper	Year	Contribution
<i>ImageNet Classification with Deep CNNs</i> (AlexNet)	2012	Deep learning breakthrough
<i>Deep Residual Learning</i> (ResNet)	2015	Skip connections
<i>Attention Is All You Need</i> (Transformer)	2017	Attention mechanism
<i>BERT: Pre-training of Deep Bidirectional Transformers</i>	2018	Pre-training paradigm
<i>Language Models are Few-Shot Learners</i> (GPT–3)	2020	Scaling law

F-4 Tools & Frameworks

Tool	Purpose
PyTorch	Deep learning framework
HuggingFace Transformers	Pre-trained models
Weights & Biases	Experiment tracking
TensorBoard	Visualization

F-5 Online Resources

Official Framework Documentation

- **PyTorch Official Docs:** <https://pytorch.org/docs/stable/>
- **PyTorch Tutorials:** <https://pytorch.org/tutorials/>
- **Hugging Face Transformers:** <https://huggingface.co/docs/transformers/>

Interactive Learning

- **TensorFlow Playground:** <https://playground.tensorflow.org/> — Intuitive neural network experimentation
- **Distill.pub:** <https://distill.pub/> — Visualized ML research papers
- **3Blue1Brown Neural Networks Series:** YouTube — Best visual explanations

Blogs & Tutorials

Blog	URL	Focus
The Gradient	thegradientpub.com	Deep learning frontier insights
Lil'Log (Lilian Weng)	lilianweng.github.io	Systematic technical blog
Sebastian Ruder	ruder.io	NLP & optimizer deep analysis
Colah's Blog	colah.github.io	Visual math explanations

F-6 Development Tools

Tool	Purpose
PyTorch	Deep learning framework (used in this book)
Jupyter Notebook / JupyterLab	Interactive programming environment
TensorBoard / Weights & Biases	Experiment visualization & tracking
MLflow	ML lifecycle management

[← Appendix E](#) | [Table of Contents](#) |